MINISTRY OF EDUCATION AND TRAINING
CAN THO UNIVERSITY
COLLEGE OF INFORMATION AND
COMMUNICATION TECHNOLOGY
DEPARTMENT OF SOFTWARE ENGINEERING

GRADUATION THESIS
BACHELOR OF ENGINEERING IN
INFORMATION TECHNOLOGY

(HIGH-QUALITY PROGRAM)

BUILDING A FASHION E-COMMERCE APPLICATION WITH IMAGE-BASED PRODUCT SEARCH AND MODEL BENCHMARKING

Student: Nguyen Thanh Phat
Student ID: B2005853
Class: DI20V7F1 (K46)
Advisor: Dr. Tran Cong An

Can Tho, December 2025

MINISTRY OF EDUCATION AND TRAINING
CAN THO UNIVERSITY
COLLEGE OF INFORMATION AND COMMUNICATION TECHNOLOGY
DEPARTMENT OF SOFTWARE ENGINEERING

GRADUATION THESIS
BACHELOR OF ENGINEERING IN INFORMATION TECHNOLOGY

(HIGH-QUALITY PROGRAM)

BUILDING A FASHION E-COMMERCE APPLICATION WITH IMAGE-BASED PRODUCT SEARCH AND MODEL BENCHMARKING

Student: Nguyen Thanh Phat
Student ID: B2005853
Class: DI20V7F1 (K46)
Advisor: Dr. Tran Cong An

Can Tho, 01/2026

EVALUATION OF ADVISOR

.....

.....

.....

.....

.....

.....

.....

Advisor:

Dr. Tran Cong An

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to Dr. Tran Cong An, my thesis advisor, for his guidance, continuous feedback, and support throughout this research.

I also thank the members of the Thesis Defense Committee for their time, thorough evaluation, and constructive comments.

My sincere appreciation goes to the Faculty of Information Technology, Can Tho University, for providing the academic foundation that supported this work, and to the High-Quality Program for fostering the engineering discipline applied throughout this thesis.

Finally, I am deeply grateful to my family for their constant encouragement, patience, and unwavering support throughout my studies and the completion of this work.

Sincerely,

Can Tho, December 2025

Nguyen Thanh Phat

TABLE OF CONTENTS

EVALUATION OF ADVISOR ..... **III**
ACKNOWLEDGEMENTS ..... **IV**
TABLE OF CONTENTS ..... **V**
LIST OF FIGURES ..... **VIII**
LIST OF TABLES ..... **X**
LIST OF ABBREVIATIONS ..... **XIII**
ABSTRACT ..... **XIV**
TÓM TẮT ..... **XV**
PART 1: INTRODUCTION **2**
I. Context and Motivation 2
II. Problem Statement 2
III. Objectives 3
IV. Scope and Limitations 5
V. Research Methodology 6
VI. Thesis Outline 6
PART 2: THESIS CONTENT **8**
CHAPTER 1. BACKGROUND AND RELATED WORK **8**
1.1 Fashion E-Commerce 8
1.2 Content-Based Image Retrieval 8
1.2.1 Visual Search Concepts 9
1.2.2 The Semantic Gap 9
1.2.3 Mathematical Foundations of Embeddings 9
1.3 Machine Learning Models 10
1.3.1 Convolutional Neural Networks 11
1.3.2 Vision Transformers 14
1.3.3 CLIP and Fashion-CLIP 16
1.3.4 Model Selection and Justification 20
1.4 Vector Databases 22
1.4.1 The Search Challenge 22
1.4.2 Approximate Nearest Neighbour Search 22
1.4.3 HNSW: Hierarchical Navigable Small World 23
1.4.4 IVFFlat: Inverted File with Flat Compression 23
1.4.5 pgvector: Vector Search in PostgreSQL 24
1.4.6 Architectural Decision and Trade-offs 25
1.5 Platform Architecture and Technology Stack 25
1.5.1 Architectural Patterns 26
1.5.2 .NET Backend 27

1.5.3 Vue.js Frontend 28
1.5.4 PostgreSQL and pgvector 28
1.5.5 Redis Caching 29
1.5.6 Python ML Sidecar 29
1.5.7 .NET Aspire Orchestration 29
1.5.8 Hangfire Background Jobs 30
1.5.9 Identity, Authentication, and Authorisation 30
1.5.10 Benchmark Framework 31
1.5.11 Technology Stack Summary 32
1.6 Related Work and Research Gap 32
1.6.1 Academic Research 32
1.6.2 Commercial Systems 33
1.6.3 Contribution Differentiators 33
CHAPTER 2. DESIGN AND IMPLEMENTATION 35
2.1 Requirements Specification 35
2.1.1 Functional Requirements 35
2.1.2 Non-Functional Requirements 43
2.1.3 Feature Classification 45
2.2 System Modeling 46
2.2.1 System Actors 47
2.2.2 Functional Decomposition 48
2.2.3 Use Case Summary Matrix 48
2.2.4 Administrator Use Cases 50
2.2.5 Customer Use Cases 73
2.2.6 System Use Cases 87
2.3 System Architecture & Design 91
2.3.1 System Overview 92
2.3.2 Domain-Driven Design 93
2.3.3 C4 Architecture 99
2.3.4 Database Design 104
2.3.5 API Design 107
2.3.6 Security Design 110
2.3.7 Chapter Summary 111
2.4 Implementation 112
2.4.1 Technology Stack 112
2.4.2 Vertical Slice Architecture Core 114
2.4.3 Data Persistence Architecture 118
2.4.4 ML Sidecar and CBIR Search 120
2.4.5 Frontend Applications 125
CHAPTER 3. TESTING AND EVALUATION 136
3.1 Testing Strategy 136
3.1.1 Unit Testing 136
3.1.2 Integration Testing 136

3.1.3 End-to-End Testing 137
3.2 Benchmark Protocol 137
3.2.1 Dataset 137
3.2.2 Models Evaluated 138
3.2.3 Metrics 139
3.2.4 Methodology 140
3.2.5 Hardware Environment 141
3.3 Retrieval Performance 141
3.4 Efficiency Metrics 143
3.5 Model Comparison & Discussion 145
3.5.1 Accuracy-Efficiency Trade-off 145
3.5.2 Deployment Recommendations 146
3.5.3 Discussion of Limitations 147
3.5.4 Lessons Learned 148
PART 3: CONCLUSION AND FUTURE WORK 150
CHAPTER 5. CONCLUSION & FUTURE WORK 150
5.1 Summary of Work 150
5.1.1 Answering the Research Questions 151
5.1.2 Achievement of Technical Objectives 152
5.2 Contributions 153
5.3 Limitations 154
5.4 Future Work 155
5.5 Requirements Traceability 157
REFERENCES 160
APPENDIX A. APPENDIX A, FULL BENCHMARK RESULTS 164
A.1 A.1 Category-Only Ground Truth (5 Categories) 164
A.2 A.2 Category + Colour Ground Truth 164
A.3 A.3 Category + Colour + Pattern Ground Truth 165
A.4 A.4 Operational Efficiency (3-Fold CV) 166
A.5 A.5 Per-Fold Variability 167
APPENDIX B. APPENDIX B, DATASET COMPOSITION 167
B.1 B.1 Source and Selection 167
B.2 B.2 Category Distribution 168
B.3 B.3 Ground-Truth Label Schemes 168
B.4 B.4 Image Preprocessing 169
APPENDIX C. APPENDIX C, HARDWARE SPECIFICATIONS 169
C.1 C.1 Compute Hardware 169
C.2 C.2 Software Stack 170
C.3 C.3 Precision and Determinism Settings 170
C.4 C.4 Reproducibility Notes 171

LIST OF FIGURES

Figure 1 ResNet architecture with residual skip connections enabling gradient flow across very deep networks 12
Figure 2 EfficientNet-B0 architecture showing the flow from input image to feature vector 13
Figure 3 DINOv2 architecture: image patches pass through transformer layers with self-attention to produce a feature vector 15
Figure 4 CLIP ViT-B/16 dual-tower architecture: images and text are converted to vectors in the same embedding space, allowing direct comparison 17
Figure 5 Fashion-CLIP dual-tower architecture: image and text towers independently encode their inputs into 512-dimensional embeddings converging in a shared latent space 19
Figure 6 Functional decomposition of ReSys.Shop using a Work Breakdown Structure (WBS), showing the hierarchical breakdown into three functional areas and their constituent modules and sub-functions. 48
Figure 7 Use case diagram for Product Management (UC-ADM-PROD). . 51
Figure 8 Use case diagram for Variant and Pricing (UC-ADM-VAR). 52
Figure 9 Use case diagram for Image and Embedding Management (UC-ADM-IMG). 54
Figure 10 Use case diagram for Taxonomy and Classification (UC-ADM-TAX). 55
Figure 11 Use case diagram for Option Type Configuration (UC-ADM-OPT). 57
Figure 12 Use case diagram for Order Lifecycle (UC-ADM-ORD, UC-ADM-ORD-ITEMS). 58
Figure 13 Use case diagram for Payment Processing (UC-ADM-PAY). 61
Figure 14 Use case diagram for Payment Method Configuration (UC-ADM-PAY-METHOD). 62
Figure 15 Use case diagram for Stock Location Management (UC-ADM-LOC). 63
Figure 16 Use case diagram for Stock Item Management (UC-ADM-STK). 65
Figure 17 Use case diagram for User Management (UC-ADM-USR). 67
Figure 18 Use case diagram for Role and Permission Governance (UC-ADM-ROL). 68
Figure 19 Use case diagram for Shipping Method Configuration (UC-ADM-SHP). 70
Figure 20 Use case diagram for Reference Data Management (UC-ADM-REF). 71

Figure 21 Use case diagram for Catalog Browsing (UC-STR-BRW). 73
Figure 22 Use case diagram for Search (UC-STR-SRC). 75
Figure 23 Use case diagram for Cart Management (UC-STR-CRT). 76
Figure 24 Use case diagram for Checkout Flow (UC-STR-CHK). 78
Figure 25 Use case diagram for Order History (UC-STR-OHI). 80
Figure 26 Use case diagram for Payment Processing (UC-STR-PAY). 81
Figure 27 Use case diagram for Authentication (UC-STR-AUT). 82
Figure 28 Use case diagram for Session Management (UC-STR-SES). 84
Figure 29 Use case diagram for Profile and Preferences (UC-STR-PRF). . . 85
Figure 30 Use case diagram for Embedding Operations (UC-SYS-EMB). . 87
Figure 31 ML embedding pipeline: step-by-step flow from image upload to normalised vector response. 89
Figure 32 Use case diagram for Background Maintenance (UC-SYS-MNT). 89
Figure 33 Bounded Context Map showing the eight business contexts and the Published Language identifiers exchanged between them. All integration uses in-process MediatR dispatch; no context directly references another context's namespace. 94
Figure 34 Order checkout state machine showing forward-only stages and cancellation paths. 98
Figure 35 Payment intent lifecycle and terminal states. 99
Figure 36 System Context diagram: ReSys.Shop system boundary showing user roles and external integration dependencies. 100
Figure 37 Container diagram showing Vue 3 SPAs, .NET 10 API backend, Python ML sidecar, PostgreSQL with pgvector, and Redis. 101
Figure 38 Component diagram detailing the API Backend architecture and the three-layer Python ML sidecar. 102
Figure 39 Deployment topology showing containerized orchestration under .NET Aspire with horizontal scaling. 103
Figure 40 Core entity-relationship model across eight bounded contexts. . 105
Figure 41 CBIR search sequence: end-to-end flow spanning customer upload, embedding extraction, pgvector search, and ranked results rendering. 124

LIST OF TABLES

Table 1 What different CNN layers learn to detect in fashion images 11
Table 2 CNN-based models evaluated 13
Table 3 DINOv2 model specifications evaluated in this project 15
Table 4 CLIP variants evaluated 19
Table 5 Candidate embedding models evaluated for fashion product retrieval. All models are pre-trained and publicly available. 20
Table 6 Comparison of pgvector with specialised vector databases 25
Table 7 Technology stack of the ReSys.Shop platform 32
Table 8 Comparison of commercial visual search products 33
Table 9 Catalog module functional requirements. 36
Table 10 Identity module functional requirements. 37
Table 11 Inventory module functional requirements. 39
Table 12 Ordering module functional requirements. 40
Table 13 Payment module functional requirements. 41
Table 14 Shipping module functional requirements. 42
Table 15 Profile module functional requirements. 42
Table 16 Location module functional requirements. 43
Table 17 Dashboard module functional requirements. 43
Table 18 Performance latency targets for CBIR search and standard API endpoints. 44
Table 19 Security requirements for authentication, authorization, rate limiting, and transport safety. 44
Table 20 Modularity requirements governing architectural isolation and communication boundaries. 44
Table 21 Observability requirements for tracing, correlation logging, and health monitoring. 45
Table 22 Reliability constraints covering background durability, idempotency, and state timeouts. 45
Table 23 Classification of system feature areas into Core Research contributions and Supporting Infrastructure components. 46
Table 24 Use case summary matrix. All use cases are listed with their actor, associated business module, and traceability to functional requirements defined in Section 2.1. 48
Table 25 Manage Products. 51
Table 26 Manage Variants. 53
Table 27 Manage Images and Embeddings. 54
Table 28 Manage Taxonomies and Classification. 55
Table 29 Manage Option Types. 57
Table 30 Manage Orders. 58
Table 31 Manage Order Details. 59

Table 32 Manage Payments. 61
Table 33 Manage Payment Methods. 63
Table 34 Manage Stock Locations. 64
Table 35 Manage Stock. 65
Table 36 Manage Users. 67
Table 37 Manage Roles and Permissions. 68
Table 38 Manage Shipping. 70
Table 39 Manage Reference Data. 71
Table 40 Browse and Search Catalog. 73
Table 41 Visual Search. 75
Table 42 Manage Cart. 77
Table 43 Checkout. 78
Table 44 Order History. 80
Table 45 Payment Processing. 81
Table 46 Authentication. 83
Table 47 Session Management. 84
Table 48 Profile Management. 86
Table 49 Embedding Operations. 87
Table 50 System Maintenance. 89
Table 51 System services and their technology stacks. Each service communicates through well-defined HTTP contracts. 92
Table 52 Bounded contexts with aggregate roots and key domain entities. Each context owns its database schema and communicates through MediatR dispatch only. 92
Table 53 Bounded context responsibilities and Published Language identifiers. The Published Language column lists the value types that other contexts may reference by identifier only, never by importing the source context's namespace. 94
Table 54 Ubiquitous Language Glossary across domain contexts. 96
Table 55 ReSys.Shop API endpoint contract. The platform exposes 257 Carter endpoints across nine business modules (184 admin, 73 storefront). Admin routes provide full CRUD with activate/deactivate and sync/assign/revoke patterns. Storefront routes expose read-optimised public surfaces for product discovery, cart, checkout, and account management. 109
Table 56 Principal technologies grouped by architectural role with pinned version specifications. 112
Table 57 Sequential execution flow within the CreateProduct command handler. 114
Table 58 MediatR pipeline behavior execution sequence and responsibilities. 117
Table 59 Schema-per-context mapping. Cross-schema references use UUID attributes without database-level foreign key constraints. 118

Table 60 Comparison of pgvector ANN index algorithms using the cosine distance operator (<=>). 119
Table 61 Concurrency control mechanisms across system domains. 120
Table 62 Registered embedding models with output dimensionality and architectural family. 121
Table 63 Embedding generation pipeline stages. 122
Table 64 ML sidecar API endpoints. All inference endpoints require `X-API-Key` header authentication. 123
Table 65 Visual search UI state model. 127
Table 66 Embedding models supported by the benchmark framework, grouped by architecture family. Four representative models were evaluated in the thesis. All models use pre-trained weights from public model repositories. 138
Table 67 Hardware environment for benchmark evaluation. 141
Table 68 Aggregate Retrieval Metrics, 3-Fold Cross-Validation 141
Table 69 Efficiency Metrics, 3-Fold Cross-Validation 143
Table 70 Combined Accuracy and Efficiency Comparison 145
Table 71 Requirements traceability: mapping from Chapter 1 objectives and research questions to the chapters where they are addressed, with the key finding that confirms each objective was met. 157
Table 72 Retrieval Accuracy, Category-Only Ground Truth (3-Fold CV, Mean ± SD) 164
Table 73 Retrieval Accuracy, Category + Colour Ground Truth (3-Fold CV, Mean ± SD) 165
Table 74 Retrieval Accuracy, Category + Colour + Pattern Ground Truth (3-Fold CV, Mean ± SD) 165
Table 75 Operational Efficiency, All Models (3-Fold CV, Mean ± SD) . . . 166
Table 76 Per-Fold Breakdown, Category + Colour Ground Truth 167
Table 77 Dataset Category Distribution 168
Table 78 Benchmark Workstation Configuration 169
Table 79 Benchmark Software Environment 170

LIST OF ABBREVIATIONS

Abbreviation	Description
API	Application Programming Interface
ANN	Approximate Nearest Neighbor
CBIR	Content-Based Image Retrieval
CNN	Convolutional Neural Network
CQRS	Command Query Responsibility Segregation
DDD	Domain-Driven Design
DSR	Design Science Research
EF Core	Entity Framework Core
HNSW	Hierarchical Navigable Small World
JWT	JSON Web Token
mAP	Mean Average Precision
P@K	Precision at K
R@K	Recall at K
REST	Representational State Transfer
SPA	Single Page Application
ViT	Vision Transformer
VSA	Vertical Slice Architecture

ABSTRACT

E-commerce platforms rely primarily on text-based search, yet fashion products are inherently visual. Patterns, silhouettes, and textures resist keyword description. This thesis presents a fashion e-commerce platform with integrated Content-Based Image Retrieval (CBIR) that enables customers to search for products by uploading images rather than typing keywords. The system implements a modular architecture with a .NET backend, Vue.js frontend, and a Python machine learning sidecar for embedding generation.

A systematic benchmark evaluates four pre-trained deep learning models across both retrieval accuracy and operational efficiency on a curated fashion product dataset. Fashion-CLIP, a CLIP variant fine-tuned on over 700,000 fashion images, achieved the highest mean Average Precision at 0.8788 (SD 0.0022), outperforming the generic CLIP (mAP 0.8341, SD 0.0043) by 5.4%, EfficientNet-B0 (mAP 0.8158, SD 0.0007) by 7.7%, and ResNet-50 (mAP 0.8120, SD 0.0052) by 8.2%. At the opposite end of the speed spectrum, EfficientNet-B0 delivered inference at 23.9 ms per image (SD 2.5) while maintaining competitive retrieval quality, representing a 3.8x speed advantage over Fashion-CLIP (92.0 ms, SD 5.8) for a 7.7% accuracy trade-off.

Results demonstrate that domain-specific pre-training provides measurable advantages for visual fashion retrieval. The pluggable model architecture, switchable via a single environment variable, allows production deployments to select the optimal accuracy-speed profile for their infrastructure. The thesis provides a reference implementation for systematic comparison of embedding models in the fashion domain.

Keywords: e-commerce, visual search, deep learning, modular architecture, computer vision, benchmarking

TÓM TẮT

Các sàn thương mại điện tử phụ thuộc nhiều vào tìm kiếm văn bản, một cơ chế kém hiệu quả trong lĩnh vực thời trang, nơi hoa văn, chất liệu và kiểu dáng khó diễn đạt chính xác qua từ khóa. Hệ thống được xây dựng theo hướng module, bao gồm ba thành phần chính: backend .NET xử lý logic nghiệp vụ, frontend Vue.js phục vụ giao diện người dùng, và dịch vụ Python chuyên biệt cho tác vụ trích xuất đặc trưng hình ảnh từ các mô hình học sâu tiền huấn luyện.

Thực nghiệm được thiết lập trên bộ dữ liệu ảnh sản phẩm thời trang với quy trình đánh giá chéo ba lần (3-fold cross-validation), so sánh bốn mô hình embedding trên hai chiều: chất lượng truy xuất (đo bằng mAP, Precision at K, Recall at K) và hiệu năng vận hành (đo bằng độ trễ suy luận, thông lượng, và dung lượng lưu trữ). Mô hình Fashion-CLIP, được tinh chỉnh trên tập dữ liệu hơn 700.000 ảnh thời trang, đạt mAP cao nhất 0,8788 với độ lệch chuẩn 0,0022. CLIP gốc đạt mAP 0,8341 (SD 0,0043) và EfficientNet-B0 đạt 0,8158 (SD 0,0007). Ở chiều ngược lại, mô hình EfficientNet-B0 cho tốc độ suy luận nhanh nhất, 23,9 ms mỗi ảnh (SD 2,5), nhanh gấp 3,8 lần Fashion-CLIP (92,0 ms, SD 5,8) trong khi chỉ đánh đổi 7,7% về chất lượng truy xuất.

Kết quả thực nghiệm khẳng định giá trị của huấn luyện chuyên biệt theo lĩnh vực trong bài toán truy xuất hình ảnh thời trang, đồng thời cung cấp dữ liệu định lượng cho việc lựa chọn mô hình dựa trên ràng buộc hạ tầng thực tế. Kiến trúc mô hình hoán đổi linh hoạt, được điều khiển qua biến môi trường, cho phép hệ thống thích ứng với nhiều cấu hình triển khai khác nhau mà không cần thay đổi mã nguồn.

Từ khóa: thương mại điện tử, tìm kiếm hình ảnh, học sâu, kiến trúc module, thị giác máy tính, đánh giá hiệu năng

PART 1: INTRODUCTION

I. CONTEXT AND MOTIVATION

Global fashion e-commerce revenue exceeded **770 billion USD** in 2024, with projections surpassing **one trillion by 2030** [1]. Yet its dominant interface, keyword search, fails where the domain succeeds: fashion products are defined by silhouette, drape, print density, and colour relationships – attributes that resist textual description. This **semantic gap**, the discrepancy between a garment's visual richness and a user's ability to express it in words, is well documented. A customer can recognise a desired aesthetic instantly from a photograph yet cannot produce query terms that retrieve it. When catalogue indexing uses inconsistent terminology (one vendor tags a pattern as "floral," another as "botanical," a third as "flower print"), relevant products are systematically excluded. Industry estimates place the **session abandonment rate** after an unsuccessful search at approximately **30 percent** [2].

Content-Based Image Retrieval (CBIR) addresses this gap by replacing textual intermediaries with direct visual comparison. Products are indexed not by human-authored labels but by **dense vector embeddings** computed from images, with similarity measured through mathematical distance functions. A query image of a dress with a particular neckline and print pattern retrieves visually similar products without any keyword translation step. Pre-trained convolutional neural networks [3], [4], vision transformers [5], and fashion-specific models [6] have substantially advanced this capability.

The contribution of this work is **architectural**, not algorithmic. It investigates how to embed existing CBIR capabilities into a practical e-commerce system built with conventional web technologies, and provides empirical data on which embedding models deliver the optimal balance of accuracy, latency, and resource efficiency. The work bridges two distinct software ecosystems, the **Python machine learning stack** and the **.NET enterprise web stack**, under real-time latency constraints appropriate for interactive search.

II. PROBLEM STATEMENT

Keyword-reliant fashion search suffers from four compounding inefficiencies.

Catalogue vocabulary mismatch. Descriptors vary across vendors, such that a single visual pattern appears under multiple labels and fragments result sets. Users must reformulate queries iteratively as the gap between catalogue

indexing vocabulary and customer search vocabulary silently excludes relevant products.

Visual inexpressibility. Attributes such as fabric drape, texture gradient, silhouette proportion, and pattern rhythm cannot be captured reliably through text queries. A customer identifies a desired aesthetic instantly in a photograph yet cannot produce keywords that retrieve it. The search engine cannot match what the user cannot name.

Cold-start invisibility. Recommendation models based on collaborative filtering depend on historical user-item interactions. Newly listed products lack this data at their point of highest commercial value: initial release. Visual feature extraction bypasses this limitation, as product images are available from catalogue ingestion and embeddings enable similarity-based discovery without interaction history.

Polyglot integration cost. Integrating pre-trained vision models into a .NET transactional backend introduces a recurring engineering challenge in applied machine learning. The Python deep learning ecosystem (PyTorch, HuggingFace) does not natively interoperate with the .NET enterprise stack. Achieving **sub-second response latency** across this boundary requires architectural design that isolates the ML workload from the main application while bridging incompatible package managers, runtime environments, and deployment conventions.

III. OBJECTIVES

This project builds a functional fashion e-commerce platform with integrated image-based search and evaluates the effectiveness of pre-trained deep learning models within that system. The contribution is not a novel AI architecture but the **engineering demonstration** of embedding existing models into a conventional web application stack.

Technical Objectives

- **Model integration.** Integrate pre-trained vision models into a PostgreSQL and .NET e-commerce stack, establishing a reference pattern for teams with existing web infrastructure.
- **Polyglot architecture.** Architect a polyglot system in which a dedicated Python sidecar handles AI inference while the .NET backend manages transactional logic, business rules, and API routing.
- **Vector storage validation.** Validate **pgvector** (an open-source PostgreSQL extension) as the sole vector storage and retrieval layer, evaluating whether it meets real-time search latency requirements at catalogue scales representative of small-to-medium fashion retailers.

- **Empirical benchmarking.** Benchmark multiple embedding models spanning convolutional and transformer architectures on shared hardware, producing empirical guidance for model selection in resource-constrained deployments.

Research Questions

Three questions guide the investigation and are answered empirically in Chapter 3.

RQ1: Model comparison. How do fashion-specific embedding models compare with general-purpose CNN and ViT architectures on fashion product retrieval? This question tests whether domain-specific training on fashion data yields measurable improvements over models trained on generic image corpora.

RQ2: Accuracy-speed trade-off. What trade-offs exist between retrieval accuracy and inference latency across pre-trained embedding models, and which model offers the best balance for real-time search? The most accurate model is rarely the fastest; deployment decisions require weighing both dimensions.

RQ3: Architecture viability. Can a service-oriented architecture with a dedicated AI sidecar separate image inference from the main application while maintaining interactive response times? This question evaluates whether the chosen polyglot pattern (Python ML service alongside a .NET application) is practical for production use.

Tasks Completed

- **Build an AI service.** A Python FastAPI service that loads multiple pre-trained embedding models and generates feature vectors from product images within interactive latency bounds.
- **Set up vector search.** PostgreSQL configured with pgvector to store and index high-dimensional embeddings, with similarity queries validated for correctness and performance on a catalogue of representative size.
- **Connect the services.** A .NET backend layer that orchestrates image upload, embedding generation, vector database query, and result assembly into a single end-to-end search flow.
- **Create the user interface.** A Vue.js storefront with drag-and-drop image upload and a results grid displaying visually similar products with similarity scores.
- **Evaluate the results.** A systematic benchmark measuring retrieval accuracy via standard information retrieval metrics, comparing inference speed across models, and analyzing operational trade-offs that inform production model selection.

IV. SCOPE AND LIMITATIONS

The thesis encompasses the design, implementation, and empirical evaluation of a fashion e-commerce platform with integrated visual search. Five areas define the included scope:

- Visual search from the storefront, with image upload via drag-and-drop or file selection.
- Product recommendations derived from embedding similarity scores.
- Core e-commerce functionality: product catalogue browsing, shopping cart, and simulated checkout.
- Multi-model comparison spanning several CNN and transformer architectures.
- End-to-end latency, throughput, and storage footprint measurement.

The following areas are explicitly excluded:

- Real payment processing (transactions are simulated for demonstration purposes).
- Shipping, logistics, and warehouse management workflows.
- User authentication via social login providers.
- Mobile application development (the system targets desktop and tablet web browsers).
- Custom model training or fine-tuning (all models are used as published).

Known Limitations

Four limitations define the boundaries of this work and are revisited in the concluding chapter.

- **Dataset size.** Evaluation uses 5,000 fashion product images from the Fashion Product Images dataset [7]. Controlled comparative benchmarking is feasible at this scale, but results may not extrapolate to production catalogues containing millions of items.
- **Hardware.** Experiments ran on consumer-grade hardware (Intel i7-1165G7, 16 GB RAM) with all inference executed on CPU. Reported latency and throughput figures are relative to this CPU-only profile. A dedicated inference server with GPU acceleration would likely improve both metrics.
- **Evaluation method.** Evaluation is exclusively quantitative: retrieval accuracy, inference latency, and throughput. Search output was reviewed qualitatively through visual inspection, but no formal user experience study was conducted. The relationship between measured accuracy metrics and subjective user satisfaction remains an open question.
- **Model training.** All embedding models were used as published by their original authors, without fine-tuning on the evaluation dataset. Domain-specific

fine-tuning, particularly for models pre-trained on generic image corpora rather than fashion data, might improve retrieval quality but was beyond scope.

V. RESEARCH METHODOLOGY

This thesis follows a **Design Science Research (DSR)** methodology [8], [9], a problem-solving paradigm that produces and evaluates an IT artifact (here, the e-commerce platform with integrated visual search) to address a defined problem domain.

The project progressed through four phases:

- **Research and planning.** Literature survey on visual search in fashion and e-commerce architectures; exploration of available pre-trained embedding models; evaluation of vector database options; technology stack selection.
- **Design.** Formalization of a modular .NET monolith architecture with a Python AI sidecar communicating via HTTP; database schema design supporting both relational and vector data within a single PostgreSQL instance.
- **Implementation.** Construction of three principal components: the .NET backend following vertical slice architecture, the Python embedding service with lazy model loading, and the Vue.js storefront.
- **Test and evaluation.** Systematic benchmark measuring retrieval accuracy through standard information retrieval metrics; latency and throughput measurement on consumer-grade hardware; qualitative review of search output.

This methodology suits the project's engineering focus: the primary output is not a theoretical contribution but a working system accompanied by empirical data on performance trade-offs practitioners encounter when integrating academic models into production web stacks.

VI. THESIS OUTLINE

The thesis is organized into five chapters across three parts.

Part I: Introduction. Chapter 1 establishes research context, defines the problem, states objectives and research questions, delineates scope and limitations, describes the methodology, and provides the present outline.

Part II: Thesis Content contains three chapters:

- **Chapter 1: Background and Related Work.** Surveys vector embeddings, convolutional and transformer-based neural architectures, vector database technologies including pgvector, prior work in fashion image retrieval, and the technology stack selected for this project.
- **Chapter 2: Design and Implementation.** Translates the problem into functional and non-functional requirements (Section 2.1), presents the system

architecture including domain-driven design, C4 diagrams, database design, API design, and security model (Section 2.2), and describes the concrete implementation of the .NET backend, Python ML sidecar, and Vue.js storefront (Section 2.3).

- **Chapter 3: Testing and Evaluation.** Reports a systematic benchmark comparing retrieval accuracy and inference efficiency across multiple embedding models using a cross-validation protocol on 5,000 fashion product images, and analyzes the accuracy-speed trade-offs that inform model selection.

Part III: Conclusion. Chapter 4 synthesizes findings against each research objective, evaluates contributions and limitations, and proposes directions for future work.

PART 2: THESIS CONTENT

1 BACKGROUND AND RELATED WORK

This chapter surveys the theoretical foundations and technical prerequisites for the project.

- **Fashion E-commerce.** Market context, semantic gap, business case.
- **Content-Based Image Retrieval.** Embeddings, cosine similarity, CBIR pipeline.
- **Machine Learning Models.** CNN, ViT, CLIP, Fashion-CLIP architectures.
- **Vector Databases.** ANN search, HNSW, IVFFlat, pgvector.
- **Platform Architecture and Technology Stack.** Modular monolith, vertical slice, .NET, Vue, PostgreSQL, Redis, Python sidecar, orchestration, auth, benchmarks.
- **Related Work and Research Gap.** Academic research, commercial systems, contribution differentiators.

1.1 FASHION E-COMMERCE

Global fashion e-commerce reached **770 billion USD** in 2024 and is projected to surpass **one trillion USD by 2030** [1]. However, traditional text search bars cause a **30 percent search abandonment rate** because keyword queries struggle to match visual intent [2].

Fashion is fundamentally visual. Attributes like silhouette, drape, color harmony, and pattern density do not translate easily into search terms. A customer can easily recognize a garment in an image but often fails to find it using keywords.

Major platforms like ASOS, Zalando, and Shopee invest in visual search because millions of catalog items rely on inconsistent vendor metadata. Identical patterns often carry different labels across suppliers, hiding relevant products from keyword search. Visual search solves this metadata problem, directly preventing lost revenue from search abandonment.

1.2 CONTENT-BASED IMAGE RETRIEVAL

Content-Based Image Retrieval replaces text queries with image queries. Rather than indexing products by human-authored labels, CBIR systems encode images into dense vector representations and measure similarity through mathematical

distance functions. This section introduces the core concepts and mathematical foundations of CBIR as applied to fashion product search.

1.2.1 Visual Search Concepts

Content-Based Image Retrieval (CBIR) replaces text queries with image queries. Rather than requiring users to describe what they want in words, CBIR lets them search by example.

The system encodes a query image into a **dense vector embedding**: a fixed-length sequence of numbers that captures shape, texture, colour, and pattern. It then retrieves catalog items whose embeddings are nearest in vector space. Visually similar products produce similar vectors; dissimilar products produce distant ones.

This approach bypasses the need for consistent textual labels. A photograph of a dress with a distinctive neckline retrieves visually similar products regardless of how the catalog describes them.

1.2.2 The Semantic Gap

The semantic gap, introduced in Section 1.1, is the discrepancy between the visual richness of a garment and a user's ability to express that richness in keywords. CBIR bridges this gap by operating directly on visual content rather than on human-authored metadata. The embedding serves as a universal descriptor that captures attributes such as fabric texture, silhouette proportion, and colour gradients automatically, without any keyword translation step.

1.2.3 Mathematical Foundations of Embeddings

An embedding model processes an input image and transforms its visual content into a fixed-length numerical array:

$$\boldsymbol{e} = [e_1, e_2, e_3, ..., e_d]^T \in \mathbb{R}^d$$

For a model with a 512-dimensional output, this vector $\boldsymbol{e}$ contains 512 continuous numbers. These arrays are called **vector embeddings**. Visually similar items yield nearby numerical values across these dimensions, converting visual comparison into a standard mathematical distance calculation.

1.2.3.1 The Latent Space

Embeddings act as coordinates within a high-dimensional continuous space known as the **latent space**. A 512-dimensional vector occupies a coordinate system with 512 orthogonal axes.

Within this space:

- Visually and semantically similar items sit close to one another.
- Dissimilar items lie further apart.
- The term "latent" indicates that individual axes rarely map directly to single human visual labels like "stripes" or "red." Instead, each dimension captures abstract, non-linear feature combinations learned by the model during training.

1.2.3.2 Measuring Similarity: Cosine Similarity

To evaluate how closely two images match, the system measures the directional angle between their corresponding embedding vectors $\boldsymbol{A}$ and $\boldsymbol{B}$ using **cosine similarity**:

$$\text{Cosine Similarity}(\boldsymbol{A}, \boldsymbol{B}) = \frac{\boldsymbol{A} \cdot \boldsymbol{B}}{\|\boldsymbol{A}\|_2 \times \|\boldsymbol{B}\|_2}$$

Where $\boldsymbol{A} \cdot \boldsymbol{B}$ represents the vector dot product, and $\|\boldsymbol{A}\|_2$ and $\|\boldsymbol{B}\|_2$ denote the Euclidean norms (L_2 norms) of each vector.

Cosine similarity produces values ranging from $+1.0$ (identical vector orientation) to 0.0 (orthogonal vectors) down to -1.0 (opposite orientations). For normalized fashion embeddings, scores above 0.70 generally correspond to strong visual similarity perceptible to human shoppers.

1.2.3.3 The CBIR Pipeline

A standard Content-Based Image Retrieval system operates through four core sequential stages:

1. **Image Input:** The user uploads or selects a target query photograph.
2. **Embedding Generation:** A deep learning model processes the image to output its feature vector.
3. **Vector Comparison:** The database compares the query vector against pre-indexed catalog vectors using cosine distance or cosine similarity.
4. **Ranking & Retrieval:** The system orders products by their similarity scores and renders the top nearest neighbors to the user.

1.3 MACHINE LEARNING MODELS

This section surveys the three families of architectures used for visual feature extraction: convolutional neural networks, vision transformers, and multimodal CLIP-based models. Each subsection explains how the architecture works, introduces the specific variants evaluated, and identifies their trade-offs for fashion retrieval. The section concludes with the model selection decision.

1.3.1 Convolutional Neural Networks

Convolutional neural networks (CNNs) have been the dominant architecture for computer vision. This section explains how CNNs extract features and introduces the specific variants evaluated: ResNet and EfficientNet.

1.3.1.1 Hierarchical Feature Extraction

A CNN processes an image through a stack of learned filters. Each filter is a small window (typically 3 by 3 pixels) that slides across the image detecting local patterns [3]. This process builds increasingly complex representations:

Table 1: What different CNN layers learn to detect in fashion images

Layer	What It Detects
Early layers	Simple patterns: edges, colours, corners
Middle layers	Combinations: textures, shapes, parts (lapels, sleeves)
Late layers	High-level features: garment categories, styles

Early layers might detect the edge of a sleeve, middle layers recognise a striped pattern, and late layers understand "a formal button-down shirt." This hierarchical organisation gives CNNs an **inductive bias** toward local patterns: they excel at detecting texture and colour but may miss relationships between distant image regions.

1.3.1.2 ResNet and Skip Connections

Deeper networks capture richer features but suffer from **vanishing gradients**: training signals decay as they propagate backward through many layers. ResNet (Residual Network) solves this with **skip connections**: identity paths that bypass convolutional blocks and add the block input directly to its output [3]. This identity path lets gradients flow unimpeded, enabling networks of 50, 101, or 152 layers to train effectively.

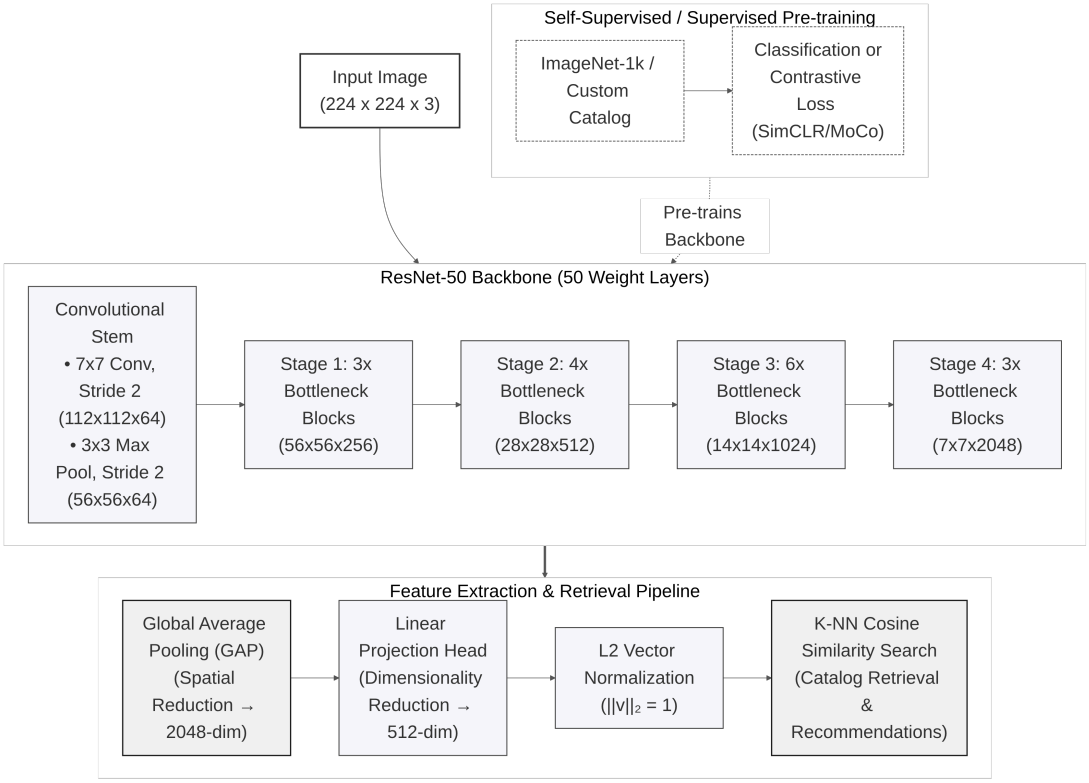

Figure 1: ResNet architecture with residual skip connections enabling gradient flow across very deep networks

ResNet-50, with 25.6 million parameters and 2,048-dimensional embeddings, remains a strong baseline for image retrieval. ResNet-101 (44.5M parameters) provides additional depth for comparison.

1.3.1.3 EfficientNet and Compound Scaling

Traditional scaling strategies enlarge a network along a single dimension: depth (more layers), width (more channels), or resolution (larger inputs). EfficientNet introduces **compound scaling**, which balances all three dimensions simultaneously using a learned coefficient [4]. This produces a family of models (B0 through B7) that achieve competitive accuracy with far fewer parameters than conventional scaling.

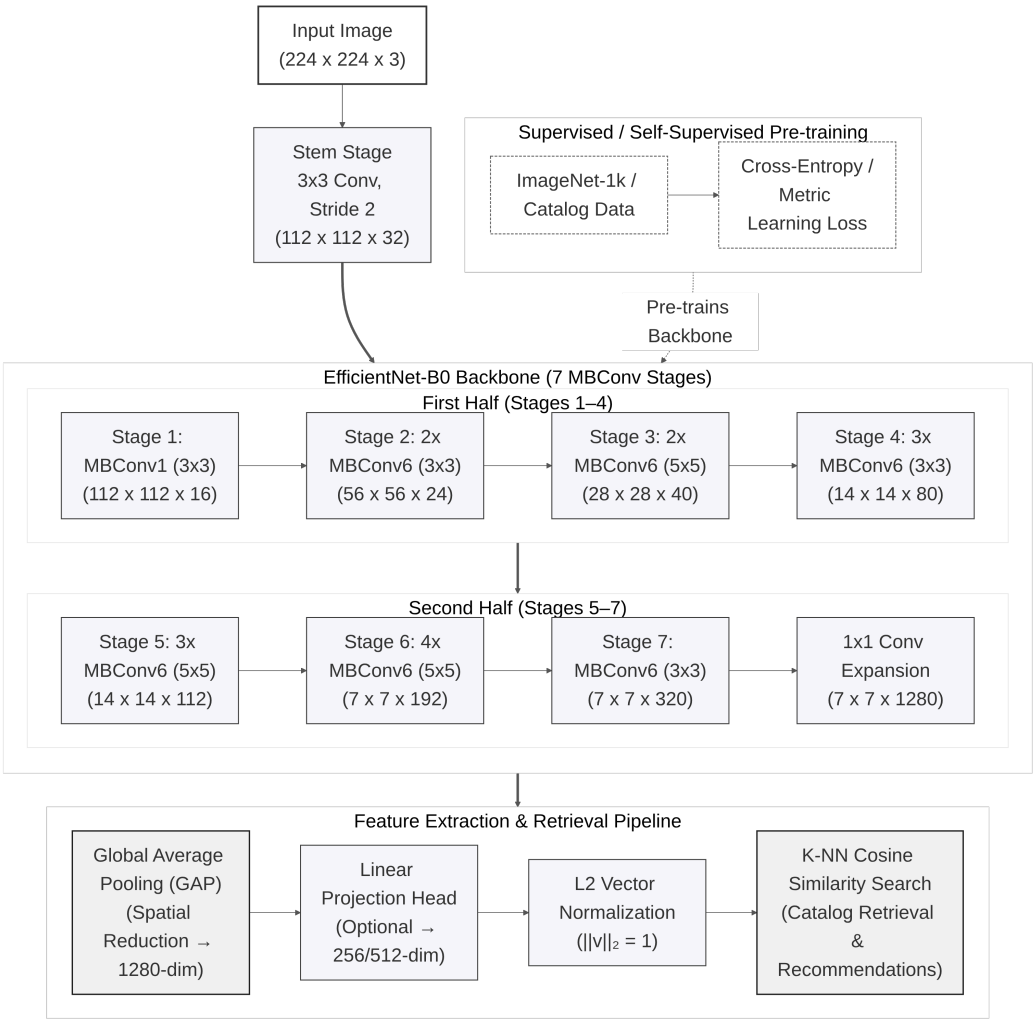

Figure 2: EfficientNet-B0 architecture showing the flow from input image to feature vector

EfficientNet-B0, the smallest variant, uses 5.3 million parameters and produces 1,280-dimensional embeddings. Its compact design makes it well suited to CPU-only deployments. EfficientNet-B4 (19.3M parameters, 1,792-dimensional embeddings) provides higher capacity at increased computational cost.

1.3.1.4 Evaluated CNN Variants

Table 2: CNN-based models evaluated

Family	Model	Parameters	Embedding Dim	Training Data
CNN	ResNet-50	25.6M	2048	ImageNet (1.2M images)
CNN	ResNet-101	44.5M	2048	ImageNet (1.2M images)
CNN	EfficientNet-B0	5.3M	1280	ImageNet (1.2M images)
CNN	EfficientNet-B4	19.3M	1792	ImageNet (1.2M images)

While CNNs excel at detecting local patterns through their convolutional layers, they have limitations in capturing long-range dependencies and global context. This motivated researchers to explore alternative architectures: vision transformers.

1.3.2 Vision Transformers

Vision Transformers (ViTs) apply the transformer architecture from natural language processing to images. This section explains how ViTs work and introduces DINOv2, a self-supervised model evaluated in this project.

1.3.2.1 Patch Embedding and Tokenization

Transformers were originally developed for NLP tasks such as translation and text generation. The key innovation was **self-attention**, which allows the model to consider relationships between all parts of the input simultaneously.

In 2020, researchers showed this approach could also work for images [10]. The idea is straightforward:

- Split the image into a grid of fixed-size patches (e.g., 14 by 14 or 16 by 16 pixels).
- Flatten each patch into a vector and treat it as a token, analogous to a word in a sentence.
- Add position encodings to preserve spatial information.
- Pass the sequence through transformer layers with multi-head self-attention.

1.3.2.2 Global Context via Self-Attention

Unlike CNNs, which focus on local patterns, self-attention captures relationships across the entire image from the first layer. A CNN processes patches in order and mainly compares nearby regions. A ViT can directly compare any two patches, even if they are far apart.

For fashion, this means a ViT can better understand that the collar of a shirt and the cuffs should match, even though they are on opposite sides of the image. This capability is valuable for garment retrieval, where silhouette and drape matter as much as local texture.

1.3.2.3 DINOv2 and Self-Supervised Pre-Training

Most AI models are trained with supervised learning, where humans label images (“this is a dress,” “this is a shoe”), and the model learns from those labels. This requires expensive manual annotation.

Self-supervised learning takes a different approach: the model learns from the images themselves, without human labels. DINOv2 uses a student-teacher self-distillation framework [11]:

- Take an image and create two different views (different crops, slightly different colours).
- Pass them through student and teacher networks with identical architecture.
- Train the student to match the teacher's output.
- Update the teacher as an exponential moving average of the student weights.
- Repeat across 142 million uncurated images.

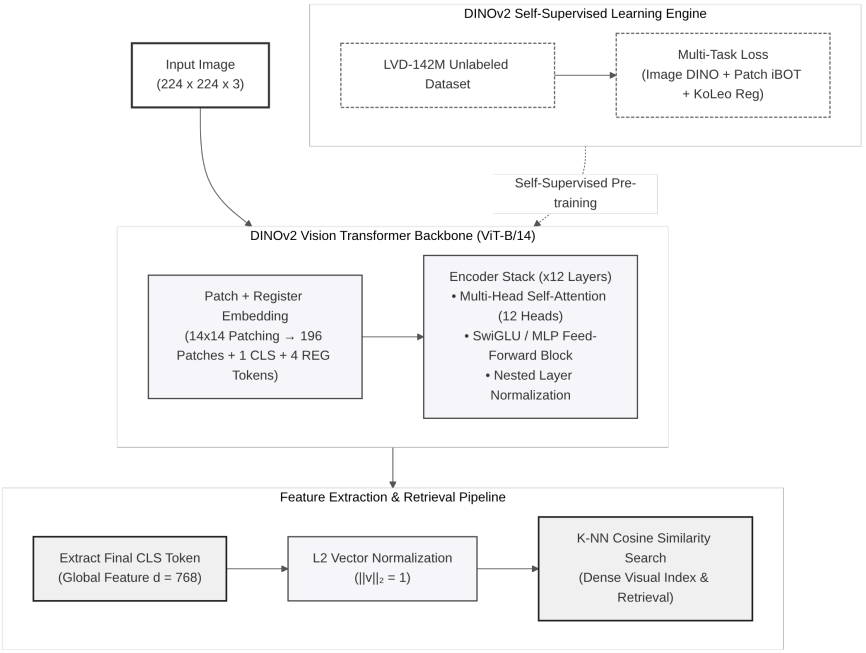

Figure 3: DINOv2 architecture: image patches pass through transformer layers with self-attention to produce a feature vector

DINOv2 produces features that exhibit strong object-level structure: silhouettes, part geometry, and garment boundaries, without being trained on category labels. This makes it adaptable to fashion domains where curated labels are scarce.

1.3.2.4 Structural Fidelity for Fashion Retrieval

DINOv2 is particularly good at capturing **structural fidelity**: the shapes, silhouettes, and proportions of objects. For fashion, this means:

- Matching garments with similar cuts (A-line, fitted, oversized).
- Finding items with similar proportions (cropped vs. full-length).
- Ignoring colour differences when the shape is similar.

This is valuable for users who might want "a dress shaped like this one, but in a different colour."

1.3.2.5 DINOv2 Model Specifications

Table 3: DINOv2 model specifications evaluated in this project

Property	DINOv2 ViT-S/14	DINOv2 ViT-B/14
Architecture	Vision Transformer (Small)	Vision Transformer (Base)
Parameters	21M	86M
Patch size	14 by 14 pixels	14 by 14 pixels
Embedding dimension	384	768
Training data	142M images	142M images
Training method	Self-supervised	Self-supervised

1.3.2.6 Trade-offs and Limitations

Vision Transformers have different trade-offs compared to CNNs:

Advantages:

- Better at understanding global structure and long-range relationships.
- Learned without human labels, so potentially more general.
- Strong structural fidelity: captures shapes and silhouettes.

Disadvantages:

- Slower than CNNs (more computation required).
- Require larger input images for best results.
- May be overkill for simple colour/pattern matching.

Both CNNs and Vision Transformers learn from images alone, mapping visual features to embedding vectors. However, fashion search often involves natural language queries like “red floral summer dress” or “casual weekend outfit.” This requires models that can understand both images and text. The next section introduces CLIP and Fashion-CLIP, which bridge this gap through multimodal learning.

1.3.3 CLIP and Fashion-CLIP

CLIP (Contrastive Language-Image Pre-training) bridges the gap between images and natural language, enabling search using both visual and textual queries. This section explains how CLIP works and introduces Fashion-CLIP, the domain-specialized variant used for visual search.

1.3.3.1 Contrastive Language-Image Pre-Training

Traditional image models classify images into fixed categories (“cat,” “dog,” “dress”). CLIP takes a different approach: it learns to match images with text descriptions [5].

During training, CLIP processed 400 million image-text pairs from the public web. For example, an image of a floral dress paired with the caption

"colourful floral summer dress," or sneakers paired with "white running shoes on grass." From these pairs, the model learned to:

- Convert images into vectors (image encoder).
- Convert text into vectors (text encoder).
- Make matching image-text pairs produce similar vectors.

1.3.3.2 Dual-Tower Architecture

CLIP has two separate towers:

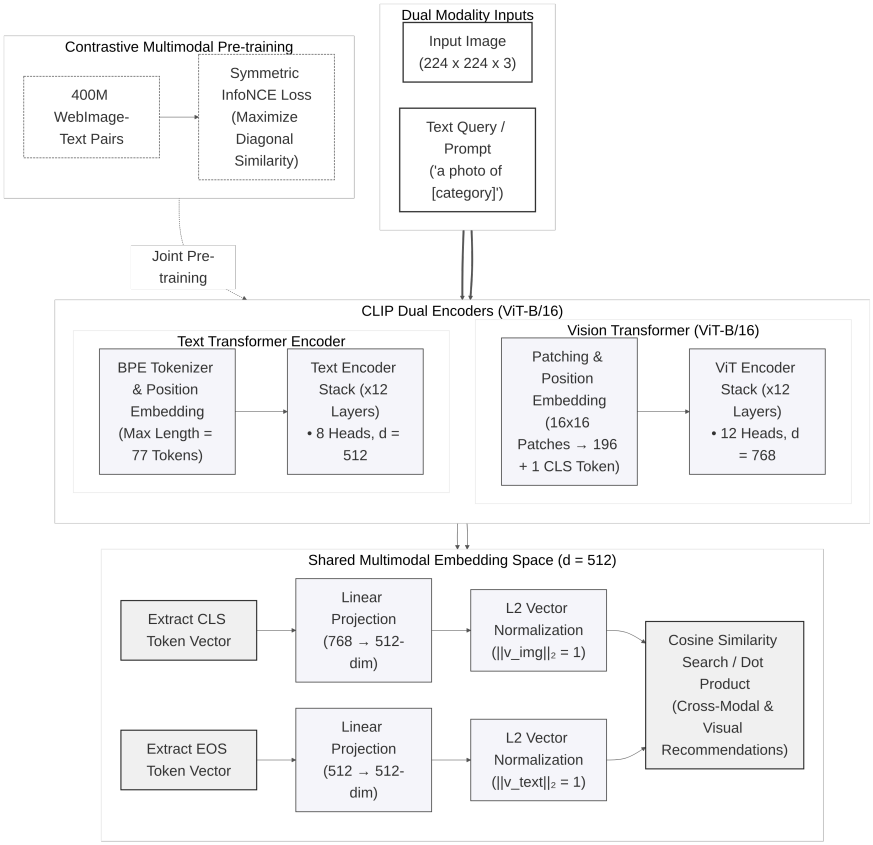

Figure 4: CLIP ViT-B/16 dual-tower architecture: images and text are converted to vectors in the same embedding space, allowing direct comparison

- **Image Tower.** Processes the image using a Vision Transformer (ViT-B/16, ViT-B/32, or ViT-L/14).
- **Text Tower.** Processes text using a transformer.

Both towers output vectors of the same size (512 dimensions for ViT-B variants, 768 for ViT-L), so images and text can be directly compared using cosine similarity.

1.3.3.3 Multimodal Embedding Space

CLIP's ability to understand natural language makes it powerful for fashion:

- It understands concepts like "bohemian style" or "minimalist design."
- It can match images to abstract descriptions ("something for a casual Friday").
- It captures semantic meaning beyond just visual appearance.

However, the general CLIP model was trained on diverse internet images, not specifically fashion. It may not distinguish "A-line dress" from "sheath dress" or "Bohemian style" from "vintage aesthetic." This is where Fashion-CLIP comes in.

1.3.3.4 Multimodal Query Capabilities

The dual-tower design enables query modalities unavailable in vision-only models such as DINOv2 or EfficientNet:

- **Text-to-image search.** A user types "red floral summer dress"; the text encoder maps the description into the same embedding space as catalog images.
- **Hybrid queries.** An uploaded photo can be combined with a textual refinement ("like this, but in blue") by encoding both modalities and merging the results.

This flexibility makes CLIP-based models the primary choice for the visual search feature.

1.3.3.5 Fashion-CLIP and Domain-Specific Fine-Tuning

Fashion-CLIP is a version of CLIP further trained on fashion-specific data [6]. The researchers used over 700,000 fashion product images paired with detailed descriptions covering garment categories, fabric textures, style descriptors, and occasion labels.

This specialization helps Fashion-CLIP understand:

- Fashion-specific vocabulary ("A-line," "empire waist," "distressed denim").
- Style categories ("streetwear," "preppy," "athleisure").
- Occasion suitability ("office wear," "cocktail party," "beach vacation").

Fashion-CLIP inherits the ViT-B/16 architecture, producing 512-dimensional embeddings. The original paper reports a 15-to-20% improvement on fashion retrieval over general CLIP, a result confirmed in the benchmark evaluation presented in Chapter 3.

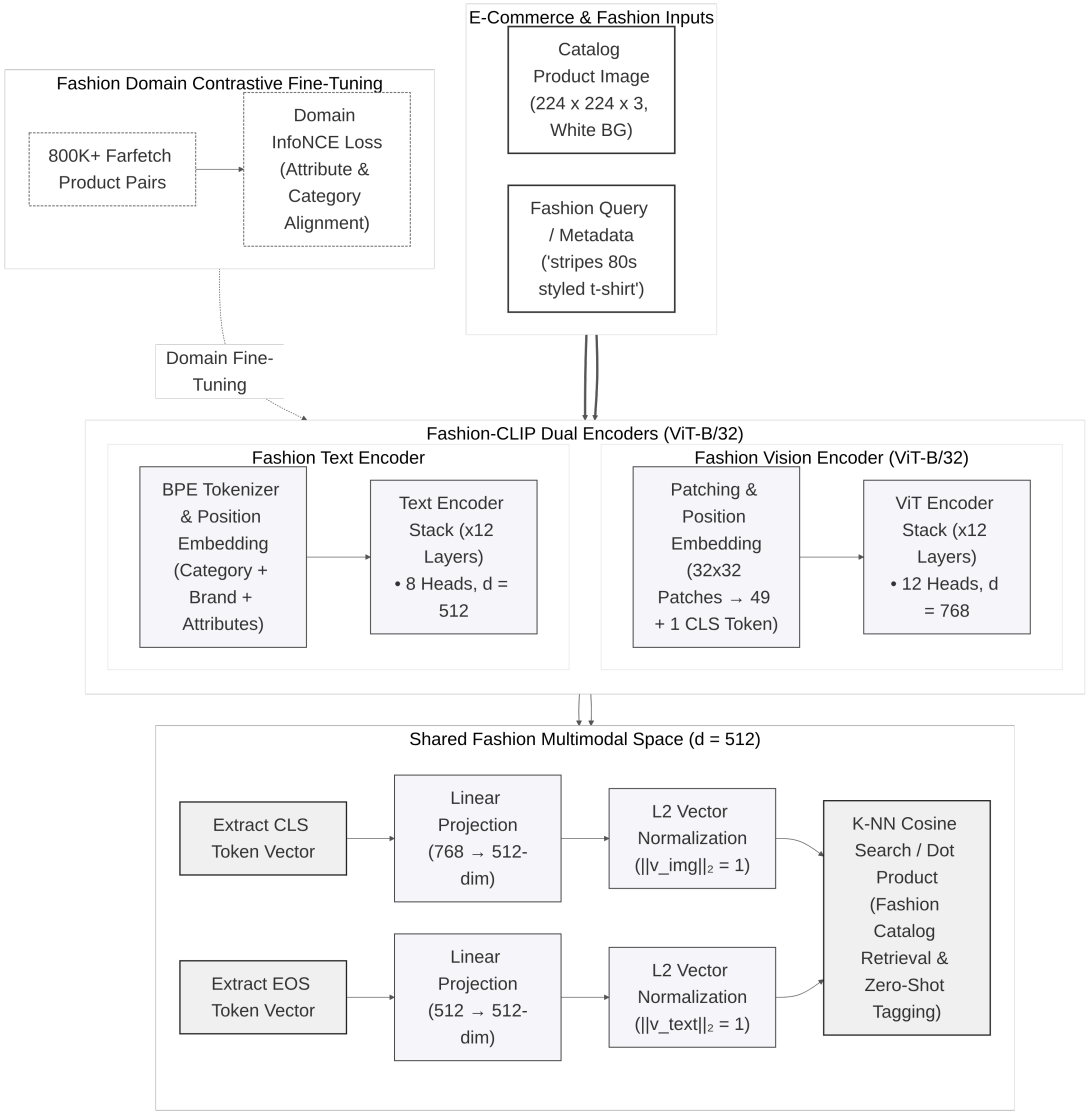

Figure 5: Fashion-CLIP dual-tower architecture: image and text towers independently encode their inputs into 512-dimensional embeddings converging in a shared latent space

1.3.3.6 Evaluated CLIP Variants

Table 4: CLIP variants evaluated

Model	Architecture	Training	Domain
CLIP ViT-B/32	Dual-tower (ViT + text transformer)	Contrastive (400M pairs)	General
CLIP ViT-B/16	Dual-tower (ViT + text transformer)	Contrastive (400M pairs)	General
CLIP ViT-L/14	Dual-tower (ViT + text transformer)	Contrastive (400M pairs)	General

Model	Architecture	Training	Domain
Fashion-CLIP	Dual-tower (ViT + text transformer)	Contrastive, fine-tuned on 700K fashion images	Fashion-specific

Having introduced four model families (CNN, ViT, general CLIP, and Fashion-CLIP), the next section presents a comparative analysis to determine which model best balances retrieval quality, inference speed, and feature capabilities for the fashion e-commerce use case.

1.3.4 Model Selection and Justification

This section presents the comparative analysis of the embedding models and the rationale for selecting Fashion-CLIP as the primary model for visual search.

1.3.4.1 Candidate Models

Eleven pre-trained models spanning three architectural families were evaluated:

Table 5: Candidate embedding models evaluated for fashion product retrieval. All models are pre-trained and publicly available.

Model	Architecture	Dim	Parameters	Training Method
ResNet-50	CNN	2048	25.6M	Supervised (ImageNet)
ResNet-101	CNN	2048	44.5M	Supervised (ImageNet)
EfficientNet-B0	CNN	1280	5.3M	Supervised (ImageNet)
EfficientNet-B4	CNN	1792	19.3M	Supervised (ImageNet)
DINOv2 ViT-S/14	ViT	384	21M	Self-supervised (142M images)
DINOv2 ViT-B/14	ViT	768	86M	Self-supervised (142M images)
CLIP ViT-B/32	ViT + Text	512	151M	Contrastive (400M pairs)
CLIP ViT-B/16	ViT + Text	512	150M	Contrastive (400M pairs)
CLIP ViT-L/14	ViT + Text	768	428M	Contrastive (400M pairs)
Fashion-CLIP	ViT + Text	512	150M	Fine-tuned (700K fashion)

1.3.4.2 Evaluation Methodology

To ensure fair comparison, all models were evaluated under identical conditions. The evaluation used 5,000 fashion product images from the Fashion Product Images dataset, split into training and query sets. Hardware was consumer-

grade: Intel i7-1165G7 CPU with 16 GB RAM, with all inference executed on CPU.

Metrics included:

- **Mean Average Precision (mAP@10).** Primary metric measuring overall retrieval quality.
- **Precision at K (P@1, P@5, P@10).** Accuracy of the top-K results.
- **Recall at 10 (R@10).** Coverage of relevant items in the top-10 results.
- **Inference latency.** Average time to generate one embedding (milliseconds).

A retrieved product was considered relevant if it belonged to the same category as the query image. The full evaluation protocol, benchmark results, and cross-validation methodology are presented in Chapter 3.

1.3.4.3 Weighted Selection Criteria

The model selection was based on four criteria:

1. **Retrieval quality.** mAP@10 and P@K scores measuring how well the model retrieves visually similar products.
2. **Inference latency.** Must support real-time search with sub-300 ms total response time.
3. **Multimodal capability.** The ability to search by both image and text, enabling text-to-image queries.
4. **Hardware compatibility.** Must operate within the memory and compute constraints of commodity hardware.

1.3.4.4 Selection Decision

Fashion-CLIP was selected as the primary embedding model for the visual search feature. Three factors drove this decision:

Retrieval quality. Fashion-CLIP achieved the highest mAP@10 among the evaluated models, with a 15-to-20% improvement over general CLIP on fashion-specific queries. This result was confirmed through the systematic benchmark presented in Chapter 3.

Multimodal capability. Fashion-CLIP's dual-tower architecture enables search by image, by text description, and by hybrid image-plus-text queries. This capability is unavailable in vision-only models such as DINOv2 and EfficientNet.

Domain specialization. Fine-tuning on 700,000 fashion images gives Fashion-CLIP an understanding of fashion-specific vocabulary, styles, and garment attributes that general-purpose models lack.

While EfficientNet-B0 offers faster inference (suitable for ultra-low-latency mobile deployments) and DINOv2 excels at structural fidelity (useful

for silhouette-based matching), Fashion-CLIP provides the best overall balance of retrieval quality, search flexibility, and inference performance for the target deployment scenario.

1.3.4.5 Alternative Deployment Scenarios

For different deployment contexts, alternative models may be preferred:

- **Ultra-low latency.** EfficientNet-B0 provides the fastest inference at 5.3M parameters. Trade-off: 3.4% lower mAP@10 and no text-to-image search.
- **Maximum structural accuracy.** DINOv2 excels at shape and silhouette matching. Trade-off: no multimodal capability.
- **Non-fashion e-commerce.** General CLIP variants are suitable for multi-category marketplaces. Trade-off: lower accuracy on fashion-specific queries.
- **High-resource environments.** CLIP ViT-L/14 offers the largest model capacity. Trade-off: 428M parameters, requiring GPU with significant VRAM.

The complete numerical comparison and error analysis across all 11 models are presented in Chapter 3.

1.4 VECTOR DATABASES

Once images are converted to embeddings, those vectors must be stored and searched efficiently. This section explains the challenge of vector similarity search at scale, introduces two indexing algorithms, and describes pgvector, the PostgreSQL extension used in this project.

1.4.1 The Search Challenge

When a user uploads a query image, the system must compare its embedding vector against every product vector in the catalog. This is **nearest neighbour search**.

A naive brute-force approach scales linearly with catalog size:

- 10,000 products = 10,000 comparisons per search.
- 100,000 products = 100,000 comparisons.
- 1,000,000 products = 1,000,000 comparisons.

For real-time search (under one second), this is too slow.

1.4.2 Approximate Nearest Neighbour Search

The solution is **Approximate Nearest Neighbour** (ANN) search [12]. Instead of checking every vector, ANN algorithms build index structures (graphs or clusters) that let a query navigate directly to the neighbourhood of likely matches, skipping irrelevant vectors.

The key trade-off: we trade perfect accuracy for speed. If the true top match has similarity 0.95 and the returned result has 0.93, that is acceptable for product search. ANN algorithms typically achieve 97 to 99% recall of the true top matches while reducing search time by orders of magnitude [12]. Two algorithms are used in this project: **HNSW** for production search and **IVFFlat** for rapid evaluation [13].

1.4.3 HNSW: Hierarchical Navigable Small World

HNSW is the preferred index for production-scale vector search [12]. It builds a multi-layered graph where each vector is a node connected to its nearest neighbours.

- **Structure.** Multiple layers form a hierarchy. Top layers are sparse and connect distant regions for long-range jumps. Bottom layers are dense and refine the search locally.
- **Search.** Start at a top-layer node, greedily traverse toward the nearest neighbour, descend one layer, and repeat. Cost scales logarithmically with catalog size.
- **Recall.** Reported at over 95% with query latencies under 10 ms, sustained across catalog scales of up to 10 million vectors [12].
- **Build cost.** Graph construction is computationally expensive. At tens of thousands of vectors, build time is measured in minutes.

Configuration parameters:

- **M.** Connections per node. Default 16. Higher improves recall at cost of memory and build time.
- **ef_construction.** Candidates evaluated during index build. Higher produces better index at cost of build time.

HNSW's logarithmic query cost makes it suitable for interactive fashion retrieval at millions of catalog items [12], where sub-100 ms latency is required.

1.4.4 IVFFlat: Inverted File with Flat Compression

IVFFlat is a simpler ANN algorithm used during model evaluation. It partitions the embedding space into clusters via **k-means** and stores each vector in its nearest cluster [13].

- **Search.** Compute distance to all cluster centroids, select the nearest few clusters, and search exhaustively within only those clusters.
- **Recall.** Moderate: 65 to 72% at sub-10 ms for catalog-scale data, per pgvector documentation [13]. Recall degrades as the catalog grows because each cluster contains more candidates.

- **Build cost.** Low. Clustering completes in under one second for 5,000 vectors with minimal memory overhead.

Configuration parameters:

- **lists.** Number of clusters. More lists means smaller clusters and faster search, but risks missing relevant vectors.
- **probes.** Number of clusters searched per query. More probes improve recall at linear cost to query latency.

IVFFlat is used for the model comparison benchmarks in Chapter 3, where the objective is ranking embedding models rather than optimising index performance. Its fast build time and simple configuration suit rapid evaluation. For production, HNSW is the designated long-term index.

1.4.5 pgvector: Vector Search in PostgreSQL

pgvector is an open-source PostgreSQL extension that adds vector operations to standard SQL [13]. It stores vectors alongside regular product data (names, prices, images) in the same database.

Key features:

- **Vector column.** `VECTOR(512)` stores 512-dimensional embeddings. Supports varying dimensions for different models.
- **Similarity operators.** `<=>` for cosine distance (range [0, 2]), `<->` for Euclidean distance.
- **Indexing.** Supports both HNSW and IVFFlat for fast approximate search.

1.4.5.1 Why pgvector

The critical advantage is **transactional consistency**. Vectors and product metadata share the same ACID boundary. An image change triggers a new embedding, and both are committed atomically. No dual-database drift between a vector store and the relational source of truth.

Combined queries are possible in a single SQL statement. A search can find visually similar products filtered by category and price range using one query plan.

1.4.5.2 Example

```
CREATE TABLE products (
    id SERIAL PRIMARY KEY,
    name TEXT,
    price DECIMAL,
    image_embedding VECTOR(512)
);

CREATE INDEX ON products
USING hnsw (image_embedding vector_cosine_ops);
```

```
SELECT id, name, price,
       1 - (image_embedding <=> query_vector) AS similarity
FROM products
ORDER BY image_embedding <=> query_vector
LIMIT 10;
```

1.4.6 Architectural Decision and Trade-offs

Specialised vector databases (Pinecone, Milvus, Weaviate) exist for large-scale search. pgvector was selected for its simplicity and transactional integration.

Table 6: Comparison of pgvector with specialised vector databases

Feature	Specialised Vector DB	pgvector
Setup	Moderate to high	Low (extension only)
Consistency	Separate database	Same transaction as product data
Query language	Custom API	Standard SQL
Cost	Often paid service	Free, open source
Scale limit	Billions of vectors	Millions of vectors

For thousands to tens of thousands of products, pgvector's simplicity outweighs the scaling advantages of specialised databases.

Limitations acknowledged:

- **Scale.** Performs well for millions of vectors; not designed for billion-vector deployments [13].
- **Maturity.** Fewer features and optimisation options than dedicated vector databases.
- **Distribution.** Does not natively distribute across multiple servers.

For this project's scope (5,000 products in evaluation), these limitations are acceptable. The primary contribution is architectural integration within a conventional e-commerce stack, not massive-scale infrastructure.

1.5 PLATFORM ARCHITECTURE AND TECHNOLOGY STACK

Building a production-capable e-commerce platform with integrated machine learning requires deliberate architectural and technology choices. This section surveys architectural patterns, describes each technology in the ReSys.Shop stack, and provides the rationale for each selection.

1.5.1 Architectural Patterns

An application's **architecture** determines how code is partitioned and how components communicate [14]. Three patterns govern system-level structure; a fourth, **vertical slice architecture**, organises code within each module.

1.5.1.1 Monolith

All code runs in a single deployable unit.

- Single codebase, build, and deployment; no network overhead between components.
- Components entangle over time: modifying one area requires understanding others, and any deployment restarts the entire application.

Suitable for small applications with one team and few business domains [15].

1.5.1.2 Microservices

Independent services each own one business capability, communicating over the network [16].

- Teams operate autonomously with different technology stacks; services scale and deploy independently.
- Introduces service discovery, inter-service authentication, partial failure handling, and distributed transaction coordination. Each network hop adds latency.

Suitable for large organisations where teams and domains evolve independently.

1.5.1.3 Modular Monolith

Code partitions into independent modules within a single process. Modules communicate through an **in-process message bus**; compile-time rules prevent direct cross-module references [15].

- Preserves module independence without networking cost. One deployment and one database serve all modules.
- Established boundaries simplify future migration to microservices.
- The single database may limit extreme-scale growth; all modules restart on any deployment.

Suitable for single-team projects requiring logical separation without distributed infrastructure.

1.5.1.4 Vertical Slice Architecture

Each feature is a self-contained folder containing its handler, validation, data access, and endpoint definition [17]. This pattern is orthogonal to those above: it organises code **within** a module, not **across** the system.

- All code for one feature is co-located; features are added, modified, or removed in isolation.
- Cross-cutting concerns (authentication, logging) require explicit extraction to avoid duplication across slices.

Suitable when features are numerous, independent, and evolve at different cadences.

1.5.1.5 Architectural Decision

ReSys.Shop combines three patterns:

- **Modular monolith** at the system level. Nine business modules (Catalog, Ordering, Payment, Inventory, Identity, Profile, Shipping, Location, Dashboard) run in a single .NET process. Modules communicate through MediatR, an in-process message bus implementing the CQRS pattern [18]. Compile-time rules prevent direct cross-module references; all inter-module communication passes through commands and queries.
- **Vertical slice architecture** within each module [17]. Each feature is a self-contained folder containing its handler, request, response, endpoint, and validator. Features are added, modified, or removed in isolation without touching code in other slices.
- **Python sidecar** as the only cross-process component. Embedding generation runs in a dedicated FastAPI service, isolated from the .NET runtime. The sidecar communicates over HTTP; a GPU failure in the sidecar does not affect e-commerce API availability.

This combination provides the deployment simplicity of a monolith, the code isolation of microservices, and machine learning capability without distributed infrastructure overhead [15].

1.5.2 .NET Backend

The backend is built on .NET 10, a high-performance runtime with ahead-of-time compilation and native asynchronous I/O [19]. Its architecture is organised around five core libraries:

- **Carter** extends ASP.NET minimal APIs with module-based endpoint registration. Each business module declares its own routes independently, keeping endpoint definitions co-located with their handlers rather than centralised in a startup file.
- **MediatR** implements CQRS, routing commands (writes) and queries (reads) to handlers through an in-process message bus [18]. Handlers are discovered at startup and dispatched by request type, with no direct coupling between modules.

- **Entity Framework Core** maps C# domain objects to PostgreSQL tables, including pgvector column types for embedding storage [20]. Migrations are version-controlled and applied at startup.
- **FluentValidation** enforces input rules at the application boundary. Each request type has an associated validator that runs before the handler executes, rejecting invalid data before it reaches business logic.
- **Vertical slice architecture** [17] (described in Section 1.5.1) organises each feature as a self-contained folder containing the handler, request, response, endpoint, and validator. This colocation keeps feature concerns together rather than spread across technical layers.

1.5.3 Vue.js Frontend

The frontend uses Vue 3 with TypeScript and the Vite build tool [21]. Two surfaces share a common component library and state management:

- **Storefront.** Product catalog browsing with category trees, faceted filters (price, size, colour), and paginated result grids. Visual search with image upload, similarity-ranked results displaying thumbnail, price, and similarity score. Cart management with session-based guest support and a multi-step checkout flow spanning address entry, delivery selection, payment confirmation, and order completion.
- **Administration interface.** Complete lifecycle management for products, variants, taxonomies, and option types. Order processing with fulfilment status tracking. User and role administration with permission assignment. Analytics overview summarising sales and inventory metrics.
- **Pinia**, a state management library, organises client-side state through typed stores. Each bounded context has its own store (catalog, cart, auth, orders), mirroring the backend module boundaries.

1.5.4 PostgreSQL and pgvector

PostgreSQL 17 hosts both relational business data and vector embeddings in a single database [13].

- **Relational schema.** Tables for products, variants, orders, users, and supporting entities are organised by bounded context. Foreign keys reference entities within the same context; cross-context references use identifier columns without database-level constraints, preserving logical isolation.
- **Performance.** Composite indexes on query-critical combinations (user status, session status, product slug) optimise frequent access patterns. Query plans combine vector similarity and relational filtering in a single execution.

The pgvector extension, HNSW indexing, and transactional consistency between product data and embeddings are described in detail in Section 1.4.

1.5.5 Redis Caching

Redis 7 operates alongside the .NET **HybridCache** abstraction in a two-tier arrangement [22].

- **L1: in-process.** Frequently accessed data (taxonomy trees, front-page product lists) is held in application memory with sub-millisecond read latency. Cache entries expire on a configurable sliding window, typically five minutes.
- **L2: Redis.** The shared tier synchronises cache across application instances. Redis also backs Hangfire job queues and guest session storage. On cache miss at L1, the value is retrieved from Redis and promoted to L1, ensuring subsequent hits stay in-process.

1.5.6 Python ML Sidecar

The machine learning capability runs as a dedicated Python 3.12 service, isolated from the .NET backend due to incompatible runtime dependencies (PyTorch requires Python, .NET requires the CLR) [23].

- **Framework.** FastAPI provides async HTTP endpoints with automatic OpenAPI schema generation. Uvicorn serves as the ASGI runtime.
- **Model management.** A singleton **ModelManager** lazy-loads models from the HuggingFace hub on first request. Once loaded, models persist in GPU memory (or CPU, if no GPU is available) for the lifetime of the service. The manager supports multiple architectures through a common embedding interface.
- **API surface.** `POST /embeddings` accepts raw image bytes (JPEG, PNG, WebP) and returns a JSON array of floating-point values. `GET /health` reports the currently loaded model, its embedding dimension, and the most recent inference latency.
- **Security.** Requests require an `X-API-Key` header validated at the middleware layer. The sidecar listens only on the internal Docker network, inaccessible from the public internet.

1.5.7 .NET Aspire Orchestration

.NET Aspire coordinates the multi-container development and deployment environment [24].

- **Service discovery.** Components resolve each other by name: the .NET backend reaches the Python sidecar at `http://ml-service`, PostgreSQL at `postgres`, and Redis at `redis`. Configuration is injected via environment variables managed by the Aspire host.
- **Lifecycle.** Aspire enforces startup ordering (database before API, sidecar before backend health check) and gates each component behind a readiness probe. Containers restart on failure with configurable back-off.
- **Observability.** OpenTelemetry SDKs in both .NET and Python services export distributed traces, request metrics, and structured logs to the Aspire dashboard. Correlation IDs propagate across the HTTP boundary between backend and sidecar, linking a search request to its embedding generation span.

1.5.8 Hangfire Background Jobs

Hangfire processes operations that should not block HTTP requests [25]. Jobs are persisted in Redis for resilience across application restarts.

- **Cart expiry.** A recurring job runs daily, removing carts with no activity for seven days. This prevents abandoned carts from accumulating indefinitely.
- **Embedding queue.** When an admin uploads new product images, embedding generation tasks are enqueued for asynchronous processing. The upload endpoint returns immediately; the embedding appears in search results once the job completes.
- **Index maintenance.** A periodic job rebuilds the HNSW index on the embedding column, optimising search performance as the catalog grows.

1.5.9 Identity, Authentication, and Authorisation

- **ASP.NET Identity** provides user account management: password hashing with salted PBKDF2, email confirmation workflows, and optional two-factor authentication via TOTP [19].
- **JWT tokens.** Access tokens carry claims (user ID, roles, permissions) and expire after 15 minutes [26]. Refresh tokens enable silent renewal: the client exchanges a refresh token for a new access token, and each refresh token is single-use. Reuse of an already-consumed refresh token triggers revocation of all tokens for that user, mitigating token theft.
- **Guest sessions.** Anonymous users receive a signed session cookie. This cookie links to a server-side session backed by Redis, enabling cart operations and product browsing without authentication. On registration or login, the guest session is transferred to the authenticated user context.

- **Permission model.** Authorisation uses granular claims in the format `domain:category:action` (e.g., `catalog:products:create`). Roles aggregate commonly used permission sets. Endpoint-level attributes evaluate claims at the middleware layer, so handler code contains no authorisation logic.

1.5.10 Benchmark Framework

The benchmark framework is a Python 3.12 pipeline for systematic, reproducible evaluation of embedding models on fashion product retrieval [23]. It is separate from the application codebase and produces the experimental results reported in Chapter 3.

- **Modes.** Three evaluation modes are supported: a one-shot comparison across all configured models, a three-fold cross-validation protocol with stratified category-based splits (the default for thesis results), and a pgvector pipeline mode that measures end-to-end latency including database query and network round-trip time.
- **Models.** 11 pre-trained architectures are implemented: CNN-based (ResNet-50, ResNet-101, ResNet-152, EfficientNet-B0, EfficientNet-B4), vision transformers (DINOv2 ViT-S/14, DINOv2 ViT-B/14), and CLIP variants (CLIP ViT-B/32, CLIP ViT-B/16, CLIP ViT-L/14, Fashion-CLIP). Each model has a thin adapter implementing a common `generate_embeddings` interface.
- **Caching.** Embeddings are cached to disk per model, per fold, and per split (query vs. catalog), avoiding recomputation when re-running evaluation on the same configuration.
- **Metrics.** Retrieval accuracy is measured through mAP, Precision at K, Recall at K, and nDCG. Operational efficiency is measured through inference latency (mean and SD per image), throughput (images per second), model load time, on-disk storage, and RAM usage.
- **Outputs.** Results are exported in JSON, CSV, Markdown, and Typst table formats. The Typst output files (`thesis_aggregate.typ`, `thesis_efficiency.typ`) embed directly into the thesis without manual transcription.
- **Multi-label pipeline.** A separate enriched-dataset mode supports evaluation across three label schemes of increasing granularity (category only, category and colour, and category, colour and pattern), enabling analysis of how embedding quality varies with annotation detail.

1.5.11 Technology Stack Summary

The preceding sections introduced the principal technologies that compose the ReSys.Shop platform. Table 7 consolidates the complete stack.

Table 7: Technology stack of the ReSys.Shop platform

Layer	Technology	Role
Frontend	Vue 3, TypeScript, Vite	Customer storefront and administration interface; reactive UI with Pinia state management
Backend API	.NET 10, Carter, MediatR	REST endpoints via minimal APIs; CQRS command-query separation across business modules
Database	PostgreSQL, pgvector	Relational data and vector embeddings in a single ACID database with HNSW-indexed similarity search
Caching	Redis, HybridCache	Two-tier cache (in-memory L1 and Redis L2); Hangfire job queue and session state backing store
ML Sidecar	Python 3.12, FastAPI, PyTorch	Dedicated embedding generation service with lazy model loading and GPU acceleration
Orchestration	.NET Aspire	Container lifecycle management, service discovery, and reproducible local development environment
Background Jobs	Hangfire	Persistent job processing for cart expiry, embedding queue, and maintenance tasks
Auth and Identity	JWT, ASP.NET Identity	Access tokens, refresh rotation, permission-based authorisation, guest sessions
Benchmarking	Python 3.12, PyTorch	Systematic 11-model comparison with cross-validation, accuracy and efficiency metrics

1.6 RELATED WORK AND RESEARCH GAP

This section positions the ReSys.Shop platform within the landscape of fashion image retrieval research and commercial visual search systems, and identifies the engineering gap that this thesis addresses.

1.6.1 Academic Research

The **DeepFashion** dataset, introduced by Liu et al., established the foundational benchmark for fashion recognition and retrieval with over 800,000 images

annotated with attributes, landmarks, and in-shop-to-consumer photo pairs [27]. This dataset catalysed much of the subsequent work in fashion AI.

FashionIQ extended retrieval to the conversational setting, where users modify queries through natural language feedback ("like this dress but shorter") [28]. While compelling, the interactive dialogue paradigm requires infrastructure beyond the scope of this project, which focuses on single-turn visual and text queries.

The **Fashion-CLIP** work demonstrated that domain-specific fine-tuning of CLIP on 700,000 fashion images improves retrieval by 15 to 20% over the general model [6]. This thesis follows that approach, using pre-trained models without custom training, and extends the evaluation to additional architectures (ResNet, EfficientNet, DINOv2) for systematic comparison.

1.6.2 Commercial Systems

Several platforms have deployed visual search at production scale.

Table 8: Comparison of commercial visual search products

Product	Key Strength	Limitation
Google Lens	Massive scale, general-domain coverage	Closed ecosystem, not customisable
Pinterest Lens	Over 600M monthly searches, style-aware	Proprietary, requires Pinterest integration
ASOS Style Match	Fashion-specific accuracy	Restricted to ASOS catalog only
ViSenze	API-based, good accuracy	Paid service with recurring per-query costs

These products share common limitations for independent projects: they are proprietary and cannot be studied or modified, API access incurs costs at query volume, and reliance on external services creates vendor lock-in. This thesis demonstrates that comparable functionality is achievable with open-source tools, providing both a reference implementation and a cost-effective alternative for smaller deployments.

1.6.3 Contribution Differentiators

This project distinguishes itself from prior work by addressing the **engineering gap** between model research and production systems. Four contributions define this gap:

1. Polyglot architecture. Python's machine learning ecosystem (PyTorch, HuggingFace) does not natively interoperate with the .NET stack common in enterprise e-commerce. This thesis presents a modular monolith with a dedicated AI sidecar, combining .NET's type safety and transactional integrity with Python's access to state-of-the-art vision models, without the operational overhead of a full microservices deployment.

2. Vector-native consistency. By using pgvector within PostgreSQL, embeddings and product metadata share the same transactional boundary. Product updates, image replacements, and index maintenance occur atomically, eliminating stale-index bugs that arise when a vector store and relational database have independent consistency guarantees.

3. Commodity hardware benchmarking. Commercial visual search runs on cloud TPU clusters. This thesis evaluates 11 models on consumer-grade hardware, establishing that production-quality visual search is achievable without specialised infrastructure, lowering the barrier for small to medium e-commerce platforms.

4. Applied model comparison. Rather than chasing leaderboard metrics, this thesis compares models within realistic deployment constraints (inference latency budget, memory limits, storage cost). The resulting accuracy-efficiency trade-off data, presented in Chapter 3, provides a pragmatic guide for practitioners selecting embedding models.

2 DESIGN AND IMPLEMENTATION

This chapter translates requirements into a concrete system design and describes the implementation of the principal components.

- **Requirements Specification.** 88 functional requirements, non-functional requirements, feature classification.
- **System Modeling.** Actors, work breakdown structure, 26 use cases with traceability to requirements.
- **System Architecture and Design.** Domain-driven design, C4 diagrams, database schema, API design, security model.
- **Implementation.** Technology stack, vertical slice architecture, data persistence with pgvector, ML sidecar and CBIR search flow, frontend applications.

2.1 REQUIREMENTS SPECIFICATION

The platform delivers 88 functional requirements across nine **business modules**, each enforcing domain invariants expressed through entity validation rules and application-layer checks. Five non-functional quality dimensions define performance, security, modularity, observability, and reliability targets that shaped architectural decisions throughout design. Feature classification distinguishes three **core research** contributions, detailed in Sections 2.3 and 2.4, from four **supporting infrastructure** modules that provide the realistic evaluation context described in Section 3.2.

- **Functional Requirements.** Traceable per module: Catalog, Identity, Inventory, Ordering, Payment, Shipping, Profile, Location, and Dashboard.
- **Non-Functional Requirements.** Five quality dimensions with measurable, atomic constraints.
- **Feature Classification.** Core Research versus Supporting Infrastructure: scope of the thesis contribution.

2.1.1 Functional Requirements

The system's capabilities are specified as traceable functional requirements organized by **business module**. Each table enumerates specific functional requirements with unique identifiers referenced throughout the design, implementation, and evaluation chapters. Features are implemented via **vertical slices**, as detailed in Section 2.4.1.

2.1.1.1 Catalog Module

The Catalog module manages the product lifecycle, including entity creation, classification, image handling, and the CBIR infrastructure powering visual product search.

Table 9: Catalog module functional requirements.

ID	Requirement	Description	Priority
CAT-FR-01	Product Creation	Create products with name, description, URL slug, and SEO metadata.	High
CAT-FR-02	Fashion Metadata	Assign fashion attributes: style code, season, material composition, care instructions, fit notes, department, and gender target.	Medium
CAT-FR-03	Product Variants	Define sellable variants combining option values with SKU, barcode, physical dimensions, weight, and independent pricing.	High
CAT-FR-21	Variant Option Values	Assign, synchronize, retrieve, and revoke option values to define attribute combinations for variants.	High
CAT-FR-22	Variant Pricing	Set, list, synchronize, and remove time-bound prices per variant with currency specification.	Medium
CAT-FR-04	Variant Images	Upload variant images with automatic thumbnail generation, supporting multiple images per variant with display ordering.	High
CAT-FR-14	Variant Image CRUD	Delete, download, list, and update image metadata (alt text, display order) for variant images.	Medium
CAT-FR-15	Embedding Regeneration	Regenerate vector embeddings for images when the active model configuration changes or corrupted embeddings are detected.	Medium
CAT-FR-05	Embedding Generation	Generate vector embeddings for uploaded variant images via the ML sidecar; store in pgvector [13] with model metadata.	High
CAT-FR-06	Image-Based Search	Enable CBIR: accept query images, extract embeddings, query pgvector via HNSW indexing [12] using cosine distance, and return ranked results.	High
CAT-FR-16	Product Availability	Expose real-time per-variant stock availability through storefront product detail endpoints.	High
CAT-FR-17	Similar Products	Retrieve visually similar products based on vector embedding similarity for a target product.	Medium

ID	Requirement	Description	Priority
CAT-FR-07	Search Configuration	Configure minimum similarity thresholds and maximum result limits for CBIR queries.	Medium
CAT-FR-08	Pluggable Models	Switch embedding models via environment variables without code modifications; filter search results by active model.	High
CAT-FR-09	Taxonomy	Organize products via hierarchical taxonomies with nested taxon trees.	Medium
CAT-FR-18	Taxon Rules	Define business rules on taxon nodes with update, synchronization, and retrieval capabilities.	Low
CAT-FR-19	Auto-Classification	Automatically classify products into taxons based on taxon rules when product attributes change.	Low
CAT-FR-10	Option Types	Define option types (e.g., Size, Color, Material) with ordered values reusable across products.	Medium
CAT-FR-20	Option Association	Assign, synchronize, retrieve, and revoke option types on a per-product basis.	Medium
CAT-FR-11	Slug Uniqueness	Enforce URL slug uniqueness across the catalog at the database constraint level.	High
CAT-FR-12	Status Lifecycle	Support product lifecycles (Draft, Active, Archived) with configurable availability dates.	Medium
CAT-FR-13	Master Variant	Designate one variant per product as the master representation for catalog displays.	High

2.1.1.2 Identity Module

The Identity module provides authentication, dynamic authorization, and user management using JWT access tokens [26] with refresh token rotation.

Table 10: Identity module functional requirements.

ID	Requirement	Description	Priority
IDN-FR-01	Registration	Register customer accounts with email, password, and basic profile information.	High

ID	Requirement	Description	Priority
IDN-FR-02	Password Login	Authenticate via email and password, issuing a 15-minute JWT access token and refresh token.	High
IDN-FR-03	OAuth Login	Authenticate via Google OAuth 2.0 as an alternative to password credentials.	Medium
IDN-FR-04	Token Rotation	Invalidate previous refresh tokens upon issuance, returning a new access/refresh pair.	High
IDN-FR-05	Reuse Detection	Revoke all active refresh tokens for a user if a previously consumed token is presented.	High
IDN-FR-06	Guest Sessions	Support guest sessions via signed cookies, enabling anonymous cart management and browsing.	High
IDN-FR-07	Role-Based Access	Enforce RBAC with permission claims formatted as `domain:category:action` at middleware boundaries.	High
IDN-FR-11	Role Management	Create, update, delete, and list roles; manage permission assignments per role.	Medium
IDN-FR-12	User-Role Assignment	Assign and revoke roles per user, supporting direct permission overrides.	Medium
IDN-FR-13	User Status	Enable or disable user accounts without purging user record history.	Medium
IDN-FR-08	Password Reset	Reset passwords via single-use, time-limited email verification tokens.	Medium
IDN-FR-14	Password Change	Allow authenticated users to update passwords upon verifying current credentials.	Medium
IDN-FR-15	Email Lifecycle	Confirm email addresses via verification tokens and support email modification requests.	Medium
IDN-FR-16	Session Management	Retrieve current sessions, inspect refresh tokens, and terminate sessions via logout.	High
IDN-FR-09	User Governance	Manage user accounts, role bindings, and permission grants via administrative endpoints.	Medium
IDN-FR-10	Two-Factor Auth	Provide optional TOTP-based two-factor authentication.	Low

2.1.1.3 Inventory Module

The Inventory module tracks stock quantities across physical locations, manages checkout reservations to prevent overselling, and maintains audit ledgers.

Table 11: Inventory module functional requirements.

ID	Requirement	Description	Priority
INV-FR-01	Stock Locations	Define physical stock locations with address metadata and active flags.	Medium
INV-FR-02	Stock Quantities	Track on-hand and reserved quantities per variant per location.	High
INV-FR-08	Stock Item CRUD	Create, update, delete, and list stock items; support bulk adjustments and CSV imports.	Medium
INV-FR-09	Low Stock Alerting	Identify inventory falling below configured thresholds for replenishment notifications.	Low
INV-FR-03	Checkout Reservation	Reserve inventory during checkout; release upon cart expiry, cancellation, or timeout.	High
INV-FR-11	Cart Reservations	Reserve stock at cart level, inspect status, and auto-release upon abandonment.	High
INV-FR-04	Overselling Protection	Reject order confirmation when requested quantities exceed available unreserved stock.	High
INV-FR-05	Stock Transfers	Record stock movements between locations with origin, destination, quantity, and timestamp.	Medium
INV-FR-10	Transfer Lifecycle	Manage transfer states (Created, In-Transit, Received, Cancelled) with paged listings.	Medium
INV-FR-06	Audit Logging	Maintain an immutable audit log for stock changes recording operator identity and reason codes.	Medium
INV-FR-12	Movement History	Provide detailed, paged audit trails for stock movements as first-class domain entities.	Medium
INV-FR-07	Reservation Expiry	Expire stale inventory holds after a configurable timeout (default: 15 minutes).	Medium

2.1.1.4 Ordering Module

The Ordering module governs purchase workflows from shopping cart creation through checkout state transitions to completed orders.

Table 12: Ordering module functional requirements.

ID	Requirement	Description	Priority
ORD-FR-01	Shopping Cart	Support guest and customer carts with variant item addition and quantity adjustments.	High
ORD-FR-10	Cart Management	Create, clear, and delete carts, as well as modify item quantities and line items.	High
ORD-FR-02	Cart Association	Merge guest cart contents into customer accounts upon authentication without data loss.	Medium
ORD-FR-03	Cart Expiry	Purge abandoned carts inactive for 7 days via scheduled background maintenance tasks.	Medium
ORD-FR-04	Checkout Pipeline	Enforce strict sequential checkout state transitions: Address → Delivery → Payment → Confirm → Complete.	High
ORD-FR-11	Checkout Validation	Validate inventory levels, address completeness, and shipping method selection prior to payment.	High
ORD-FR-12	Shipping Selection	Select applicable shipping rates within cart boundaries prior to finalizing payment.	Medium
ORD-FR-05	Order Totals	Calculate monetary balances independently: $\text{Item Total} + \text{Adjustments} + \text{Shipping} = \text{Grand Total}$.	High
ORD-FR-06	Dual State Tracking	Maintain parallel, decoupled state machine counters for payment state and fulfillment state.	Medium
ORD-FR-07	Order Cancellation	Cancel orders prior to fulfillment confirmation, releasing inventory holds and voiding payments.	Medium
ORD-FR-13	Admin Order Management	Approve, complete, resume, and modify pending order details via administrative surfaces.	Medium
ORD-FR-14	Customer History	Expose paged customer order histories with line item detail, payment status, and tracking info.	Medium

ID	Requirement	Description	Priority
ORD-FR-08	Order Numbering	Generate unique, sequential order reference numbers within database isolation transactions.	Medium
ORD-FR-09	Order Immutability	Lock finalized orders as immutable to prevent modification post-completion.	High

2.1.1.5 Payment Module

The Payment module handles payment intent lifecycles across external payment providers (Stripe) and internal test gateways.

Table 13: Payment module functional requirements.

ID	Requirement	Description	Priority
PAY-FR-01	Intent Creation	Generate payment intents specifying amount, currency, order reference, and gateway target.	High
PAY-FR-02	Intent Confirmation	Confirm payment intents to authorize fund capture during checkout finalization.	High
PAY-FR-08	Setup Intent	Create Stripe SetupIntents to securely save customer payment methods for future checkouts.	Low
PAY-FR-03	Capture and Refund	Execute capture, void, and refund operations using idempotency keys.	High
PAY-FR-04	Webhook Processing	Verify and process incoming gateway webhooks using HMAC cryptographic signatures.	High
PAY-FR-09	Payment Voiding	Void authorized uncaptured payments and release authorization holds on cancelled orders.	Medium
PAY-FR-10	Method Management	Create, update, toggle, and delete configured payment gateway methods via admin panels.	Medium
PAY-FR-05	Refund Validation	Enforce validation rules preventing refund amounts from exceeding total captured funds.	Medium
PAY-FR-06	Bogus Gateway	Provide a mock payment gateway simulating transaction lifecycles without external API calls.	Medium
PAY-FR-07	State Tracking	Maintain local payment state records parallel to external gateway records for offline query support.	Medium

2.1.1.6 Shipping Module

The Shipping module configures delivery options, geographic destination zones, and dynamic rate calculation algorithms.

Table 14: Shipping module functional requirements.

ID	Requirement	Description	Priority
SHP-FR-01	Delivery Methods	Configure shipping methods with carrier details, rate rules, and geographic zones.	Medium
SHP-FR-04	Method Management	Create, update, activate, deactivate, and delete shipping methods via administrative endpoints.	Medium
SHP-FR-05	Rate Lifecycle	Manage rate matrices mapped to shipping methods, geographic zones, and weight/value tiers.	Medium
SHP-FR-02	Rate Calculation	Calculate shipping costs dynamically based on address zone, method, cart weight, and total value.	Medium
SHP-FR-06	Storefront Evaluation	Evaluate and display eligible shipping methods and rates during checkout.	High
SHP-FR-03	Shipment Tracking	Assign carrier identifiers and tracking tracking numbers to active shipments.	Low

2.1.1.7 Profile Module

The Profile module connects user identity records to customer preferences, addresses, and saved items.

Table 15: Profile module functional requirements.

ID	Requirement	Description	Priority
PRF-FR-01	Address Book	Manage shipping and billing addresses with default selections and custom labels.	Medium
PRF-FR-02	Wishlists	Create, update, and remove named wishlists containing targeted product variants and notes.	Low
PRF-FR-03	Notification Settings	Manage channel-specific notification preferences (email, SMS) with opt-in flags.	Low

2.1.1.8 Location Module

The Location module manages reference data for countries and regional states used in address validation, tax, and shipping zone evaluation.

Table 16: Location module functional requirements.

ID	Requirement	Description	Priority
LOC-FR-01	Country Directory	Maintain country reference entries with ISO 3166-1 alpha-2 codes, names, and active flags.	Medium
LOC-FR-02	State Directory	Maintain state/province entries with ISO 3166-2 codes linked to parent countries.	Medium
LOC-FR-03	Country Management	Create, update, delete, and retrieve country records by database ID or ISO code.	Medium
LOC-FR-04	State Management	Create, update, delete, and list state records filtered by parent country.	Medium

2.1.1.9 Dashboard Module

The Dashboard module consolidates key performance indicators across all business domains for operational review.

Table 17: Dashboard module functional requirements.

ID	Requirement	Description	Priority
DSH-FR-01	Aggregate Dashboard	Aggregate metrics (catalog size, order volumes, stock alerts, user activity) into a unified administrative view.	Medium

2.1.2 Non-Functional Requirements

Five quality dimensions define the operational constraints required for production readiness. Each requirement is specified as an atomic, measurable target that directly influenced key architectural decisions: the 1-second CBIR latency limit (NFR-01a) mandated synchronous vector generation over queued pipelines; the strict modularity constraints (NFR-03a–c) drove the in-process MediatR dispatch architecture; and the reliability baseline (NFR-05a) required Redis-backed persistence for Hangfire background processing. Verification against each target is evaluated in Chapter 3.

2.1.2.1 Performance

Table 18: Performance latency targets for CBIR search and standard API endpoints.

ID	Requirement Target
NFR-01a	**End-to-End Search Latency:** Complete CBIR search workflows (image upload, embedding generation, pgvector similarity lookup, and response assembly) within **1 second** per query.
NFR-01b	**API Response Latency:** Respond to non-search REST API endpoints within **200 milliseconds** under standard operating load.
NFR-01c	**Non-Blocking Processing:** Enforce asynchronous I/O across all data access and HTTP pipelines to eliminate thread pool starvation under concurrent loads.

2.1.2.2 Security

Table 19: Security requirements for authentication, authorization, rate limiting, and transport safety.

ID	Requirement Target
NFR-02a	**Token Lifecycle:** Expire JWT access tokens [26] after **15 minutes** and enforce single-use refresh token rotation with automated reuse detection.
NFR-02b	**Endpoint Authorization:** Enforce fine-grained claims (`domain:category:action`) at the API middleware boundary using dynamic policy resolution.
NFR-02c	**Rate Limiting:** Restrict authentication requests to **5 attempts per minute** and account registration to **3 attempts per hour** per client IP.
NFR-02d	**Upload Hardening:** Validate uploaded files via magic-byte header inspection, strictly enforce file extension allowlists, and cap upload sizes at **10 MB**.
NFR-02e	**Transport Security:** Inject security headers (`CSP`, `HSTS`, `X-Frame-Options`, `X-Content-Type-Options`) across all HTTP responses.

2.1.2.3 Modularity

Table 20: Modularity requirements governing architectural isolation and communication boundaries.

ID	Requirement Target
NFR-03a	**Compile-Time Isolation:** Maintain business modules within a single .NET assembly while enforcing **zero direct cross-references** via build policy checks.

ID	Requirement Target
NFR-03b	**Decoupled Messaging:** Route all inter-module communications exclusively through MediatR in-process message dispatch.
NFR-03c	**Module Testability:** Ensure every module is independently testable without initializing foreign contexts or external database schemas.

2.1.2.4 Observability

Table 21: Observability requirements for tracing, correlation logging, and health monitoring.

ID	Requirement Target
NFR-04a	**Distributed Tracing:** Propagate OpenTelemetry trace contexts across .NET application and Python ML sidecar boundaries via standard HTTP headers.
NFR-04b	**Structured Logging:** Inject a unique correlation identifier into every log entry to track requests across distributed execution paths.
NFR-04c	**Health Monitoring:** Expose dedicated health endpoints verifying database, cache, and sidecar connectivity for orchestrator probes.

2.1.2.5 Reliability

Table 22: Reliability constraints covering background durability, idempotency, and state timeouts.

ID	Requirement Target
NFR-05a	**Job Durability:** Execute background tasks via Hangfire [25] backed by Redis [22] storage to survive process restarts.
NFR-05b	**Automated Maintenance:** Run daily cleanup routines to purge carts inactive for **7 days** and release associated inventory reservations.
NFR-05c	**Webhook Idempotency:** Enforce idempotency key checks on Stripe webhook payloads to prevent duplicate transaction state updates.
NFR-05d	**Reservation Timeouts:** Automatically expire unconfirmed checkout inventory holds after **15 minutes** of user inactivity.

2.1.3 Feature Classification

Platform features are categorized into **Core Research** contributions and **Supporting Infrastructure** components to define the precise scope of this thesis. The core research contributions are detailed in Sections 2.3 and 2.4, while

supporting infrastructure modules are included to provide a realistic e-commerce environment for evaluation, as described in Section 3.2.

Table 23: Classification of system feature areas into Core Research contributions and Supporting Infrastructure components.

Feature Area	Classification	Rationale
Visual Search (CBIR)	Core Research	**Primary Contribution:** An integrated multi-model CBIR system [29] featuring a pluggable architecture. Enables image-based product discovery across multiple embedding model families, including CNNs [3], ViTs [10], and CLIP-based architectures [5], [6].
ML Embedding Pipeline	Core Research	**Operational Backbone:** An automated ingestion pipeline that processes product images, generates vector embeddings via a PyTorch sidecar [23], and manages storage and HNSW indexing [12] using pgvector [13].
Model Benchmark System	Core Research	**Secondary Contribution:** A systematic benchmarking methodology evaluating retrieval accuracy and execution latency across 11 embedding models, establishing model selection guidelines for production deployments.
Product Catalog	Supporting	**Evaluation Domain:** Maintains structured fashion product data (variants, image sets, taxonomies, and attributes) that serves as the vector search target.
Order System	Supporting	**Conversion Tracking:** Captures end-to-end shopping workflows (cart additions, checkouts) to validate visual search utility using real-world interaction metrics.
Inventory	Supporting	**Availability Constraints:** Filters search queries by real-time stock levels, ensuring retrieved items reflect purchasable inventory.
Authentication	Supporting	**Security Boundary:** Protects administrative surfaces and user session state, establishing a production-representative security baseline.

2.2 SYSTEM MODELING

Three actors interact with the platform across 26 use cases, organised into a functional work breakdown structure (WBS) and a summary matrix with detailed scenario specifications. The use case specifications provide full traceability to the functional requirements defined in Section 2.1.

- **System Actors.** Customer, Administrator, and System: roles, responsibilities, interaction surfaces.
- **Functional Decomposition.** WBS: three functional areas with constituent modules and sub-functions.
- **Use Cases.** 26 specifications: summary matrix, detailed scenarios with traceability.

2.2.1 System Actors

Three categories of actors interact with the platform, distinguished by access level and interaction surface.

2.2.1.1 Customer

The **Customer** accesses the browser-based **storefront**.

- **Discovery.** Browse catalog with faceted filters, keyword search, and visual CBIR (Section 2.3.3). Guests browse and cart without registration; session promoted on login.
- **Purchase.** Persistent cart, multi-step checkout (address, delivery, payment, confirm).
- **Account.** Register or Google OAuth login; manage profile, addresses, wishlists, order history.

2.2.1.2 Administrator

The **Administrator** operates a separate administration interface.

- **Catalog.** CRUD products with fashion metadata; variants and pricing; image uploads triggering embedding pipeline; taxonomies, option types, product classification.
- **Operations.** Order review, payment capture/refund, shipment management; real-time stock monitoring per location with audit history.
- **Governance.** User CRUD; role assignment (Customer, Administrator); permission grants (domain:category:action).

2.2.1.3 System

The **System** actor runs automated background jobs via **Hangfire** [25] and **Redis** [22].

- **Embedding Generation.** Process uploaded images via ML sidecar.
- **Cart Expiry.** Daily job: delete carts inactive 7 days, release inventory.
- **Inventory Reservation.** Hold stock during checkout; expire after 15-minute inactivity.
- **Maintenance.** Periodic HNSW index rebuilds; async payment webhook validation.

2.2.2 Functional Decomposition

Figure 6 presents the work breakdown structure (WBS) of the platform across three functional areas:

- **Administration.** Seven modules: Catalog, Ordering, Payment, Inventory, Identity, Shipping, Location.
- **Storefront.** Three modules: Product Discovery (browse, search, visual search), Purchase Flow (cart, checkout, payment, order history), Account Management (authentication, session, profile).
- **Background Services.** Five processes: Embedding Generation, Cart Expiry, Inventory Reservation, Payment Webhook Processing, Index Optimisation.

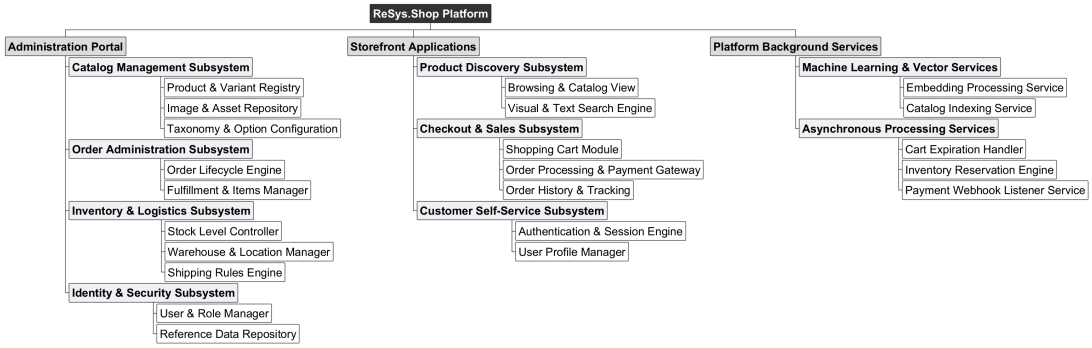

Figure 6: Functional decomposition of ReSys.Shop using a Work Breakdown Structure (WBS), showing the hierarchical breakdown into three functional areas and their constituent modules and sub-functions.

2.2.3 Use Case Summary Matrix

Table 24 consolidates all use cases across the three actors and provides traceability to the functional requirements defined in Section 2.1.

Table 24: Use case summary matrix. All use cases are listed with their actor, associated business module, and traceability to functional requirements defined in Section 2.1.

UC-ID	Use Case	Actor	Module	Related FR
UC-ADM-PROD	Manage products	Admin	Catalog	CAT-FR-01, CAT-FR-02, CAT-FR-03, CAT-FR-11, CAT-FR-12, CAT-FR-13
UC-ADM-VAR	Manage variants	Admin	Catalog	CAT-FR-03, CAT-FR-10, CAT-FR-21, CAT-FR-22

UC-ID	Use Case	Actor	Module	Related FR
UC-ADM-IMG	Manage images and embeddings	Admin	Catalog	CAT-FR-04, CAT-FR-05, CAT-FR-14, CAT-FR-15
UC-ADM-TAX	Manage taxonomies and classification	Admin	Catalog	CAT-FR-09, CAT-FR-18, CAT-FR-19
UC-ADM-OPT	Manage option types	Admin	Catalog	CAT-FR-10, CAT-FR-20
UC-ADM-ORD	Manage orders	Admin	Ordering	ORD-FR-04, ORD-FR-05, ORD-FR-06, ORD-FR-07, ORD-FR-09, ORD-FR-13
UC-ADM-ORD-ITEMS	Manage order details	Admin	Ordering	ORD-FR-13
UC-ADM-PAY	Manage payments	Admin	Payment	PAY-FR-03, PAY-FR-05, PAY-FR-07, PAY-FR-09
UC-ADM-PAY-METHOD	Manage payment methods	Admin	Payment	PAY-FR-10
UC-ADM-STK	Manage stock	Admin	Inventory	INV-FR-02, INV-FR-05, INV-FR-06, INV-FR-08, INV-FR-09, INV-FR-10, INV-FR-12
UC-ADM-LOC	Manage stock locations	Admin	Inventory	INV-FR-01
UC-ADM-USR	Manage users	Admin	Identity	IDN-FR-09, IDN-FR-13
UC-ADM-ROL	Manage roles and permissions	Admin	Identity	IDN-FR-11, IDN-FR-12
UC-ADM-SHP	Manage shipping	Admin	Shipping	SHP-FR-01, SHP-FR-04, SHP-FR-05
UC-ADM-REF	Manage reference data	Admin	Location	LOC-FR-01, LOC-FR-02, LOC-FR-03, LOC-FR-04
UC-STR-BRW	Browse and search catalog	Customer	Catalog	CAT-FR-01, CAT-FR-02, CAT-FR-03, CAT-FR-09,

UC-ID	Use Case	Actor	Module	Related FR
				CAT-FR-16, CAT-FR-22
UC-STR-SRC	Visual search	Customer	Catalog	CAT-FR-06, CAT-FR-07, CAT-FR-08, CAT-FR-17
UC-STR-CRT	Manage cart	Customer	Ordering	ORD-FR-01, ORD-FR-02, ORD-FR-10
UC-STR-CHK	Checkout	Customer	Ordering	ORD-FR-04, ORD-FR-05, ORD-FR-08, ORD-FR-11, ORD-FR-12
UC-STR-OHI	Order history	Customer	Ordering	ORD-FR-07, ORD-FR-14
UC-STR-PAY	Payment processing	Customer	Payment	PAY-FR-01, PAY-FR-02
UC-STR-AUT	Authentication	Customer	Identity	IDN-FR-01, IDN-FR-02, IDN-FR-03, IDN-FR-08, IDN-FR-14
UC-STR-SES	Session management	Customer	Identity	IDN-FR-04, IDN-FR-05, IDN-FR-16
UC-STR-PRF	Profile management	Customer	Profile	PRF-FR-01, PRF-FR-02, PRF-FR-03
UC-SYS-EMB	Embedding operations	System	Embedding	CAT-FR-05, CAT-FR-15
UC-SYS-MNT	System maintenance	System	Infrastructure	CAT-FR-06, CAT-FR-08, ORD-FR-03, INV-FR-07, PAY-FR-04

2.2.4 Administrator Use Cases

The Administrator actor manages data and operational workflows across all business modules through a dedicated administration interface. Each specification includes the actor's goal, trigger, preconditions, postconditions, a numbered main success scenario, alternative and exception flows, and related functional requirements.

2.2.4.1 Product Management

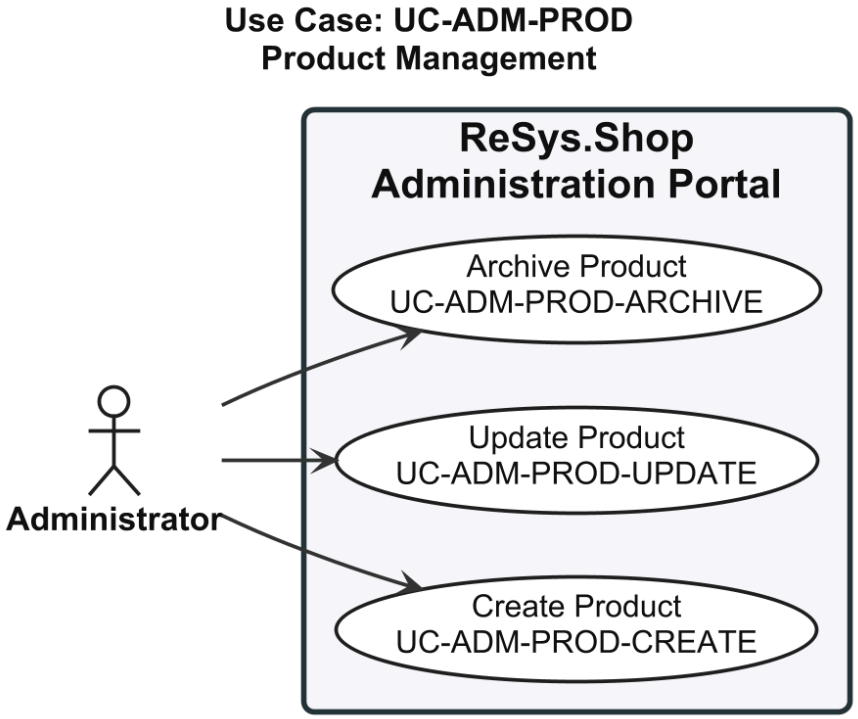

Figure 7: Use case diagram for Product Management (UC-ADM-PROD).

2.2.4.2 UC-ADM-PROD: Manage Products

Table 25: Manage Products.

Field	Description
Use Case ID	UC-ADM-PROD
Use Case Name	Manage Products
Primary Actor	Administrator
Supporting Actors	None
Goal	Create, update, and archive products in the catalog.
Trigger	Administrator accesses the product management section of the administration interface.
Preconditions	• Authenticated with catalog management permissions.
Postconditions	• Product catalog reflects the performed operation. • Status transitions logged for audit.
Main Success Scenario	**Create Product** 1. Selects the create product option. 2. System presents the creation form. 3. Enters product name, description, slug, and fashion metadata (style code, season, material, department, gender target).

	4. Defines at least one variant with SKU and price as the master variant. 5. Assigns the product to relevant taxons. 6. Submits. System validates slug uniqueness and persists. Confirms creation with Draft status. , **Update Product** 1. Selects a product from the catalog listing. 2. System displays the edit form with current values. 3. Modifies fields: name, description, slug, fashion metadata, SEO attributes, or status. 4. Submits. System validates, persists, and confirms the update. , **Archive Product** 1. Selects a product and chooses the archive option. 2. System requests confirmation. 3. Confirms. Product status changes to Archived and is hidden from the storefront.
Alternative Flows	A1. Slug not unique (Create/Update): system rejects and prompts for a different slug. A2. No master variant (Create): system rejects and instructs to designate one. A3. Product has active orders (Archive): system warns and requests explicit confirmation.
Exception Flows	E1. System fails to persist: reports failure and retains form data for retry.
Related Requirements	CAT-FR-01, CAT-FR-02, CAT-FR-03, CAT-FR-11, CAT-FR-12, CAT-FR-13

2.2.4.3 Variant and Pricing

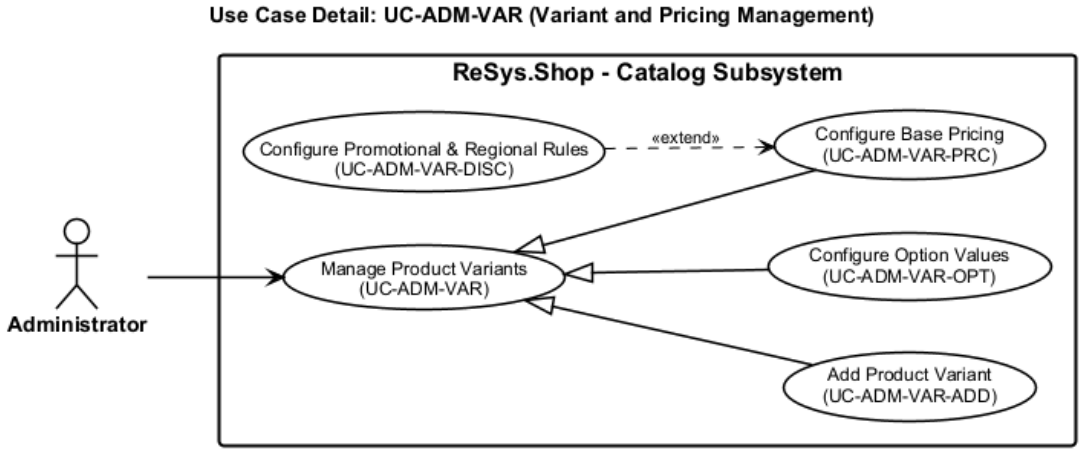

Figure 8: Use case diagram for Variant and Pricing (UC-ADM-VAR).

2.2.4.4 UC-ADM-VAR: Manage Variants

Table 26: Manage Variants.

Field	Description
Use Case ID	UC-ADM-VAR
Use Case Name	Manage Variants
Primary Actor	Administrator
Supporting Actors	None
Goal	Add and manage product variants including option-value configuration and pricing.
Trigger	Administrator opens the variant management section from the product detail page.
Preconditions	• Authenticated with catalog permissions. • Parent product exists.
Postconditions	• Variant created or updated with option assignments and pricing.
Main Success Scenario	**Add Variant** 1. Navigates to product detail and selects add variant. 2. System presents the variant creation form. 3. Enters variant details: SKU, barcode, dimensions, weight, and position. 4. Assigns option values (e.g. Size M, Colour Red) from available option types. 5. Submits. System validates SKU uniqueness and option combination. Creates the variant and confirms. , **Configure Options** 1. Selects a variant from the product detail page. 2. System displays variant detail with current option assignments. 3. Selects an option type and chooses a value; repeats for additional types. 4. Saves. System validates the combination and persists. Confirms the change. , **Configure Pricing** 1. Navigates to pricing management for a product. 2. System displays current prices for all variants grouped by currency. 3. Selects variants and specifies new price with currency and optional validity dates. 4. Submits. System validates non-negative prices and valid currency. Persists and confirms.
Alternative Flows	A1. SKU not unique: system rejects and prompts for a different SKU. A2. Duplicate option combination: system rejects and highlights the conflict. A3. Zero price: system accepts but warns variant appears as free. A4. Overlapping date ranges for same variant and currency: system rejects.

Exception Flows	E1. Persistence failure: system reports and retains form data for retry.
Related Requirements	CAT-FR-03, CAT-FR-10, CAT-FR-21, CAT-FR-22

2.2.4.5 Image and Embedding Management

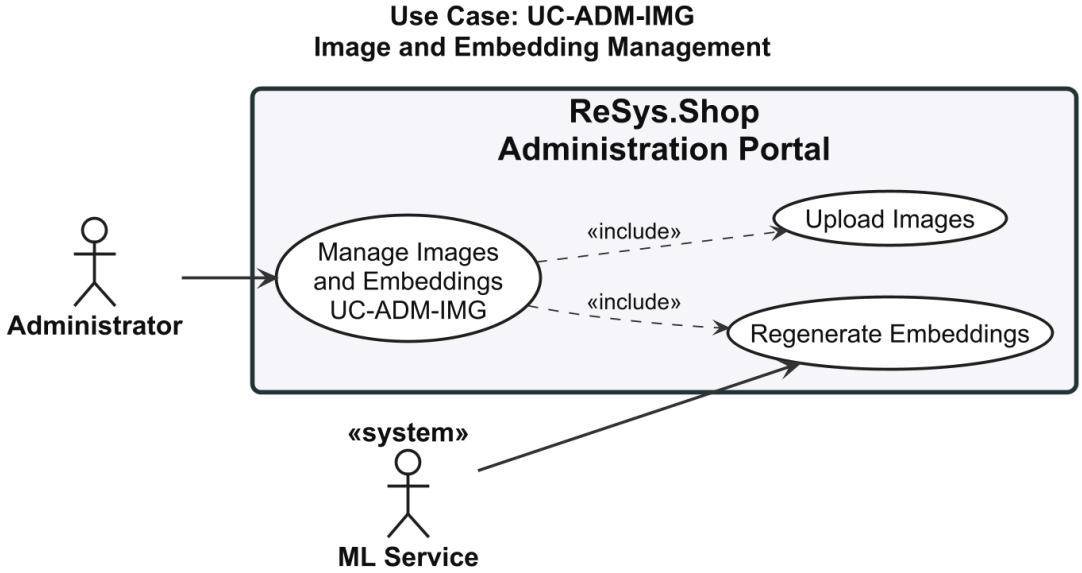

Figure 9: Use case diagram for Image and Embedding Management (UC-ADM-IMG).

2.2.4.6 UC-ADM-IMG: Manage Images and Embeddings

Table 27: Manage Images and Embeddings.

Field	Description
Use Case ID	UC-ADM-IMG
Use Case Name	Manage Images and Embeddings
Primary Actor	Administrator
Supporting Actors	ML Service
Goal	Upload variant images and manage embedding generation.
Trigger	Administrator navigates to image management for a product variant.
Preconditions	• Authenticated with image management permissions. • Variant exists.
Postconditions	• Image stored and associated with variant. Embeddings available for visual search.
Main Success Scenario	**Upload Images** 1. Selects a product variant and initiates image upload.

	2. Selects an image file from the local file system. 3. System validates format (JPEG, PNG, WebP) and size (max 10 MB). 4. Provides alt text and display order position. 5. Confirms and submits. System stores the image, generates thumbnails, and schedules embedding generation. Confirms upload. , **Regenerate Embeddings** 1. Navigates to image management and selects images. 2. Initiates the regenerate embeddings action. 3. System displays confirmation with affected image count. 4. Confirms. System sends each image to the ML service for embedding generation. Stores embeddings with model metadata. Reports completion with success count.
Alternative Flows	A1. Unsupported format: system rejects and lists accepted formats. A2. Exceeds max size: system rejects and displays size constraint. A3. No images selected for regeneration: system disables the action and prompts to select.
Exception Flows	E1. ML service unavailable: system reports failure and suggests retry when operational. E2. Processing cannot be scheduled: system stores image and notifies search will exclude it until processing succeeds.
Related Requirements	CAT-FR-04, CAT-FR-05, CAT-FR-14, CAT-FR-15

2.2.4.7 Taxonomy and Classification

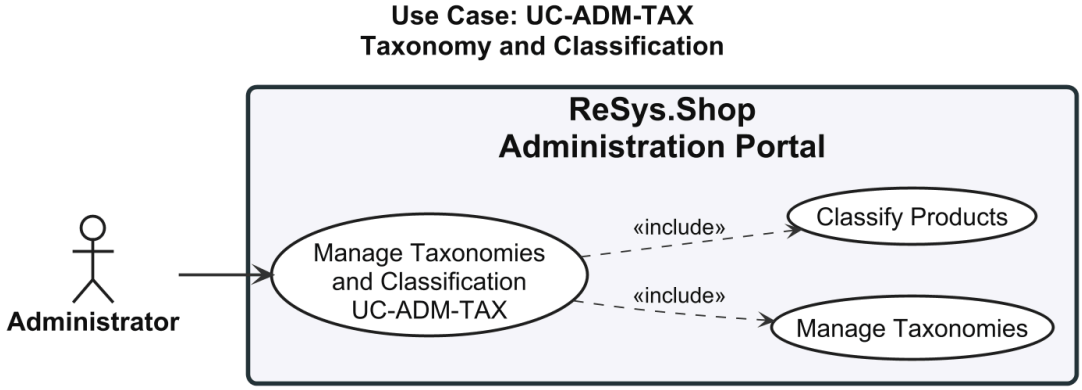

Figure 10: Use case diagram for Taxonomy and Classification (UC-ADM-TAX).

2.2.4.8 UC-ADM-TAX: Manage Taxonomies and Classification

Table 28: Manage Taxonomies and Classification.

Field	Description

Use Case ID	UC-ADM-TAX
Use Case Name	Manage Taxonomies and Classification
Primary Actor	Administrator
Supporting Actors	None
Goal	Create and modify taxonomy structures and assign product classifications.
Trigger	Administrator navigates to taxonomy management or product classification panel.
Preconditions	• Authenticated with taxonomy management permissions.
Postconditions	• Taxonomy structure and product classifications updated.
Main Success Scenario	**Manage Taxonomies** 1. Navigates to taxonomy management. 2. System displays the taxonomy tree with existing taxons. 3. Creates a new taxonomy root or selects an existing taxonomy. 4. Adds, edits, reorders, or removes taxon nodes; optionally defines business rules. 5. Saves. System persists the updated taxonomy and confirms. , **Classify Products** 1. Selects products from the catalog listing. 2. Opens the classification panel. 3. System displays taxonomy tree with current classifications. 4. Selects taxons to assign or deselects to remove. 5. Saves. System validates each taxon path, persists associations, and confirms.
Alternative Flows	A1. Delete taxon with children: system prompts to cascade-delete or reassign. A2. Delete taxon with products: system warns products lose classification. A3. All taxons removed from product: system warns product will not appear in category browsing.
Exception Flows	E1. Concurrent modification: system refreshes data and asks to retry.
Related Requirements	CAT-FR-09, CAT-FR-18, CAT-FR-19

2.2.4.9 Option Type Configuration

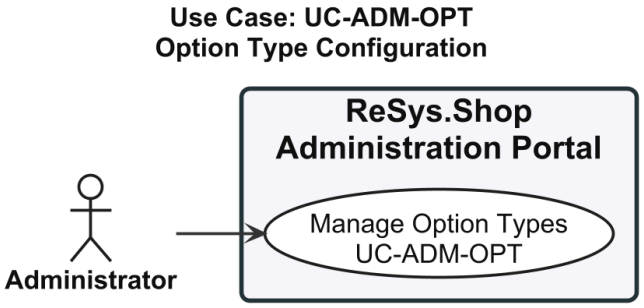

Figure 11: Use case diagram for Option Type Configuration (UC-ADM-OPT).

2.2.4.10 UC-ADM-OPT: Manage Option Types

Table 29: Manage Option Types.

Field	Description
Use Case ID	UC-ADM-OPT
Use Case Name	Manage Option Types
Primary Actor	Administrator
Supporting Actors	None
Goal	Create, modify, and remove option types and their associated values.
Trigger	Administrator navigates to option type management.
Preconditions	• Authenticated with option type management permissions.
Postconditions	• Option types updated and available for product configuration.
Main Success Scenario	1. Navigates to option type management. 2. System displays all option types with their values. 3. Creates a new option type with name and presentation style. 4. Adds ordered option values (e.g. S, M, L, XL for Size). 5. Optionally edits, reorders, or removes existing types and values. 6. Saves. System validates name uniqueness and non-empty values. Persists and confirms.
Alternative Flows	A1. Delete type in use by products: system warns and asks for confirmation. A2. Remove value in use by variants: system warns and asks for confirmation.
Exception Flows	E1. Concurrent modification: system refreshes data and asks to retry.
Related Requirements	CAT-FR-10, CAT-FR-20

2.2.4.11 Order Lifecycle

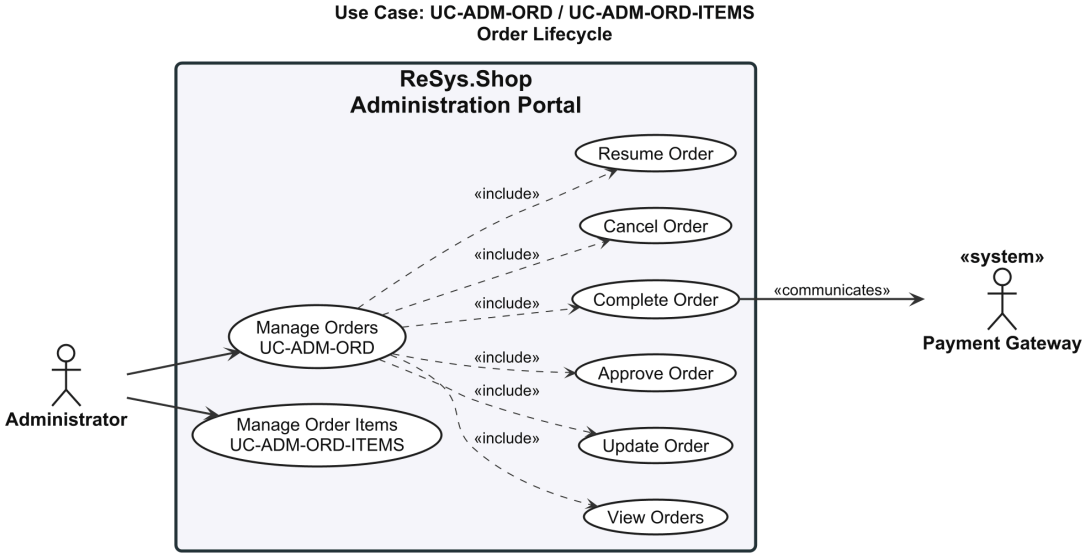

Figure 12: Use case diagram for Order Lifecycle (UC-ADM-ORD, UC-ADM-ORD-ITEMS).

2.2.4.12 UC-ADM-ORD: Manage Orders

Table 30: Manage Orders.

Field	Description
Use Case ID	UC-ADM-ORD
Use Case Name	Manage Orders
Primary Actor	Administrator
Supporting Actors	Payment Gateway
Goal	View, modify, and manage the lifecycle of customer orders.
Trigger	Administrator navigates to order management.
Preconditions	• Authenticated with order management permissions.
Postconditions	• Order state transitions performed. Status logged for audit.
Main Success Scenario	**View Orders** 1. Navigates to order management. 2. System displays order list sorted by most recent. 3. Applies optional filters: status, date range, customer email. 4. Selects an order to view detail with line items, pricing, payment, shipment, addresses, and timeline. , **Update Order** 1. Opens an order and initiates edit. 2. System presents editable form with current values.

	3. Modifies line items, addresses, or shipping method. 4. Submits. System validates modifications, recalculates totals, persists, and confirms. , **Approve Order** 1. Opens a pending order and verifies payment capture and inventory availability. 2. Selects approve. System validates and transitions order to approved. Confirms. , **Complete Order** 1. Opens an order in fulfilment state. 2. Verifies shipment dispatched. 3. Selects complete. System displays confirmation. 4. Confirms. System decrements on-hand inventory, transitions to completed, and logs. Confirms. , **Cancel Order** 1. Opens an order and selects cancel. 2. System displays confirmation with consequences summary. 3. Provides cancellation reason and confirms. 4. System releases reserved inventory, voids or refunds payment, transitions to cancelled. Confirms. , **Resume Order** 1. Locates a paused or stalled order. 2. Resolves the underlying issue. 3. Selects resume. System validates prerequisites, transitions back to pending. Confirms.
Alternative Flows	A1. No orders match: system displays empty message with suggestion to broaden filters. A2. Payment not captured (Approve): system prevents and suggests capturing first. A3. Payment gateway unreachable (Cancel): system cancels order, releases inventory, queues payment action. A4. Prerequisites not met (Resume): system prevents and displays issues to resolve. A5. Concurrent state change: system refreshes and notifies.
Exception Flows	E1. Order became immutable concurrently: system refreshes and notifies order is no longer editable. E2. Payment gateway state mismatch: system prevents and advises verifying with gateway. E3. Inventory decrement data conflict: system reports and suggests verifying stock levels.
Related Requirements	ORD-FR-04, ORD-FR-05, ORD-FR-06, ORD-FR-07, ORD-FR-09, ORD-FR-13

2.2.4.13 UC-ADM-ORD-ITEMS: Manage Order Details

Table 31: Manage Order Details.

Field	Description

Use Case ID	UC-ADM-ORD-ITEMS
Use Case Name	Manage Order Details
Primary Actor	Administrator
Supporting Actors	None
Goal	Manage line items, shipping address, and billing address on existing orders.
Trigger	Administrator opens the order edit form from the order detail view.
Preconditions	• Authenticated with order update permissions. • Order is in a mutable state.
Postconditions	• Order details updated with recalculated totals. Change logged.
Main Success Scenario	1. Opens an order from the listing. 2. System displays order detail. 3. Initiates the edit action. 4. System presents the editable form with current values. 5. Modifies line items (add, update, remove), shipping address, or billing address. 6. Submits. System validates modifications (stock, address completeness), recalculates totals, persists, and confirms.
Alternative Flows	A1. Quantity exceeds stock: system rejects and shows max available. A2. All line items removed: system rejects and warns at least one is required. A3. Address incompatible with shipping method: system warns and prompts method change.
Exception Flows	E1. Order became immutable concurrently: system refreshes and notifies order is no longer editable.
Related Requirements	ORD-FR-13

2.2.4.14 Payment Processing

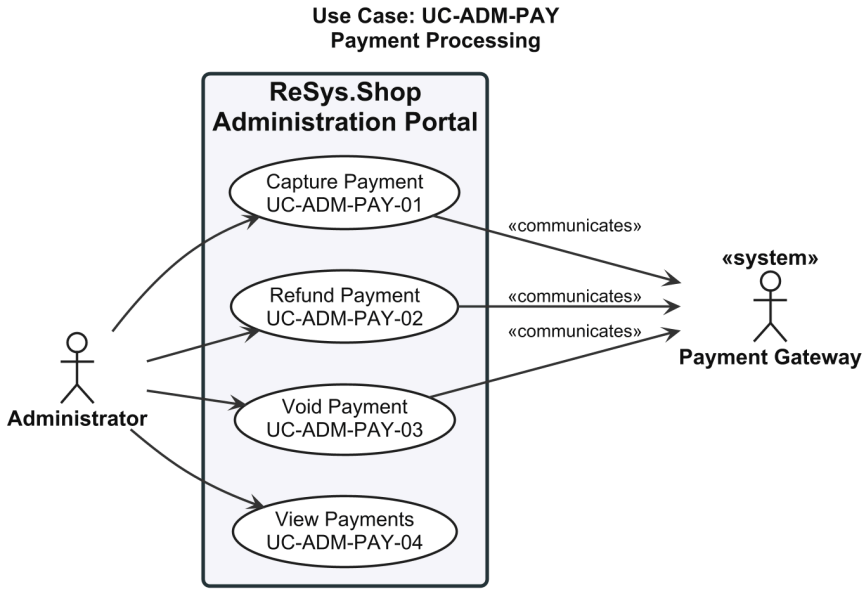

Figure 13: Use case diagram for Payment Processing (UC-ADM-PAY).

2.2.4.15 UC-ADM-PAY: Manage Payments

Table 32: Manage Payments.

Field	Description
Use Case ID	UC-ADM-PAY
Use Case Name	Manage Payments
Primary Actor	Administrator
Supporting Actors	Payment Gateway
Goal	Capture, refund, void, and review payments.
Trigger	Administrator navigates to payment management.
Preconditions	• Authenticated with payment management permissions.
Postconditions	• Payment state updated. Gateway operations recorded.
Main Success Scenario	**Capture Payment** 1. Opens payment detail view for an order. 2. System displays payment state, authorised amount, and capture eligibility. 3. Optionally adjusts capture amount (partial) not exceeding authorised amount. 4. Confirms. System sends capture request to gateway with idempotency key, updates state to captured, and confirms. , **Refund Payment** 1. Opens payment detail view for a captured payment. 2. Enters refund amount (full or partial) and reason.

	3. Confirms. System validates amount, sends refund to gateway, updates state, and confirms. , **Void Payment** 1. Opens payment detail view for an authorised but uncaptured payment. 2. Selects void action and confirms. 3. System sends void request to gateway, updates state to voided, and confirms. , **View Payments** 1. Navigates to payment management. 2. System displays payment list sorted by most recent. 3. Applies optional filters: state, date range, gateway, order number. 4. Selects a payment to view full detail: amount, currency, gateway, state timeline, order reference, capture and refund history.
Alternative Flows	A1. Partial capture/refund: amount less than authorised/captured; remainder available. A2. Already captured (Void): system prevents and suggests refund instead. A3. State mismatch (View): system highlights discrepancy and suggests syncing.
Exception Flows	E1. Gateway rejects: system reports rejection reason. E2. Gateway unreachable: system reports failure; idempotency key ensures safe retry.
Related Requirements	PAY-FR-03, PAY-FR-05, PAY-FR-07, PAY-FR-09

2.2.4.16 Payment Method Configuration

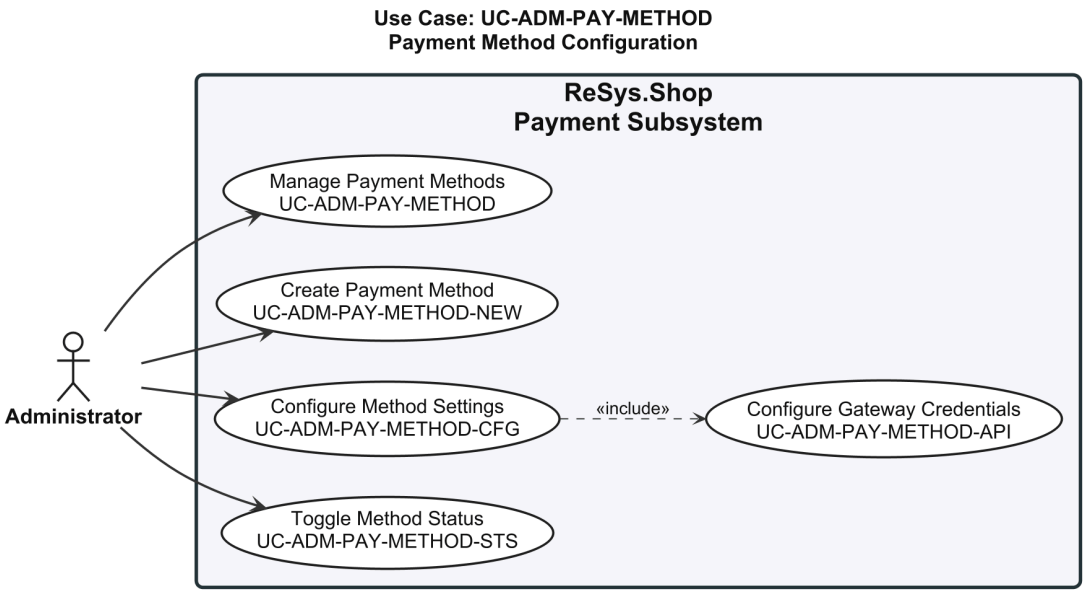

Figure 14: Use case diagram for Payment Method Configuration (UC-ADM-PAY-METHOD).

2.2.4.17 UC-ADM-PAY-METHOD: Manage Payment Methods

Table 33: Manage Payment Methods.

Field	Description
Use Case ID	UC-ADM-PAY-METHOD
Use Case Name	Manage Payment Methods
Primary Actor	Administrator
Supporting Actors	None
Goal	Create, update, activate, and deactivate payment methods.
Trigger	Administrator navigates to payment method configuration.
Preconditions	• Authenticated with payment method management permissions.
Postconditions	• Payment methods updated and available for storefront selection.
Main Success Scenario	1. Navigates to payment method configuration. 2. System displays configured payment methods with activation status. 3. Creates a new method with name, description, gateway identifier, and parameters. 4. Optionally edits, activates, deactivates, or removes methods. 5. Saves. System validates name uniqueness and gateway support. Persists and confirms.
Alternative Flows	A1. Deactivate method in use: system warns new orders cannot use it; existing unaffected. A2. Remove method: system verifies no pending payments reference it. A3. Invalid gateway parameters: system rejects and highlights invalid ones.
Exception Flows	E1. Concurrent modification: system refreshes and asks to retry.
Related Requirements	PAY-FR-10

2.2.4.18 Stock Location Management

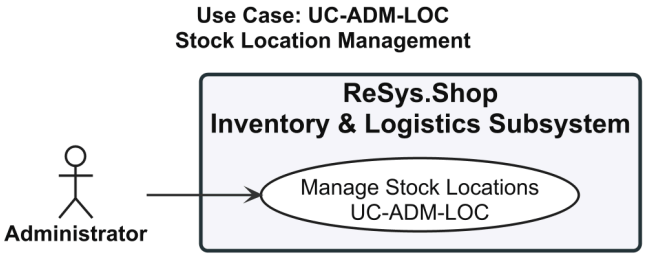

Figure 15: Use case diagram for Stock Location Management (UC-ADM-LOC).

2.2.4.19 UC-ADM-LOC: Manage Stock Locations

Table 34: Manage Stock Locations.

Field	Description
Use Case ID	UC-ADM-LOC
Use Case Name	Manage Stock Locations
Primary Actor	Administrator
Supporting Actors	None
Goal	Create, update, and remove stock locations.
Trigger	Administrator navigates to stock location management.
Preconditions	• Authenticated with location management permissions.
Postconditions	• Location configuration updated.
Main Success Scenario	1. Navigates to stock location management. 2. System displays existing stock locations with addresses and active status. 3. Creates a new location with name, address, and active status flag. 4. Optionally designates the new location as default for stock intake. 5. Optionally edits, deactivates, or removes existing locations. 6. Saves. System validates location name uniqueness. Persists and confirms.
Alternative Flows	A1. Delete location with active stock: system prevents and requires transfer first. A2. Deactivate location: system prevents new intake but allows existing movements. A3. Delete last location: system prevents; at least one must remain.
Exception Flows	E1. Concurrent modification: system refreshes and asks to retry.
Related Requirements	INV-FR-01

2.2.4.20 Stock Item Management

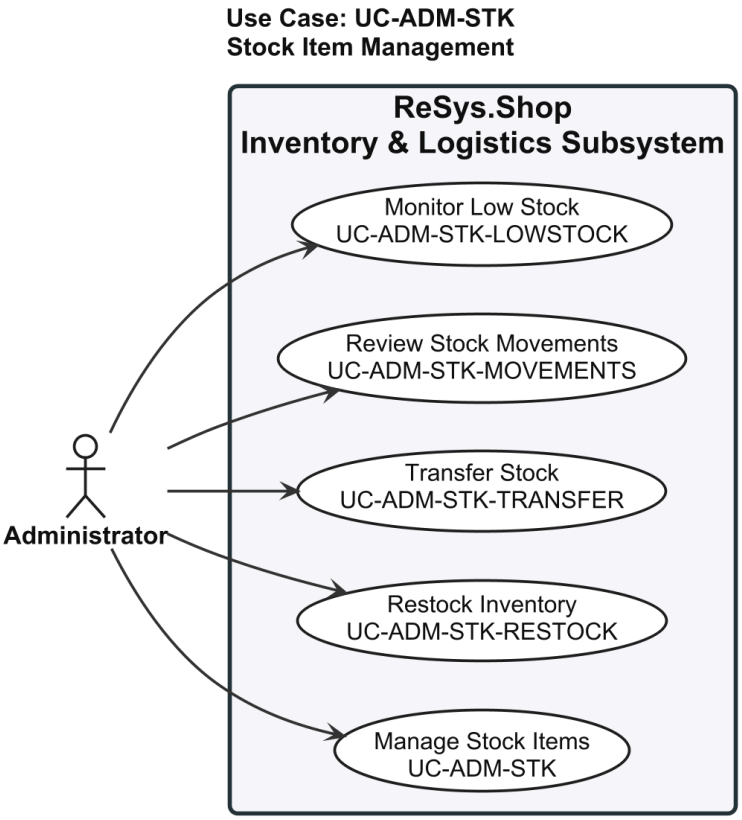

Figure 16: Use case diagram for Stock Item Management (UC-ADM-STK).

2.2.4.21 UC-ADM-STK: Manage Stock

Table 35: Manage Stock.

Field	Description
Use Case ID	UC-ADM-STK
Use Case Name	Manage Stock
Primary Actor	Administrator
Supporting Actors	None
Goal	Create, update, restock, transfer, and monitor stock levels.
Trigger	Administrator navigates to stock item management.
Preconditions	• Authenticated with stock management permissions.
Postconditions	• Stock quantities updated. Changes logged for audit.
Main Success Scenario	**Manage Stock Items** 1. Navigates to stock item management. 2. System displays stock items with variant, location, on-hand, and reserved quantities. 3. Creates a stock item by selecting variant and location, enters initial on-hand quantity.

	4. Alternatively selects existing stock item and updates on-hand quantity. 5. Provides a reason for the adjustment. 6. Saves. System validates variant-location uniqueness, persists, and records audit log. Confirms. , **Restock Inventory** 1. Locates a stock item in the management interface. 2. Enters the restock quantity and provides a reference and notes. 3. Confirms. System increments on-hand quantity and records the restock event. Confirms updated quantities. , **Transfer Stock** 1. Navigates to stock transfer and initiates a new transfer. 2. Selects source location, destination, variant, and quantity. 3. System validates sufficient stock at source. 4. Submits. System creates transfer record pending, decrements source. 5. Upon arrival, confirms receipt. System increments destination and transitions to completed. Confirms. , **Review Stock Movements** 1. Navigates to stock movement audit interface. 2. System displays all stock movements in reverse chronological order with pagination. 3. Applies optional filters: date range, variant, location, movement type. 4. Selects a movement to view full detail. , **Monitor Low Stock** 1. Navigates to low stock monitoring view. 2. System displays stock items below configured threshold with variant, location, on-hand, threshold, and days since last restock. 3. Reviews list and identifies items needing replenishment.
Alternative Flows	A1. Bulk adjustment via file upload: system processes, validates, reports success/failure counts. A2. Reduce below reserved quantity: system rejects and shows current reserved. A3. Transfer exceeds available stock: system rejects and shows maximum. A4. Cancel pending transfer: system returns deducted quantity to source and logs cancellation. A5. No items below threshold (Low Stock): system displays message that all stock levels are sufficient.
Exception Flows	E1. Variant-location pair already exists: system rejects and suggests editing existing. E2. Concurrent modification: system refreshes and asks to re-enter. E3. Retrieval failure: system displays error and offers retry.
Related Requirements	INV-FR-02, INV-FR-05, INV-FR-06, INV-FR-08, INV-FR-09, INV-FR-10, INV-FR-12

2.2.4.22 User Management

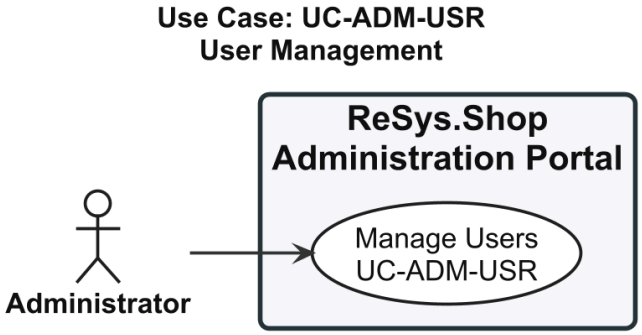

Figure 17: Use case diagram for User Management (UC-ADM-USR).

2.2.4.23 UC-ADM-USR: Manage Users

Table 36: Manage Users.

Field	Description
Use Case ID	UC-ADM-USR
Use Case Name	Manage Users
Primary Actor	Administrator
Supporting Actors	None
Goal	Create, update, enable, and disable user accounts.
Trigger	Administrator navigates to user management.
Preconditions	• Authenticated with user management permissions.
Postconditions	• User account created or modified. Status changes take effect on next authentication.
Main Success Scenario	1. Navigates to user management. 2. System displays user list with default sorting and pagination. 3. Applies optional filters: email, name, role, account status. 4. Creates a new user account by entering email, name, and assigning roles. 5. Alternatively selects an existing user to view detail or edit. 6. Modifies profile fields, enables/disables account, or adjusts role assignments. 7. Saves. System validates email uniqueness, persists, and confirms. If account was disabled, all active sessions are revoked.
Alternative Flows	A1. Disable account: system revokes all active sessions immediately. A2. Re-enable disabled account: system restores active status. A3. Email already registered: system rejects and prompts for a different email.

Exception Flows	E1. Persistence failure: system reports and retains form data for retry.
Related Requirements	IDN-FR-09, IDN-FR-13

2.2.4.24 Role and Permission Governance

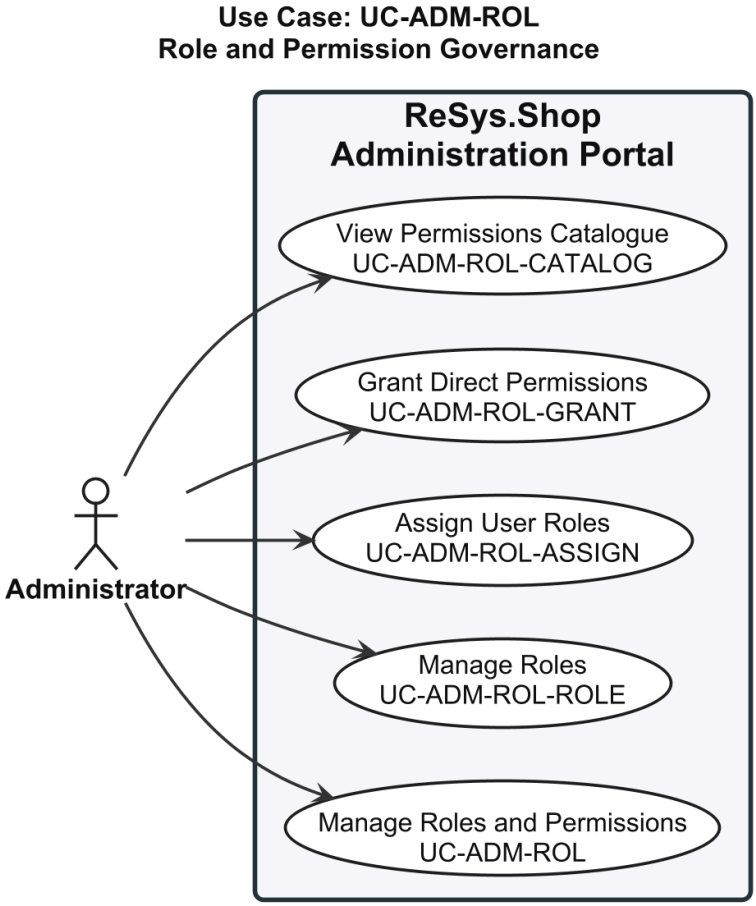

Figure 18: Use case diagram for Role and Permission Governance (UC-ADM-ROL).

2.2.4.25 UC-ADM-ROL: Manage Roles and Permissions

Table 37: Manage Roles and Permissions.

Field	Description
Use Case ID	UC-ADM-ROL
Use Case Name	Manage Roles and Permissions
Primary Actor	Administrator
Supporting Actors	None
Goal	Create and manage roles, assign permissions to roles, and grant roles to users.

Trigger	Administrator navigates to role management or user detail page.
Preconditions	• Authenticated with role and permission management rights.
Postconditions	• Role and permission configuration updated. Affected users receive updated permissions on next token.
Main Success Scenario	**Manage Roles** 1. Navigates to role management. 2. System displays list of roles with name, description, and permission count. 3. Creates a new role with name and description. 4. Assigns permissions from the permissions catalogue. 5. Optionally edits an existing role's name, description, or permission set. 6. Saves. System validates role name uniqueness, persists, and confirms. , **Assign User Roles** 1. Opens a user's detail page. 2. Opens the role assignment panel. 3. System displays all available roles with checkboxes for current assignments. 4. Selects roles to assign and deselects to revoke. 5. Saves. System persists and displays updated effective permissions. , **Grant Direct Permissions** 1. Opens a user's detail page. 2. Opens the direct permission panel. 3. System displays permissions catalogue with role-inherited and direct-grant indicators. 4. Selects permissions to grant directly and deselects to revoke. 5. Saves. System persists and recalculates effective permissions. , **View Permissions Catalogue** 1. Navigates to the permissions catalogue. 2. System displays all permission claims grouped by domain and module. 3. Applies optional filters: domain, category, keyword search. 4. Reviews the permission matrix to plan role designs or audit assignments.
Alternative Flows	A1. Remove role assigned to users: system warns affected users lose permissions. A2. Revoke permission from role: system warns all users with this role lose it on next token. A3. Revoke last role from user: system warns user loses all role-derived permissions. A4. Grant already inherited from role: system accepts; effective permission unchanged but direct grant recorded.
Exception Flows	E1. Concurrent modification or role deleted: system refreshes and asks to retry.

Related Requirements	IDN-FR-11, IDN-FR-12

2.2.4.26 Shipping Method Configuration

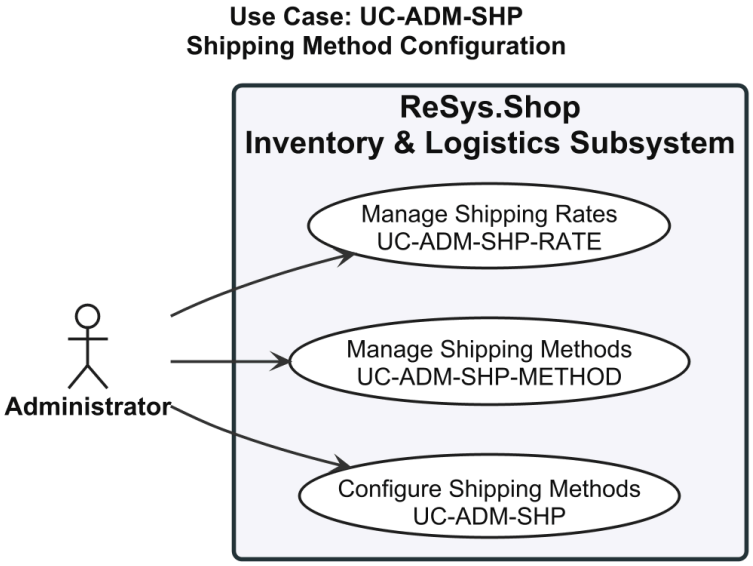

Figure 19: Use case diagram for Shipping Method Configuration (UC-ADM-SHP).

2.2.4.27 UC-ADM-SHP: Manage Shipping

Table 38: Manage Shipping.

Field	Description
Use Case ID	UC-ADM-SHP
Use Case Name	Manage Shipping
Primary Actor	Administrator
Supporting Actors	None
Goal	Configure delivery methods and their associated shipping rates.
Trigger	Administrator navigates to shipping configuration.
Preconditions	• Authenticated with shipping management permissions.
Postconditions	• Shipping methods and rates configured. Active methods available for checkout if zone-applicable.
Main Success Scenario	**Manage Shipping Methods** 1. Navigates to shipping method management. 2. System displays configured shipping methods with activation status. 3. Creates a new method with name, description, carrier identifier, and applicable zones.

	4. Optionally edits, activates, deactivates, or removes existing methods. 5. Saves. System validates method name uniqueness, persists, and confirms. , **Manage Shipping Rates** 1. Selects a shipping method from the method listing. 2. System displays current rate tiers for the method. 3. Creates a new rate tier: selects zone, defines weight and cart value ranges, enters rate amount. 4. Optionally edits or removes existing rate tiers. 5. Saves. System validates rate tier ranges do not overlap for the same zone, persists, and confirms.
Alternative Flows	A1. Deactivate method in active checkouts: system warns; existing selections unaffected. A2. Remove method with active rates: system prompts to remove rates or reassign. A3. No zones assigned: system warns method will not be selectable at checkout. A4. Ranges overlap with existing tiers: system rejects and highlights conflicting tiers.
Exception Flows	E1. Concurrent modification: system refreshes and asks to retry.
Related Requirements	SHP-FR-01, SHP-FR-04, SHP-FR-05

2.2.4.28 Reference Data Management

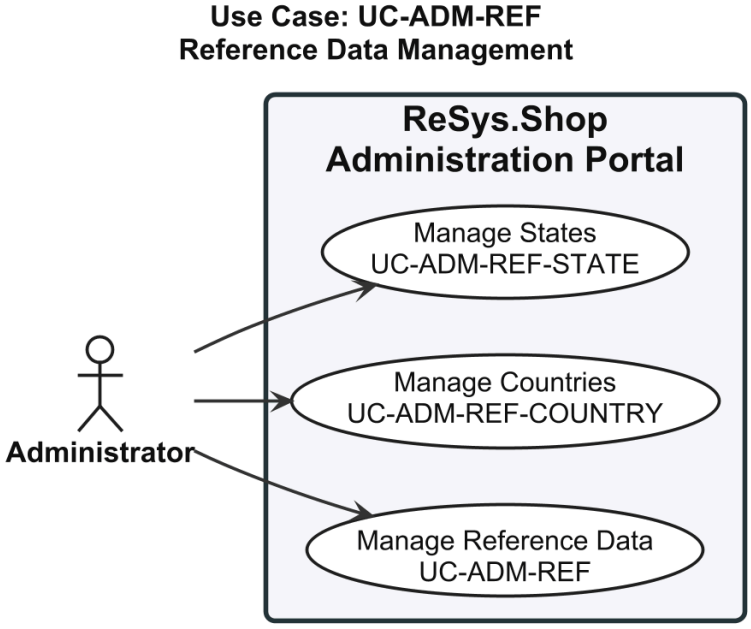

Figure 20: Use case diagram for Reference Data Management (UC-ADM-REF).

2.2.4.29 UC-ADM-REF: Manage Reference Data

Table 39: Manage Reference Data.

Field	Description
Use Case ID	UC-ADM-REF
Use Case Name	Manage Reference Data
Primary Actor	Administrator
Supporting Actors	None
Goal	Create and update country and state reference data.
Trigger	Administrator navigates to reference data management.
Preconditions	• Authenticated with reference data management permissions.
Postconditions	• Country and state data updated. Active records available in address forms and shipping zones.
Main Success Scenario	**Manage Countries** 1. Navigates to country management. 2. System displays list of countries with name, ISO code, and active status. 3. Creates a new country record with display name and ISO 3166-1 code. 4. Sets the active status flag; optionally edits or removes existing country records. 5. Saves. System validates ISO code uniqueness and format, persists, and confirms. , **Manage States** 1. Navigates to state management. 2. System displays list of states with name, ISO code, parent country, and active status. 3. Creates a new state record with display name, ISO 3166-2 code, and selects parent country. 4. Sets the active status flag; optionally edits or removes existing state records. 5. Saves. System validates ISO code uniqueness within parent country, persists, and confirms.
Alternative Flows	A1. Deactivate country: system warns addresses retained but country not in new forms. A2. Delete country with associated states: system warns states will be orphaned. A3. Delete country in shipping zones: system warns and suggests deactivating instead. A4. Deactivate parent country: system prompts whether to cascade-deactivate associated states.
Exception Flows	E1. ISO code already exists: system rejects and prompts for different code.
Related Requirements	LOC-FR-01, LOC-FR-02, LOC-FR-03, LOC-FR-04

2.2.5 Customer Use Cases

The Customer actor interacts through the browser-based storefront for product discovery, purchase, and account management. Each specification follows the same structure as the Administrator use cases.

2.2.5.1 Catalog Browsing

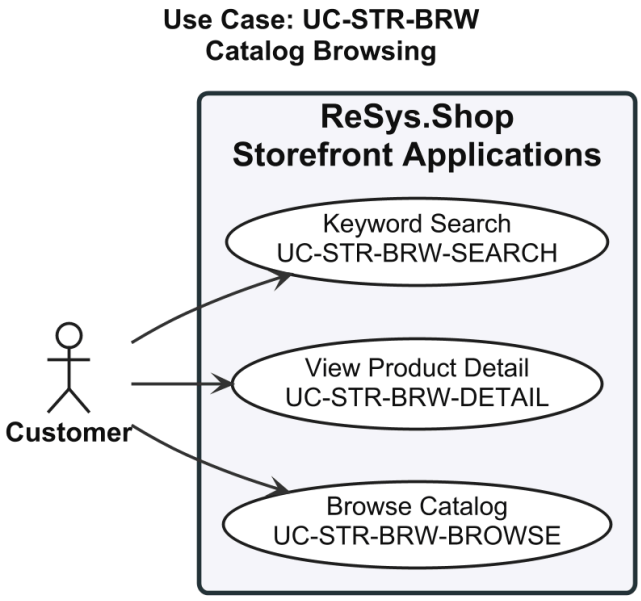

Figure 21: Use case diagram for Catalog Browsing (UC-STR-BRW).

2.2.5.2 UC-STR-BRW: Browse and Search Catalog

Table 40: Browse and Search Catalog.

Field	Description
Use Case ID	UC-STR-BRW
Use Case Name	Browse and Search Catalog
Primary Actor	Customer
Supporting Actors	None
Goal	Browse the product catalog, view product details, and search by keyword.
Trigger	Customer visits the storefront and selects a category, product, or enters a search term.
Preconditions	• Catalog data is published and searchable.
Postconditions	• Product listing or detail displayed with pricing and availability.
Main Success Scenario	**Browse Catalog**

	1. Visits the storefront home page or a category landing page. 2. System displays the taxonomy tree with top-level categories. 3. Selects a category or subcategory from the taxonomy navigation. 4. System retrieves products associated with the selected taxon and its descendants. 5. System displays a paginated grid of product cards showing thumbnail, name, price, and availability. 6. Applies optional facets to filter by option types (e.g. size, colour) and other attributes. 7. System refreshes the listing with filtered results. , **View Product Detail** 1. Clicks on a product card from a catalog listing, search result, or recommendation. 2. System displays product detail page with primary image, name, description, and price range. 3. System shows fashion metadata: style code, season, material, department, gender target. 4. System displays taxonomy breadcrumb showing category path. 5. Browses product images in the gallery. 6. Selects variant option values from available options. 7. System updates displayed price, availability, and variant-specific images based on selection. , **Keyword Search** 1. Enters a search keyword or phrase in the storefront search bar. 2. System queries the catalog for products matching name, description, or fashion attributes. 3. System ranks results by relevance and displays a paginated grid of matching product cards. 4. Refines search with additional keywords or applies filters.
Alternative Flows	A1. No products in category: system displays empty state suggesting other categories. A2. Selected variant out of stock: system shows out-of-stock status and disables add-to-cart. A3. No products match keyword: system displays suggestions to check spelling or browse categories. A4. Very short query (single character): system prompts to enter at least two characters.
Exception Flows	E1. Retrieval or search failure: system displays error and offers retry.
Related Requirements	CAT-FR-01, CAT-FR-02, CAT-FR-03, CAT-FR-09, CAT-FR-16, CAT-FR-22

2.2.5.3 Search

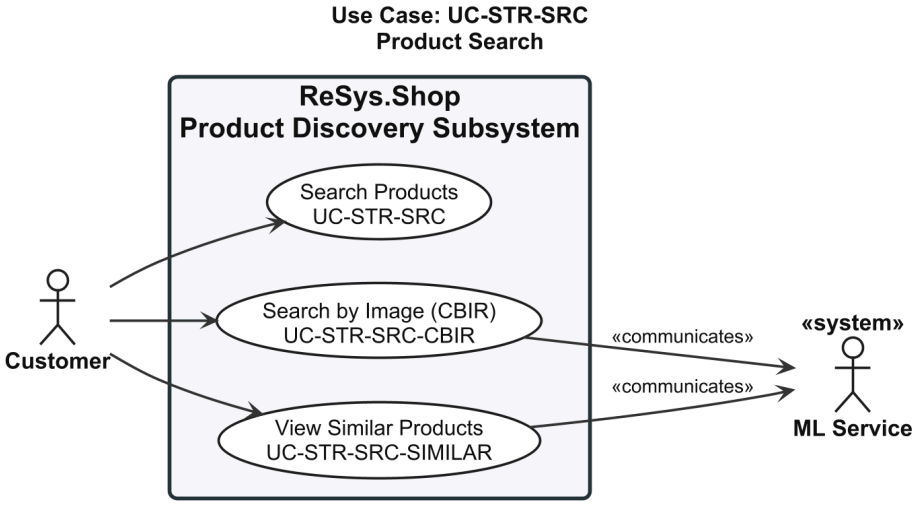

Figure 22: Use case diagram for Search (UC-STR-SRC).

2.2.5.4 UC-STR-SRC: Visual Search

Table 41: Visual Search.

Field	**Description**
Use Case ID	UC-STR-SRC
Use Case Name	Visual Search
Primary Actor	Customer
Supporting Actors	ML Service
Goal	Search for products by uploading a reference image and discover visually similar items.
Trigger	Customer navigates to visual search or views a product detail page.
Preconditions	• Catalog has products with embeddings. • ML service is operational.
Postconditions	• Visually similar products displayed with similarity scores above configured threshold.
Main Success Scenario	**Search by Image (CBIR)** 1. Navigates to the visual search interface. 2. Selects or drags an image file from the local file system. 3. System validates image format (JPEG, PNG, WebP) and size constraints. 4. System sends the image to the ML service for embedding generation. 5. System performs similarity search against the image embeddings index.

	6. System retrieves matching products ranked by similarity score above minimum threshold. 7. System displays results as a grid of product thumbnails with scores, linking to detail pages. , **View Similar Products** 1. Opens a product detail page with image embeddings available. 2. System retrieves the product's primary image embedding. 3. System performs similarity search against other product embeddings. 4. System displays a horizontal carousel or grid of visually similar product cards. 5. Clicks on a similar product card to navigate to its detail page.
Alternative Flows	A1. Invalid image format: system rejects and lists accepted formats. A2. No products above threshold: system displays empty result suggesting different image or keyword search. A3. No images or embeddings on detail page: system hides the similar products section. A4. ML service unavailable: system gracefully hides section without affecting detail page.
Exception Flows	E1. ML service unavailable: system displays error and suggests retrying later. E2. Embedding index empty: system reports visual search not yet available.
Related Requirements	CAT-FR-06, CAT-FR-07, CAT-FR-08, CAT-FR-17

2.2.5.5 Cart Management

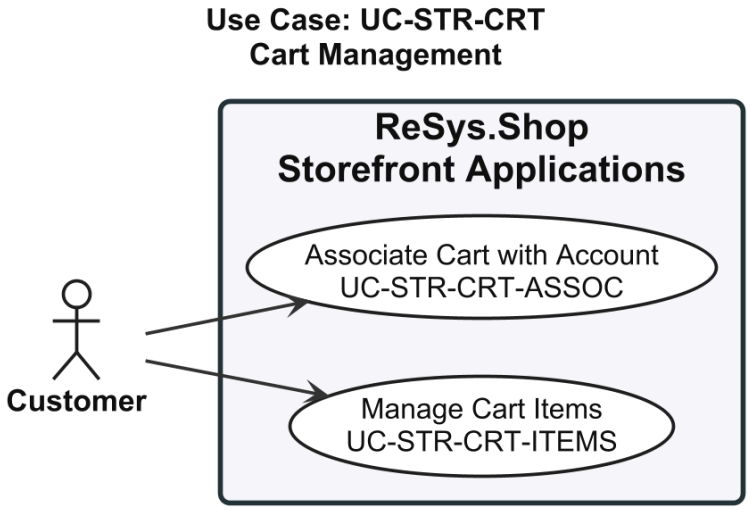

Figure 23: Use case diagram for Cart Management (UC-STR-CRT).

2.2.5.6 UC-STR-CRT: Manage Cart

Table 42: Manage Cart.

Field	Description
Use Case ID	UC-STR-CRT
Use Case Name	Manage Cart
Primary Actor	Customer
Supporting Actors	None
Goal	Add, update, and remove items in a shopping cart; associate guest cart with account.
Trigger	Customer selects a variant on the product detail page and adds it to the cart.
Preconditions	• Product variant exists and is available.
Postconditions	• Cart persisted across page navigation. Guest carts survive browser sessions.
Main Success Scenario	**Manage Cart Items** 1. On a product detail page, selects a variant and specifies quantity. 2. Clicks Add to Cart. System validates the variant and quantity. 3. System adds the variant to the customer's cart and displays confirmation. 4. Opens the cart to view all items with quantities, prices, and subtotal. 5. Updates quantity of an item or removes an item. 6. System recalculates cart subtotal and updates the display. , **Associate Cart with Account** 1. Browses storefront as guest and adds items to cart. 2. Logs in or registers for an account. 3. System detects guest cart exists and retrieves any existing user cart. 4. System merges guest and user carts: matching variants increase quantity; unique variants are added. 5. System associates the merged cart with the user account and invalidates guest cart cookie.
Alternative Flows	A1. Quantity exceeds stock: system rejects and shows max available. A2. Same variant already in cart: system increments existing quantity. A3. Guest customer: system assigns session-based cart identifier in signed cookie. A4. No existing user cart: system transfers guest cart to user account directly. A5. Merge exceeds available stock: system caps at max available and notifies customer.
Exception Flows	E1. Variant deactivated or archived: system rejects and suggests refreshing product page. E2. Merge fails due to data conflict: system creates user cart with guest items and notifies to review.

Related Requirements	ORD-FR-01, ORD-FR-02, ORD-FR-10

2.2.5.7 Checkout Flow

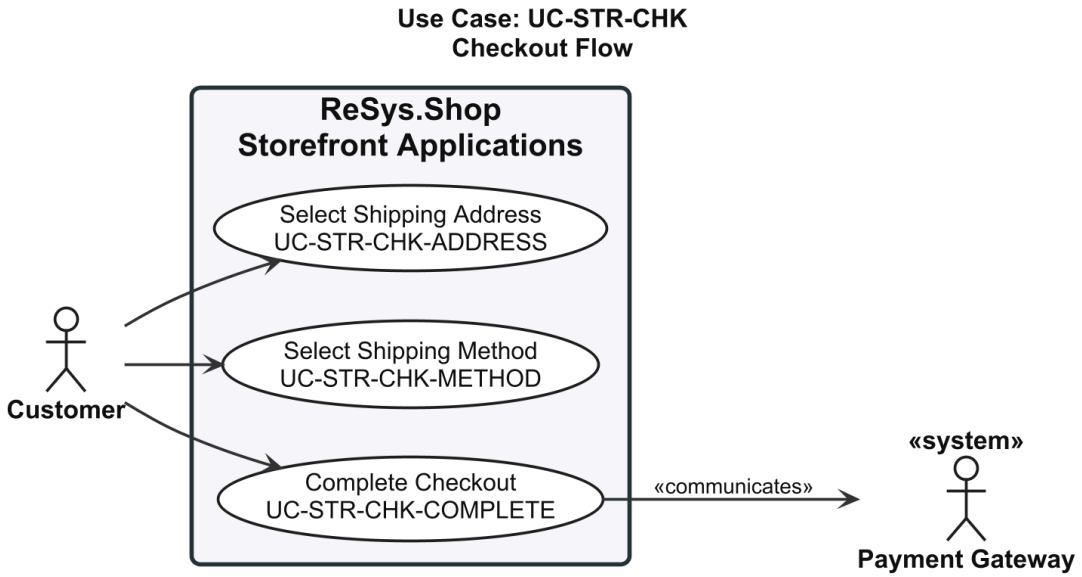

Figure 24: Use case diagram for Checkout Flow (UC-STR-CHK).

2.2.5.8 UC-STR-CHK: Checkout

Table 43: Checkout.

Field	Description
Use Case ID	UC-STR-CHK
Use Case Name	Checkout
Primary Actor	Customer
Supporting Actors	Payment Gateway
Goal	Complete the multi-step checkout: address selection, shipping method, and order confirmation.
Trigger	Customer proceeds from cart to checkout.
Preconditions	• Customer is authenticated. • Cart is not empty. • Stock is available for all items.
Postconditions	• Order created with unique number. Inventory reserved. Payment linked. Cart cleared.
Main Success Scenario	**Select Shipping Address** 1. From the cart, clicks Proceed to Checkout. 2. System transitions checkout to the Address step.

	3. System displays saved addresses with the default pre-selected. 4. Selects an existing shipping address or enters a new one. 5. System determines shipping zone and proceeds to the next step. , **Select Shipping Method** 1. System calculates available shipping methods and rates based on address zone, cart weight, and cart value. 2. System displays list of available methods with rates and estimated delivery times. 3. Reviews options and selects a preferred shipping method. 4. System applies the selected method and rate; updates shipment total in order summary. 5. System presents next checkout step (payment). , **Complete Checkout** 1. System displays order summary with line items, shipping, tax, and total. 2. Enters payment details or selects a saved payment method. 3. System creates a payment intent and reserves inventory for each line item. 4. Reviews final order summary and confirms the purchase. 5. System captures payment and generates a unique order number. 6. System transitions order to Confirmed state, clears cart, and displays order confirmation. 7. System sends order confirmation notification.
Alternative Flows	A1. No saved addresses: system presents empty address step with prompt to create new. A2. No methods available for zone: system displays message and prompts different address. A3. Only one method available: system auto-selects and proceeds. A4. Stock depleted during checkout: system notifies, removes affected items, returns to cart. A5. Payment failure: system notifies with reason; allows retry; inventory not reserved.
Exception Flows	E1. Address validation fails: system highlights missing fields and prevents progression. E2. Rate calculation fails: system displays error and suggests contacting support. E3. Payment captured but inventory reservation fails: system voids payment and notifies order not completed.
Related Requirements	ORD-FR-04, ORD-FR-05, ORD-FR-08, ORD-FR-11, ORD-FR-12

2.2.5.9 Order History

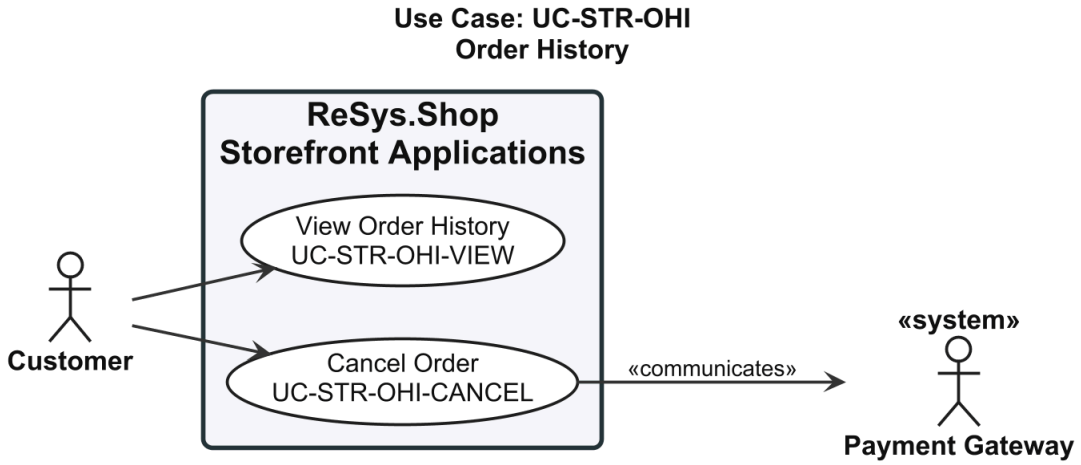

Figure 25: Use case diagram for Order History (UC-STR-OHI).

2.2.5.10 UC-STR-OHI: Order History

Table 44: Order History.

Field	Description
Use Case ID	UC-STR-OHI
Use Case Name	Order History
Primary Actor	Customer
Supporting Actors	Payment Gateway
Goal	View past orders and cancel pending orders.
Trigger	Customer navigates to order history in their account.
Preconditions	• Customer is authenticated.
Postconditions	• Complete order history visible. Cancelled orders release inventory and void payment.
Main Success Scenario	**View Order History** 1. Navigates to order history from account menu. 2. System displays past orders in reverse chronological order with pagination, showing order number, date, status, and total. 3. Applies optional date range or status filters. 4. Selects an order to view full detail: line items, shipping address, method, payment state, shipment state, and status timeline. , **Cancel Order** 1. Opens order detail from order history. 2. System displays order detail with current status and cancel action if cancellable. 3. Selects Cancel Order.

	4. System displays confirmation explaining inventory release and payment void. 5. Confirms. System releases reserved inventory, voids payment, transitions to cancelled, and sends confirmation.
Alternative Flows	A1. No orders: system displays message with prompt to browse catalog. A2. Order state changed since page loaded: system refreshes and informs cancellation unavailable.
Exception Flows	E1. Payment gateway unreachable (Cancel): system cancels order, releases inventory, queues void, notifies customer. E2. Retrieval failure (View): system displays error and offers retry.
Related Requirements	ORD-FR-07, ORD-FR-14

2.2.5.11 Payment Processing

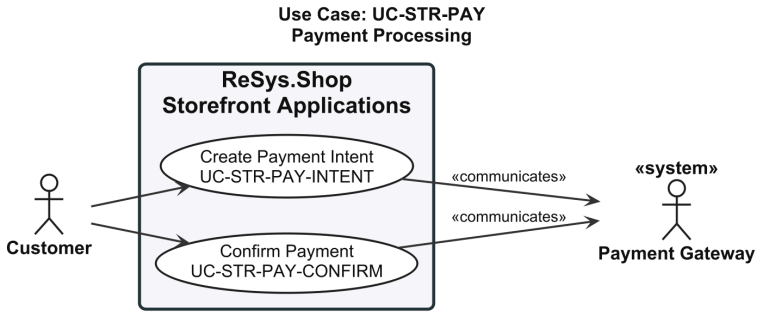

Figure 26: Use case diagram for Payment Processing (UC-STR-PAY).

2.2.5.12 UC-STR-PAY: Payment Processing

Table 45: Payment Processing.

Field	Description
Use Case ID	UC-STR-PAY
Use Case Name	Payment Processing
Primary Actor	Customer
Supporting Actors	Payment Gateway
Goal	Create and confirm payment for an order.
Trigger	Customer reaches the payment step during checkout.
Preconditions	• Customer is authenticated. • Checkout has reached payment step. • Order has a calculated total.
Postconditions	• Payment authorised. Order transitions to Confirmed state.

Main Success Scenario	**Create Payment Intent** 1. System displays payment step with order total and available payment methods. 2. Selects a payment method and enters payment details. 3. System submits payment details to gateway to create intent. 4. System receives confirmation intent is ready and displays review summary. , **Confirm Payment** 1. Reviews final order summary including line items, shipping, tax, and total. 2. Clicks Confirm Payment. 3. System submits payment intent confirmation to gateway. 4. System receives authorisation confirmation. 5. System updates payment state, transitions order to Confirmed, clears cart, and displays order confirmation.
Alternative Flows	A1. Invalid payment details: system returns gateway validation error, prompts correction. A2. Authorisation declined: system displays reason and offers retry with different method. A3. Additional authentication required: system redirects to authentication flow and resumes after.
Exception Flows	E1. Payment gateway unreachable: system displays error; checkout progress saved. E2. Payment confirms but order creation fails: system voids payment and notifies to retry. E3. Gateway timeout: system marks payment pending; advises checking order history; webhook updates state.
Related Requirements	PAY-FR-01, PAY-FR-02

2.2.5.13 Authentication

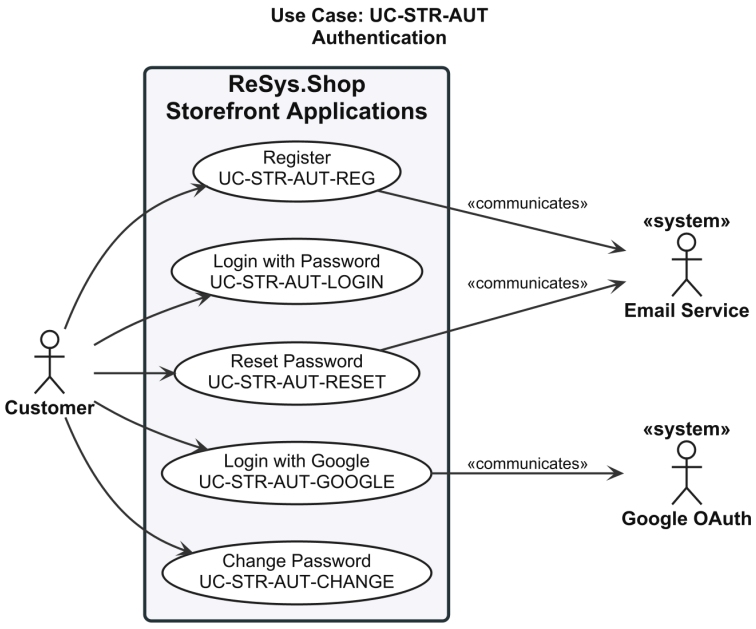

Figure 27: Use case diagram for Authentication (UC-STR-AUT).

2.2.5.14 UC-STR-AUT: Authentication

Table 46: Authentication.

Field	Description
Use Case ID	UC-STR-AUT
Use Case Name	Authentication
Primary Actor	Customer
Supporting Actors	Email Service, Google OAuth
Goal	Register, log in, and manage account credentials.
Trigger	Customer navigates to login or registration page.
Preconditions	• None (public access).
Postconditions	• Customer authenticated or credentials updated.
Main Success Scenario	**Register** 1. Navigates to registration page. 2. Enters email, password, and basic profile information. 3. Submits. System validates email not registered and password strength, creates account, sends verification. Confirms. , **Login with Password** 1. Navigates to login page and enters email and password. 2. Submits. System validates credentials, issues access and refresh tokens, associates guest cart if exists. Redirects to home page. , **Login with Google** 1. Clicks Login with Google on the storefront. 2. System redirects to Google's OAuth consent screen. 3. Grants consent. System exchanges code, looks up or creates account, issues tokens. Redirects to home page. , **Reset Password** 1. Clicks Forgot Password and submits registered email. 2. System generates time-limited reset token and sends via email. 3. Opens email, clicks reset link, enters new password. 4. Submits. System validates token, updates password, revokes all sessions. Confirms. , **Change Password** 1. Navigates to account security settings and selects Change Password. 2. Enters current password and new password. 3. Submits. System verifies current password, validates new password, updates, revokes all sessions except current. Confirms.

Alternative Flows	A1. Email already registered: system rejects and suggests login with password reset link. A2. Password does not meet strength requirements: system highlights requirements and prompts retry. A3. Invalid credentials: system rejects with generic message. A4. Account disabled: system rejects with message to contact support. A5. Reset token expired: system displays message and prompts new reset request. A6. Current password incorrect (Change): system rejects and prompts retry.
Exception Flows	E1. Verification or reset email fails to send: system creates account/request but logs failure internally. E2. Token issuance fails: system reports failure and suggests retry.
Related Requirements	IDN-FR-01, IDN-FR-02, IDN-FR-03, IDN-FR-08, IDN-FR-14

2.2.5.15 Session Management

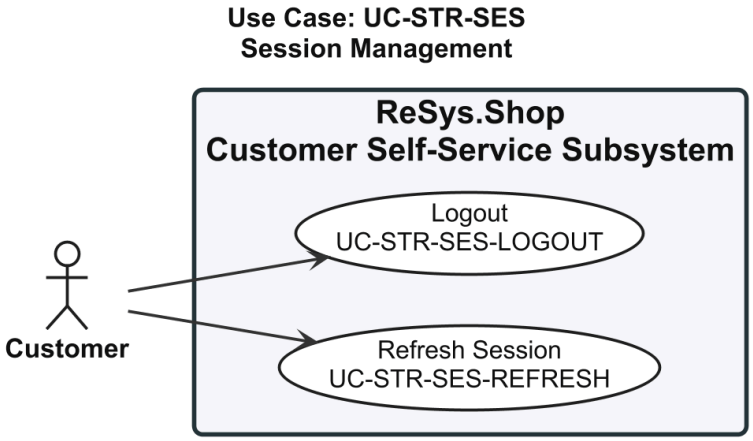

Figure 28: Use case diagram for Session Management (UC-STR-SES).

2.2.5.16 UC-STR-SES: Session Management

Table 47: Session Management.

Field	Description
Use Case ID	UC-STR-SES
Use Case Name	Session Management
Primary Actor	Customer
Supporting Actors	None
Goal	Maintain and terminate authenticated sessions.
Trigger	Client detects token expiry or customer initiates logout.
Preconditions	• Customer has an active session.

Postconditions	• Session extended or terminated.
Main Success Scenario	**Refresh Session** 1. Client detects access token will expire soon. 2. Client sends current refresh token to token endpoint. 3. System validates refresh token (not expired, not consumed). 4. System issues new access token with fresh expiry. 5. System issues new refresh token and invalidates previous (token rotation). 6. Client stores new token pair and continues session. , **Logout** 1. Clicks Logout in the storefront navigation. 2. System sends current refresh token for invalidation. 3. System invalidates the refresh token and removes the session cookie. 4. System redirects to the storefront home page.
Alternative Flows	A1. Refresh token expired: system rejects; client redirects to login. A2. Refresh token consumed (reuse detected): system revokes all tokens; client redirects to login with security notification. A3. Logout request fails due to network: system clears local tokens and cookies; session expires naturally.
Exception Flows	E1. Token issuance or invalidation fails: client retains existing pair if access token still valid; clears locally otherwise.
Related Requirements	IDN-FR-04, IDN-FR-05, IDN-FR-16

2.2.5.17 Profile and Preferences

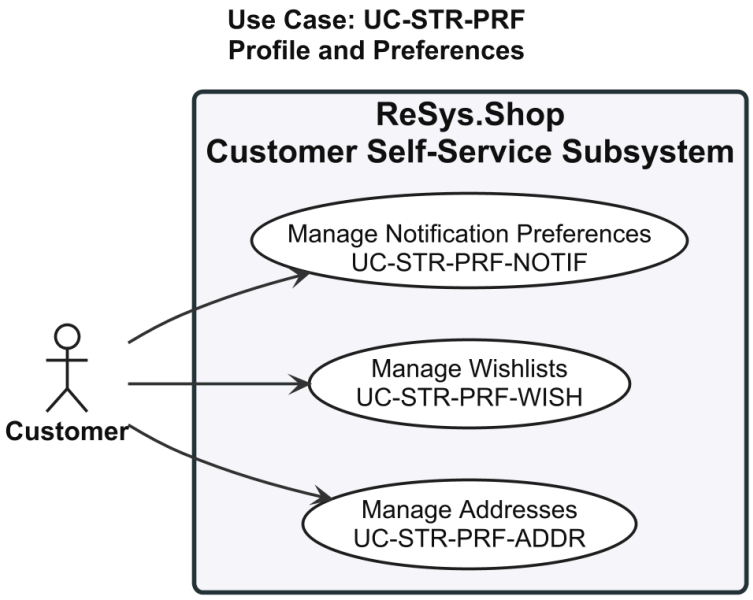

Figure 29: Use case diagram for Profile and Preferences (UC-STR-PRF).

2.2.5.18 UC-STR-PRF: Profile Management

Table 48: Profile Management.

Field	Description
Use Case ID	UC-STR-PRF
Use Case Name	Profile Management
Primary Actor	Customer
Supporting Actors	None
Goal	Manage shipping addresses, wishlists, and notification preferences.
Trigger	Customer navigates to account settings.
Preconditions	• Customer is authenticated.
Postconditions	• Profile data and preferences updated.
Main Success Scenario	**Manage Addresses** 1. Navigates to address book page. 2. System displays saved addresses with type labels and default indicators. 3. Creates a new address: enters name, street, city, postal code, country, and state. 4. Assigns address type (shipping, billing, or both) and optionally sets as default. 5. Optionally edits or removes existing addresses. 6. Saves. System validates required fields and address format, persists, and confirms. , **Manage Wishlists** 1. On a product detail page, clicks Add to Wishlist. 2. System displays dialog to select wishlist or create new. 3. Selects an existing wishlist or creates a new named wishlist; optionally adds a note. 4. System adds the product variant to the selected wishlist and confirms. 5. Navigates to wishlists section to view all with item counts and preview thumbnails. 6. Selects a wishlist to view items, remove items, rename, or delete the list. , **Manage Notification Preferences** 1. Navigates to notification preferences page. 2. System displays all notification categories with current per-channel opt-in/out status. 3. Toggles individual notification categories on or off per channel. 4. Saves. System persists and confirms.
Alternative Flows	A1. Same variant already in wishlist: system notifies and suggests adding note to existing entry. A2. Remove address used in active orders: system warns it is referenced by past orders (retained); allows soft-delete. A3. Opts out of all notifications: system warns important notifications

	also suppressed; asks confirmation. A4. Deletes wishlist: system asks for confirmation; list and items permanently removed.
Exception Flows	E1. Address validation fails for invalid country/state: system highlights mismatch and prevents save. E2. Product variant archived since added to wishlist: system displays with Unavailable label and remove suggestion. E3. Persistence failure: system reports failure and retains unsaved changes for retry.
Related Requirements	PRF-FR-01, PRF-FR-02, PRF-FR-03

2.2.6 System Use Cases

The System actor represents automated background processes that maintain data consistency, generate embeddings, and perform scheduled operations. Coordination relies on **Hangfire** [25] for job scheduling and **Redis** [22] for persistence.

2.2.6.1 Embedding Operations

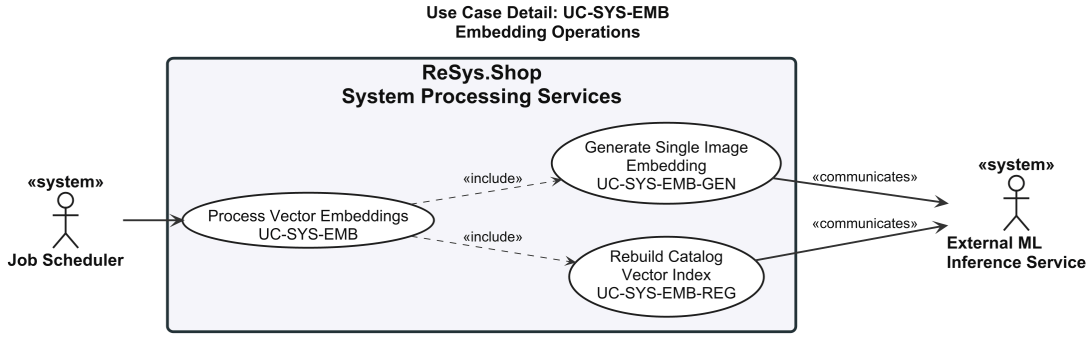

Figure 30: Use case diagram for Embedding Operations (UC-SYS-EMB).

2.2.6.2 UC-SYS-EMB: Embedding Operations

Table 49: Embedding Operations.

Field	Description
Use Case ID	UC-SYS-EMB
Use Case Name	Embedding Operations
Primary Actor	System
Supporting Actors	ML Service
Goal	Generate and regenerate image embeddings for product search.
Trigger	An image has been uploaded or embedding model configuration has changed.

Preconditions	• Images uploaded and stored. • ML service is operational.
Postconditions	• Embeddings stored in the index for visual search.
Main Success Scenario	**Generate Image Embeddings** 1. System detects a new unprocessed image in the queue. 2. System retrieves the image file from storage. 3. System preprocesses the image (resize, normalise) for model requirements. 4. System sends preprocessed image to ML service with configured model identifier. 5. ML Service computes the visual embedding vector and returns it with metadata. 6. System stores embedding in the index with image reference and metadata. 7. System marks the image as processed. , **Regenerate All Embeddings** 1. System detects change to embedding model configuration. 2. System validates new model is available via ML service health check. 3. System retrieves list of all product images requiring regeneration. 4. System sends each image to ML service using the new model. 5. ML Service computes new embedding and returns it. 6. System stores new embedding replacing previous and updates model metadata. 7. System reports completion with success/failure count summary.
Alternative Flows	A1. Image not accessible (deleted/corrupted): system marks as failed and records reason. A2. Multiple images queued: system processes sequentially, throttling to ML service capacity. A3. Triggered during peak traffic (Regenerate): system throttles to avoid impacting search performance. A4. No model change detected: system logs event and skips processing.
Exception Flows	E1. ML service returns error: system retries with exponential backoff; after max retries marks as failed. E2. ML service unreachable: system requeues image and triggers alert. E3. New model not available (Regenerate): system aborts regeneration; existing embeddings remain in use.
Related Requirements	CAT-FR-05, CAT-FR-15

The embedding generation process described in the main success scenario follows the pipeline illustrated in Figure 31: an image is authenticated, preprocessed through standardised transforms, passed through the selected model, and the resulting feature vector is serialised alongside performance metadata.

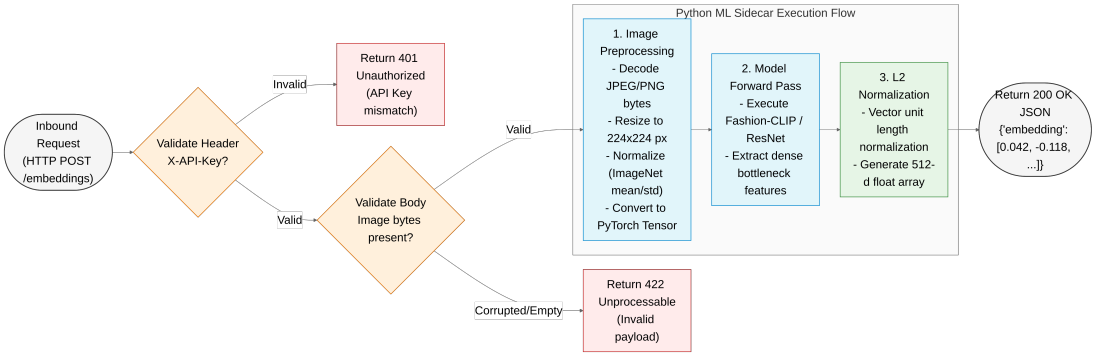

Figure 31: ML embedding pipeline: step-by-step flow from image upload to normalised vector response.

2.2.6.3 Background Maintenance

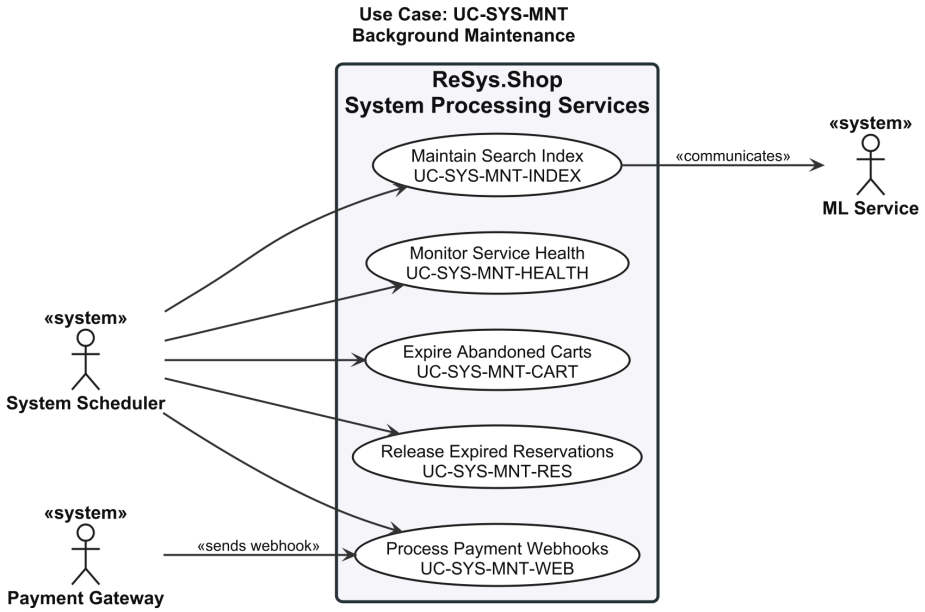

Figure 32: Use case diagram for Background Maintenance (UC-SYS-MNT).

2.2.6.4 UC-SYS-MNT: System Maintenance

Table 50: System Maintenance.

Field	Description
Use Case ID	UC-SYS-MNT
Use Case Name	System Maintenance
Primary Actor	System
Supporting Actors	ML Service, Payment Gateway

Goal	Perform scheduled and event-driven system maintenance tasks.
Trigger	Maintenance schedule fires or event-driven trigger occurs.
Preconditions	• Maintenance schedule is active. • Required services are operational.
Postconditions	• System health, data consistency, and index performance maintained.
Main Success Scenario	**Monitor Service Health** 1. System invokes health check endpoint on ML service at configured interval. 2. ML Service responds with current health status: healthy, degraded, or unhealthy. 3. System records health status and response time. 4. System updates service availability flag. If unhealthy, routes search to degraded path. , **Expire Abandoned Carts** 1. System executes scheduled job at configured time. 2. System queries for carts with no modification in past seven days. 3. For each abandoned cart, releases reserved inventory back to availability. 4. System deletes or marks abandoned cart for cleanup and logs counts. , **Release Expired Reservations** 1. System executes scheduled job at configured interval. 2. System queries for reservations with hold time exceeding configured expiry window (default 15 min). 3. For each expired reservation, releases reserved quantity back to available stock. 4. System marks reservation record as expired and logs counts. , **Process Payment Webhooks** 1. Payment Gateway sends signed webhook notification to system endpoint. 2. System validates cryptographic signature against configured gateway secret. 3. System extracts payment identifier, event type, and payload. 4. System verifies not a duplicate by checking idempotency key. 5. System updates local payment state and triggers any required order state transitions. 6. System returns success response to gateway. , **Maintain Search Index** 1. System executes scheduled job at configured interval. 2. System analyses current embedding index state: vector count, index size, recent query performance. 3. System determines whether maintenance is likely to improve performance.

	4. System performs maintenance (e.g. index rebuild, dead tuple removal, statistics update). 5. System verifies maintenance completed and logs execution with before/after metrics.
Alternative Flows	A1. No carts/items meet criteria: system logs zero and completes. A2. Duplicate webhook: system returns success without changes; duplicate logged. A3. Out-of-order webhook event: system logs anomaly and applies state change. A4. Index below maintenance threshold: system skips and logs decision with current metrics. A5. Active search queries during index maintenance: system performs lighter, non-blocking operations.
Exception Flows	E1. Health check endpoint does not respond: system marks unhealthy after timeout and triggers alert. E2. Database unavailable: system records failure and retries on next execution. E3. Webhook signature validation fails: system rejects with authentication error. E4. Index maintenance fails: system logs error and triggers alert; search continues with current index. E5. Index appears corrupted: system triggers full rebuild from stored embeddings.
Related Requirements	CAT-FR-06, CAT-FR-08, ORD-FR-03, INV-FR-07, PAY-FR-04

2.3 SYSTEM ARCHITECTURE & DESIGN

Six dimensions define the platform architecture, from service composition through domain modelling to database, API, and security layers. The design follows a service-oriented approach: a Vue 3 frontend, a .NET 10 modular monolith backend, and a Python FastAPI machine learning sidecar, each independently deployable.

- **System Overview.** Three services, eight bounded contexts, technology stack summary.
- **Domain-Driven Design.** Context map, aggregate roots with invariants, state machines.
- **C4 Architecture.** Context, container, and component-level structural views.
- **Database Design.** Per-context schemas, pgvector integration, core design decisions.
- **API Design.** Carter minimal APIs, MediatR CQRS, endpoint conventions.
- **Security Design.** JWT authentication, permission-based authorisation, defensive hardening.

2.3.1 System Overview

ReSys.Shop comprises three independently deployable **services**: a Vue 3 **TypeScript** frontend, a .NET 10 **ASP.NET Core** backend [19], and a Python **FastAPI** machine learning sidecar running **PyTorch** [23] models. Table 51 summarises their technology stacks and responsibilities.

Table 51: System services and their technology stacks. Each service communicates through well-defined HTTP contracts.

Service	Technology Stack	Responsibilities
Vue Frontend	Vue 3 + TypeScript + Vite	• Customer storefront (Nuxt UI) • Administrator dashboard (PrimeVue) • Pinia state management • Image upload and visual search UI
.NET Backend	.NET 10 + ASP.NET Core + Carter + EF Core	• REST API endpoints via Carter minimal APIs • Business logic via MediatR CQRS pattern • PostgreSQL persistence with pgvector vector search • JWT authentication and RBAC authorisation
Python ML	Python 3.12 + FastAPI + PyTorch	• Fashion-CLIP and other embedding model inference • Vector embedding generation from product images • Multi-model support with lazy-loading strategy

Internally, the backend is partitioned into eight **bounded contexts** following **Domain-Driven Design** (DDD) principles. Each context owns a dedicated database schema and communicates with others exclusively through **MediatR** in-process dispatch – there are no direct namespace references between business modules. Table 52 lists each context, its aggregate root, and key domain entities.

Table 52: Bounded contexts with aggregate roots and key domain entities. Each context owns its database schema and communicates through MediatR dispatch only.

Bounded Context	Aggregate Root	Key Domain Entities
Catalog	Product	Variant, VariantImage, OptionType, OptionValue, Classification, Taxonomy, Taxon
Ordering	Order	LineItem, Adjustment, Shipment
Payment	PaymentIntent	PaymentCapture
Inventory	StockItem	StockLocation, StockMovement, StockReservation
Identity	User	Role, UserRole, RefreshToken, UserLogin
Profile	UserProfile	Address, Wishlist
Shipping	ShippingMethod	ShippingRate, ShippingZone

Bounded Context	Aggregate Root	Key Domain Entities
Location	Country	State

This federated structure enables independent evolution of each domain while the **modular monolith** deployment model [15] avoids the operational overhead of distributed microservices. Domain modelling is detailed in Section 2.3.2.

2.3.2 Domain-Driven Design

The backend architecture is structured around eight **bounded contexts** adhering to **Domain-Driven Design** (DDD) principles. Each context manages explicit aggregate boundaries, enforces domain invariants, and uses a shared ubiquitous language across its domain operations.

- **Bounded Context Map:** Eight domain contexts operating under a Conformist integration pattern with Published Language contracts.
- **Aggregates and Invariants:** Four architecturally critical aggregate roots: `Product`, `Order`, `PaymentIntent`, and `StockItem`.
- **Ubiquitous Language:** Formally defined terminology establishing clear conceptual boundaries across distinct contexts.
- **State Machines:** Explicit transition boundaries governing forward-only checkout progression and local-to-gateway payment lifecycles.

2.3.2.1 Bounded Context Map

The platform is partitioned into eight **bounded contexts** along business capability boundaries. Each context independently owns its state model, business logic, and vocabulary: a **Variant** in Catalog represents a sellable entity with pricing and SKU attributes, whereas a **LineItem** in Ordering references that same variant strictly from a purchasing context.

Integration follows a **Conformist** pattern. All contexts inherit common primitive abstractions from the Core Shared Kernel (`Result<T>`, `ICommand`, `IQuery`, and entity base types). Context-to-context communication relies exclusively on **MediatR** `ISender` in-process dispatch: a context emits a command, query, or event notification, and receiving contexts process the request without establishing direct compile-time project references. This in-process model preserves modular isolation without introducing distributed network latency.

Figure 33 illustrates the eight contexts and their inter-module communication pathways. The **Published Language** defines shared identifiers and value objects exchanged between contexts. Table 53 details each context's business capabilities and public data boundaries.

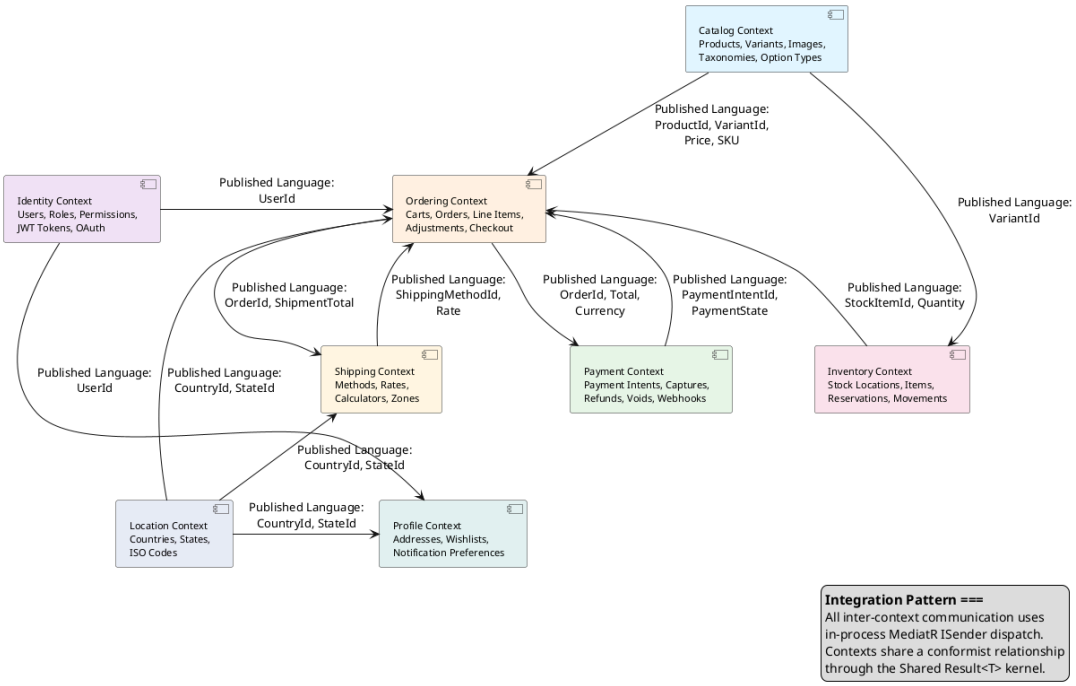

Figure 33: Bounded Context Map showing the eight business contexts and the Published Language identifiers exchanged between them. All integration uses in-process MediatR dispatch; no context directly references another context's namespace.

Table 53: Bounded context responsibilities and Published Language identifiers. The Published Language column lists the value types that other contexts may reference by identifier only, never by importing the source context's namespace.

Context	Business Responsibility	Published Language
Catalog	• Product lifecycle: creation, update, archiving, and fashion metadata • Variants with SKUs, barcodes, dimensions, and independent pricing • Image management triggering automated vector embedding generation • Hierarchical taxonomy classification structures	`ProductId`, `VariantId`, `Sku`, `Price`, `Slug`
Ordering	• Customer purchasing workflow from cart through checkout completion • Cart management with automated 7-day inactivity purging • Forward-only checkout state machine execution • Line item capture with locked price snapshots and fee adjustments • Pre-confirmation order cancellation handling	`OrderId`, `OrderNumber`, `Total`, `Currency`, `CheckoutState`

Context	Business Responsibility	Published Language
Payment	• Payment intent lifecycles: intent creation, authorization, capture, refund, void • Dual gateway abstractions (Stripe production and Bogus test environments) • Local payment state mirroring for offline operation and resilience	`PaymentIntentId`, `PaymentState`, `Amount`
Inventory	• Warehouse stock balance tracking across locations • Temporary inventory reservations for active checkouts • Append-only audit logging for stock adjustments • Inter-warehouse transfer processing	`StockItemId`, `QuantityOnHand`, `QuantityReserved`
Identity	• JWT-based authentication with single-use refresh token rotation • Automated refresh token reuse detection and session revocation • Role-Based Access Control enforcing `domain:category:action` claims • Anonymous guest session management	`UserId`, `Email`, `PermissionClaim`
Profile	• Customer address book management for billing and shipping • Personalized wishlists for product bookmarking • Communication preferences across notification channels	`ProfileId`, `AddressId`
Shipping	• Delivery method definition and management • Geographic shipping rate evaluation • Weight- and order-value-based cost calculation rules	`ShippingMethodId`, `Rate`
Location	• Reference data for countries and regional states via ISO 3166 standards • Shared reference lookup for Shipping, Profile, and Ordering domains	`CountryId`, `StateId`, `IsoCode`

2.3.2.2 Aggregates and Invariants

An aggregate root defines a transactional consistency boundary, managing child entities and enforcing core business rules. ReSys.Shop uses a pragmatic DDD strategy without unnecessary base class overhead or forced global domain event dispatches.

- **Product (Catalog Root):** Controls product families, variants, media assets, options, and taxonomy mappings. Enforces catalog-wide URL slug uniqueness, mandates valid option types prior to variant instantiation, and ensures exactly one master variant exists per product family. Variant images store 512-dimensional vector embeddings targeting high-dimensional visual search.
- **Order (Ordering Root):** Coordinates purchasing pipelines from cart assembly to order completion. Locks historical line-item price snapshots at checkout initialization to protect completed orders from subsequent catalog price updates. Enforces the structural balance invariant

$$\text{Total} = \text{ItemTotal} + \text{AdjustmentTotal} + \text{ShipmentTotal}$$

  . Checkout state progression is strictly forward-only; completed orders remain immutable except through formal administrative cancellations.
- **PaymentIntent (Payment Root):** Models transactional authorization across `Pending`, `RequiresAction`, `Processing`, `Succeeded`, `Canceled`, and `Failed` operational states. Enforces debit limits ensuring cumulative captures cannot exceed initial authorized amounts. Maintains an isolated internal state machine mirroring external gateway responses for testability and offline querying.
- **StockItem (Inventory Root):** Tracks real-time on-hand and reserved stock per warehouse location. Enforces non-negative balance constraints

$$\text{QuantityOnHand} \geq 0$$

  , writing backorders to an isolated ledger. All stock adjustments write to an append-only `StockMovement` ledger to ensure absolute auditability.

2.3.2.3 Ubiquitous Language Glossary

The ubiquitous language establishes a common domain vocabulary across software design and business rules. Table 54 defines key domain terms.

Table 54: Ubiquitous Language Glossary across domain contexts.

Term	Context	Definition
Product	Catalog	Core fashion entity containing shared catalog metadata (description, slug, taxonomy). Prices and SKUs belong exclusively to Variants.
Variant	Catalog	Purchasable unit containing SKU, barcode, price, physical dimensions, and stock management flags.
Master Variant	Catalog	The primary variant displayed on product listing components. Exactly one master variant is required per product family.
Option Type	Catalog	Reusable attribute categories (e.g., Color, Size) shared across catalog entities.

Term	Context	Definition
Taxonomy / Taxon	Catalog	Hierarchical classification structures (Taxonomy) consisting of nested category nodes (Taxons).
Cart	Ordering	Temporary container holding intended purchases. Purged after 7 consecutive days of inactivity.
Line Item	Ordering	Cart or order entry linking a specific variant to a quantity and a locked price snapshot.
Checkout State	Ordering	Sequential stages: Address → Delivery → Payment → Confirm → Complete. Enforces forward-only progression.
Payment Intent	Payment	Tracks authorization states and transaction steps across payment gateway interactions.
Payment Capture	Payment	Full or partial monetary debit executed against an authorized payment intent.
Stock Item	Inventory	Real-time warehouse inventory record tracking available on-hand and reserved stock quantities.
Stock Movement	Inventory	Immutable audit entry recording quantity deltas, updated balances, operator identity, and change reasons.
Refresh Token	Identity	Single-use token used to generate short-lived JWT access tokens. Reuse attempts trigger session revocation.

2.3.2.4 State Machines

Explicit state machines inside domain entities govern checkout and payment workflows, validating transitions prior to database persistence.

2.3.2.4.1 Order Checkout State Machine

Checkout advances strictly through five stages: Address, Delivery, Payment, Confirm, and Complete (Figure 34). Users can cancel at any pre-completion stage without financial impact. Reaching `Complete` finalizes the order, reserves inventory, and locks order data against further edits.

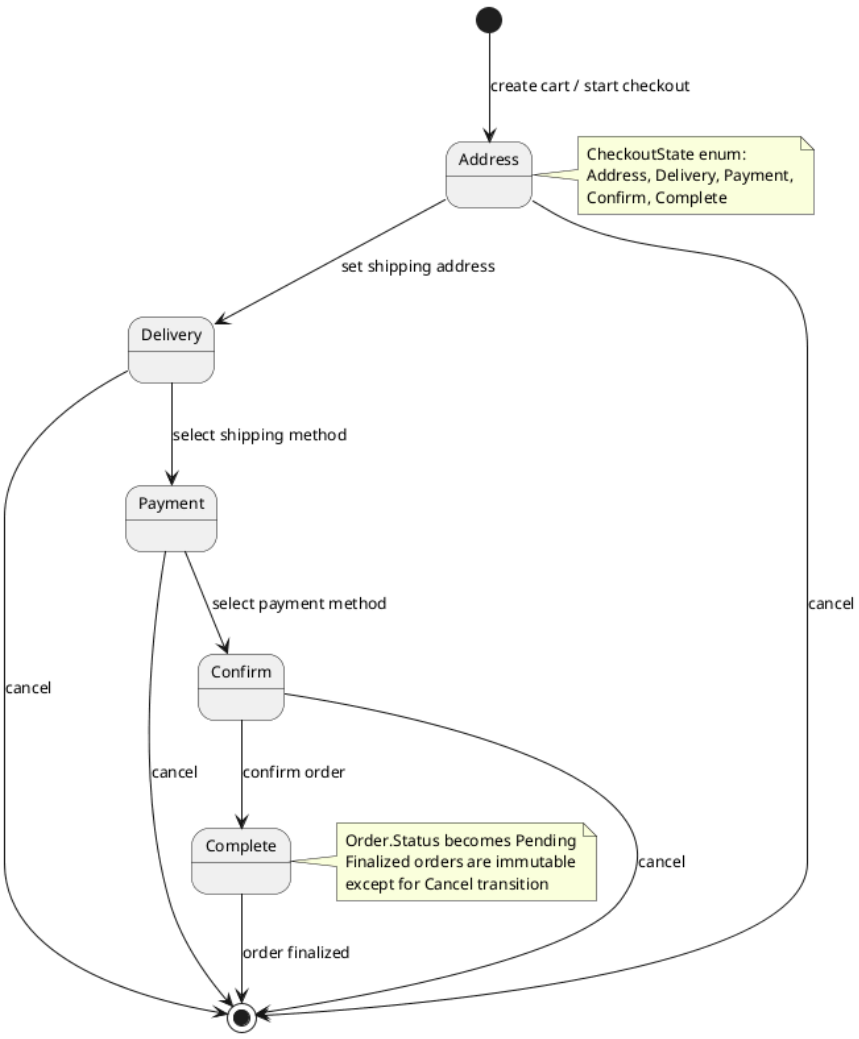

Figure 34: Order checkout state machine showing forward-only stages and cancellation paths.

2.3.2.4.2 Payment Intent State Machine

Payment intents initialize as `Pending` and transition to `RequiresAction` (for 3D Secure) or `Processing` (Figure 35). Final outcomes resolve to `Succeeded` or `Failed`. Successful authorizations can be captured or refunded. Mirroring gateway states locally enables offline test execution and resilience during network drops.

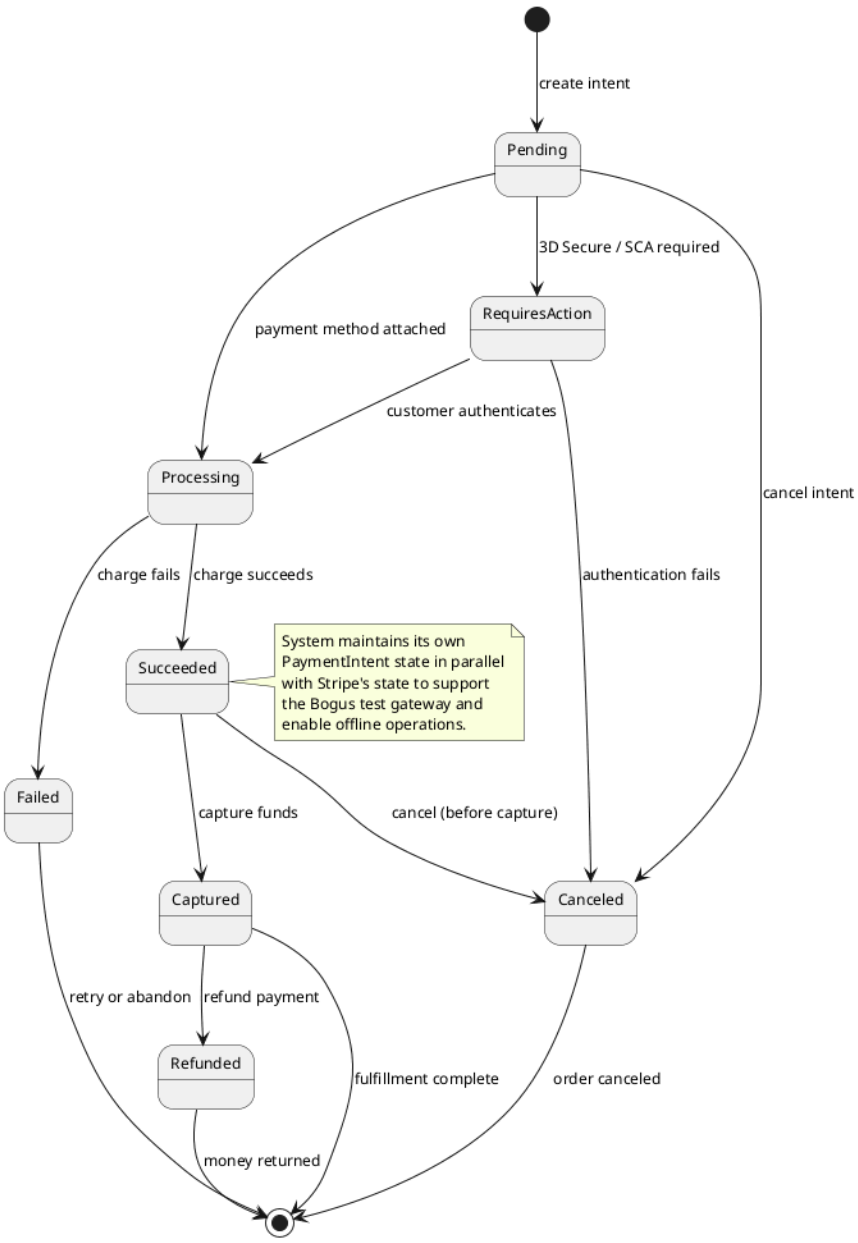

Figure 35: Payment intent lifecycle and terminal states.

2.3.3 C4 Architecture

The C4 model structures software architecture across four abstraction levels: system context, container, component, and code. This section presents the first three C4 levels for ReSys.Shop alongside a deployment view, omitting code-level structures addressed in later implementation sections.

2.3.3.1 System Context

The system context positions ReSys.Shop within its operational environment, defining user roles and external dependencies (Figure 36).

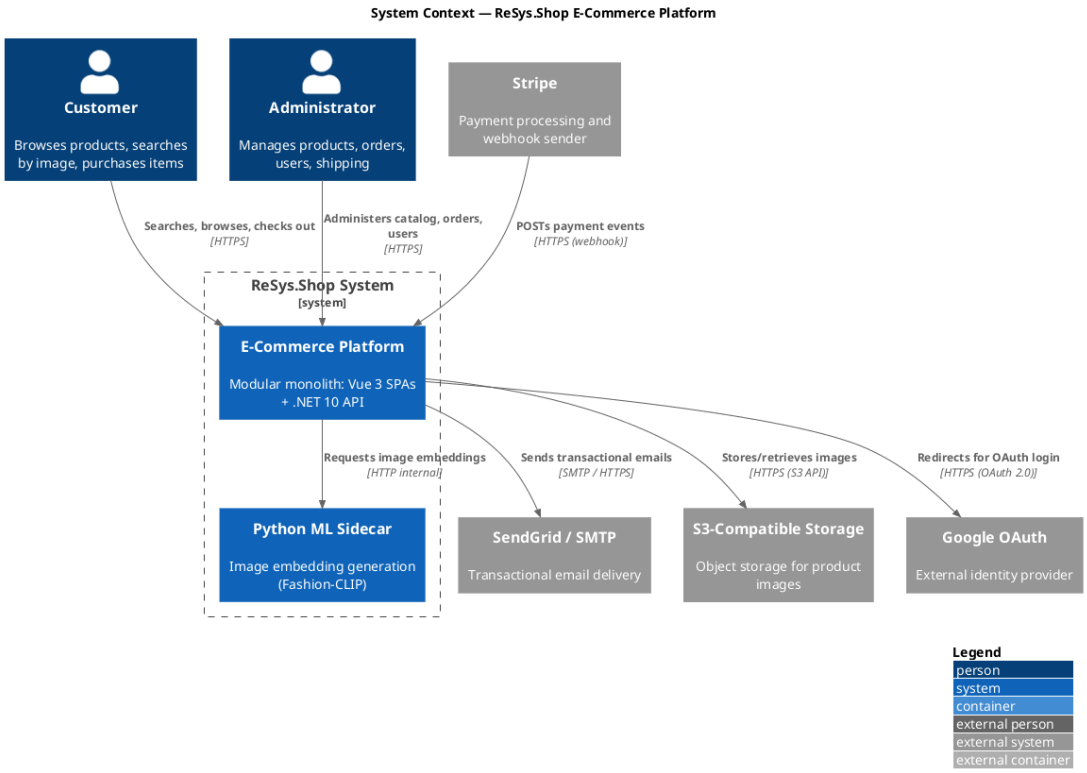

Figure 36: System Context diagram: ReSys.Shop system boundary showing user roles and external integration dependencies.

The platform interacts with two human user groups:

- **Customers:** Browse the catalog, run visual and keyword searches, manage carts, and complete checkouts via the Vue 3 storefront SPA.
- **Administrators:** Manage products, process orders, track inventory, and administer user accounts via the Vue 3 admin SPA.

Five external integrations support platform operations:

- **Stripe:** Manages payment intent lifecycles and sends webhook notifications validated locally via Stripe signature verification.
- **SendGrid:** Dispatches transactional emails including order confirmations, password reset links, and shipping updates.
- **S3-Compatible Storage:** Persists product assets uploaded through the admin interface.
- **Google OAuth:** Offers customer single sign-on authentication.
- **Python ML Sidecar:** Generates image embeddings for visual search within the Aspire orchestration boundary.

2.3.3.2 Container

The container view decomposes ReSys.Shop into six standalone deployable processes and data stores (Figure 37).

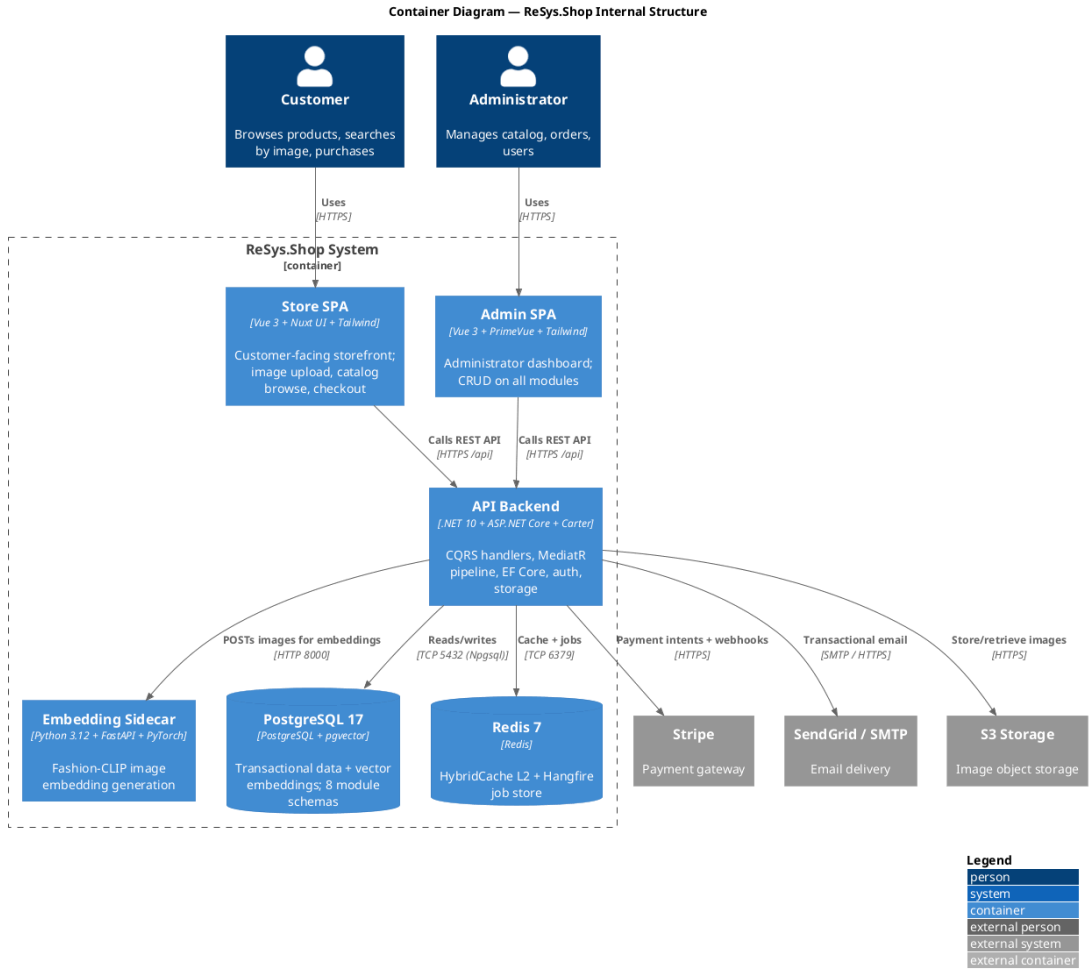

Figure 37: Container diagram showing Vue 3 SPAs, .NET 10 API backend, Python ML sidecar, PostgreSQL with pgvector, and Redis.

The deployable units comprise:

- **Store & Admin SPAs:** Vue 3 single-page applications served as static assets, interacting with the backend strictly over HTTPS REST endpoints.
- **API Backend:** A .NET 10 (ASP.NET Core) application executing domain logic across eight modules via Carter minimal APIs and MediatR CQRS pipelines.
- **Embedding Sidecar:** A Python 3.12 FastAPI service loading ML models into GPU/CPU memory to generate image embeddings on demand over internal HTTP.
- **PostgreSQL 17 (with pgvector):** Stores relational domain schemas and high-dimensional vector embeddings for visual similarity search.
- **Redis 7:** Serves as an L2 distributed cache for `HybridCache` and a persistent job store for Hangfire background processing.

Communication is strictly centralized through the API Backend. SPAs never query databases or external APIs directly, enforcing all security boundaries server-side.

2.3.3.3 Component

The component view details the internal structure of the API Backend container (Figure 38).

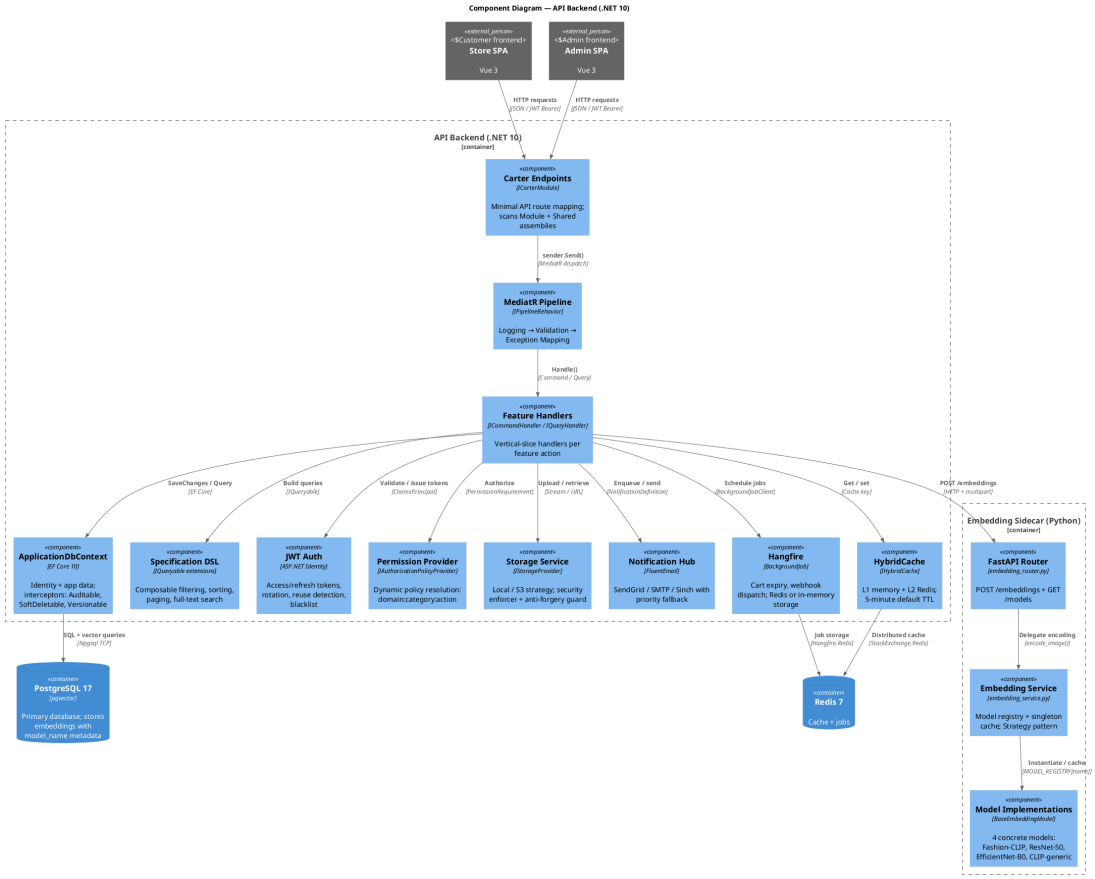

Figure 38: Component diagram detailing the API Backend architecture and the three-layer Python ML sidecar.

HTTP requests enter through Carter minimal API modules, which validate parameters and dispatch commands or queries via MediatR's `ISender`. MediatR pipelines wrap execution in logging, FluentValidation checks, and global exception-handling behaviors.

Handlers delegate infrastructure tasks to eight internal components:

1. **ApplicationDbContext:** EF Core 10 context managing interceptors for auditing, soft-deletes, and optimistic concurrency.
2. **Specification DSL:** Composable `IQueryable` extensions for filtering, sorting, paging, and full-text search.

3. **Identity Provider:** ASP.NET Identity with JWT management handling access/refresh token rotation and revocation.
4. **Dynamic Permission Provider:** Resolves `{domain}:{category}:{action}` policy claims dynamically at runtime.
5. **Storage Service:** Provides interchangeable local or S3-compatible file storage with upload validation.
6. **Notification Hub:** Routes email (SendGrid/SMTP) and SMS (Sinch) with fallback routing.
7. **Hangfire Engine:** Executes background tasks including cart expiration, webhook dispatch, and maintenance jobs.
8. **HybridCache:** Combines L1 in-memory caching with L2 Redis caching for cross-instance consistency.

The Python ML Sidecar uses a three-layer layout: a FastAPI router for request validation, a singleton model registry for lazy loading, and an interchangeable strategy interface supporting Fashion-CLIP, ResNet-50, EfficientNet-B0, and standard CLIP.

2.3.3.4 Deployment

The deployment diagram illustrates the production infrastructure topology (Figure 39).

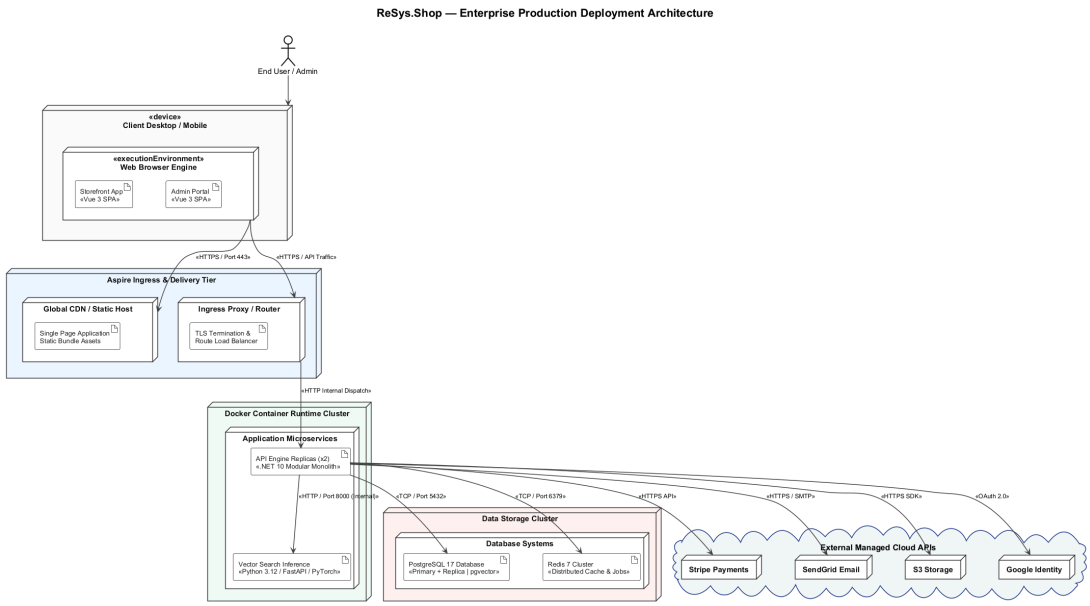

Figure 39: Deployment topology showing containerized orchestration under .NET Aspire with horizontal scaling.

All services run as Docker containers orchestrated by .NET Aspire for service discovery, configuration injection, and health monitoring. Static Vue SPA bundles deploy to a CDN or reverse proxy.

The API Backend scales horizontally across container replicas sharing PostgreSQL and Redis. The stateless ML sidecar scales independently, serving vector generation requests from any API instance. PostgreSQL runs a primary instance for writes alongside read replicas for analytical queries, with `pgvector` enabled across all nodes. External secrets and API keys are injected via Aspire configuration environments at runtime.

2.3.4 Database Design

The ReSys.Shop database consists of a single PostgreSQL 17 instance partitioned into per-context schemas. Each schema is owned by a bounded context and managed via Entity Framework Core migrations.

2.3.4.1 Schema Organisation

Each of the eight bounded contexts manages its own dedicated database schema:

- **Catalog:** `products`, `variants`, `variant_images`, `option_types`, `option_values`, `taxonomies`, and `taxons`.
- **Ordering:** `orders`, `line_items`, `adjustments`, `shipments`, and `inventory_units`.
- **Payment:** `payment_intents`, `payment_captures`, and `payment_logs`.
- **Inventory:** `stock_locations`, `stock_items`, `stock_movements`, and `stock_reservations`.
- **Identity:** `users`, `roles`, `user_roles`, `refresh_tokens`, and `external_logins` (built on ASP.NET Identity Core).
- **Profile:** `user_addresses`, `wishlists`, and `notification_preferences`.
- **Shipping:** `shipping_methods`, `shipping_rates`, and `shipping_zones`.
- **Location:** `countries` and `states` reference data.

Cross-context relationships use identifier references (UUIDs) without database-level foreign key constraints. An order references `UserId` and `VariantId` as loose attributes, maintaining logical module isolation while avoiding distributed transaction overhead.

2.3.4.2 Core Entity-Relationship Model

Figure 40 illustrates the entity-relationship model across all eight bounded contexts. Dotted lines indicate cross-context identifier references.

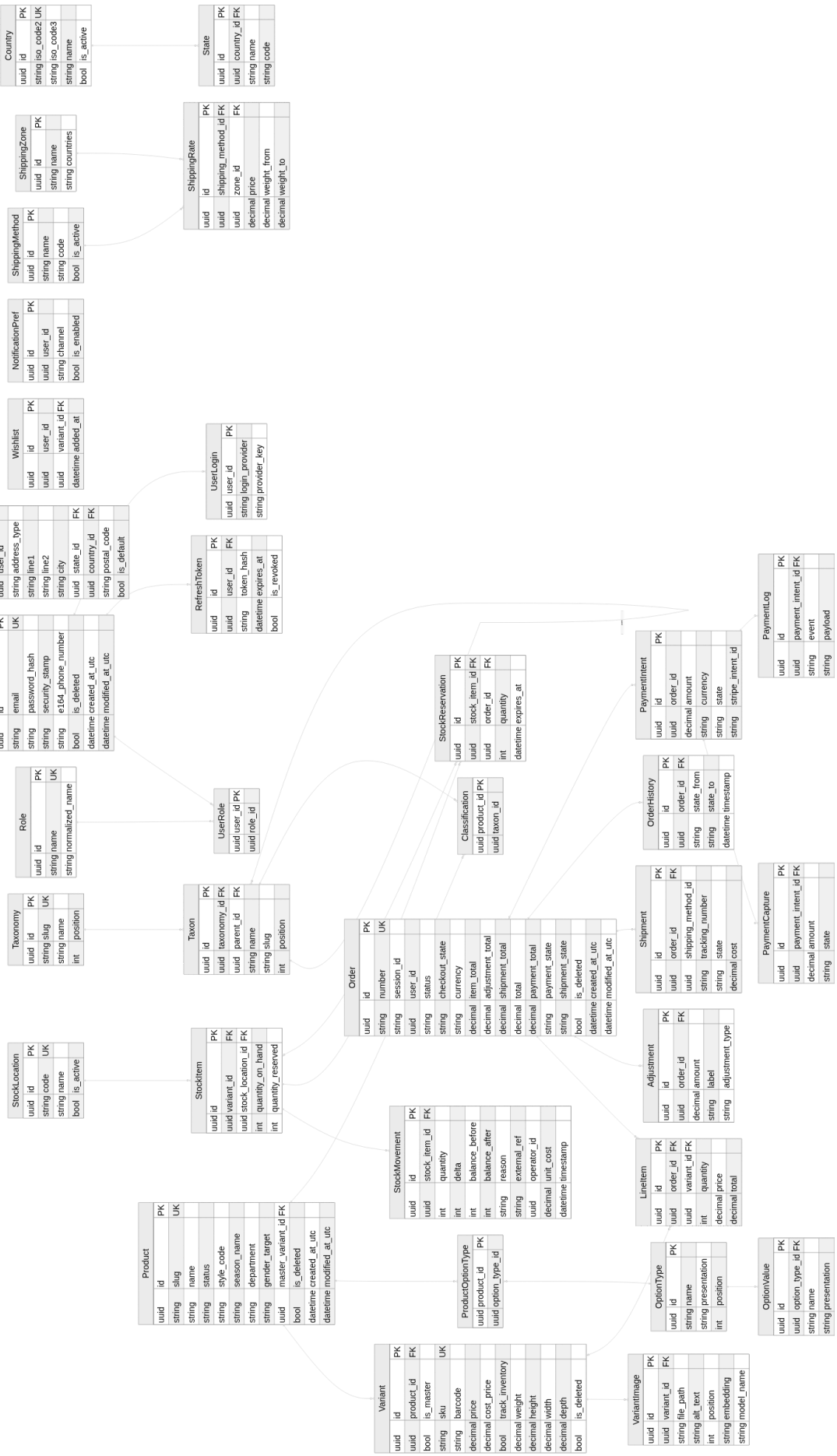

Figure 40: Core entity-relationship model across eight bounded contexts.

The **Catalog** domain centers on `Product`, which links one-to-many with `Variant`. Each variant stores SKU, pricing, and inventory tracking flags, with one designated as the master variant. `VariantImage` records hold image paths and an optional `vector(512)` embedding column. `Taxonomy` and `Taxon` trees manage hierarchical product classification via self-referencing foreign keys.

The **Ordering** domain centers on `Order`, linking one-to-many with `LineItem`. Line items reference specific variants and capture price snapshots at purchase time to decouple order history from catalog updates. `UserId` references support both registered user checkouts and nullable guest sessions.

2.3.4.3 pgvector Integration

PostgreSQL's **pgvector** extension [13] executes vector similarity searches directly within the relational engine. The `variant_images` table stores feature vectors in an `embedding` column defined as `vector(512)`. Two approximate nearest neighbour (ANN) index types support distinct operational profiles:

- **HNSW (Hierarchical Navigable Small World)** [12]: Constructs a multi-layer graph where nodes connect locally and hierarchically.
  - **Query Speed:** Logarithmic search complexity (< 2 ms on 100,000-vector corpora).
  - **Build Cost:** Memory-intensive construction, ideal for read-heavy production workloads.
  - **Recall:** Delivers 95–99% recall at standard configurations (`ef_search = 100`).
- **IVFFlat (Inverted File with Flat Compression):** Partitions vector space into k-means clusters for targeted inverted list searches.
  - **Query Speed:** Slightly higher latency than HNSW due to cluster probe scans.
  - **Build Cost:** Rapid index creation with minimal memory overhead, optimal for frequent rebuilds or constrained hardware.
  - **Recall:** Achieves comparable recall when configured with optimal probes (`lists = sqrt(n)`, `probes = sqrt(lists)`).

The platform defaults to **HNSW** using cosine distance to meet the sub-second CBIR latency target (NFR-01a). **IVFFlat** serves as the fallback for local or GPU-constrained environments prioritizing rapid index rebuilds over raw query throughput.

- **Cosine Distance:** Query operator (`<=>`) measures angular distance. Results rank by similarity score $1 - \text{cosine\_distance}$, filtering against a configurable default threshold (0.70).

- **Model Isolation:** Every record includes a `model_name` string (e.g., `"Fashion-CLIP"`). Search queries filter by active model name to enforce strict mathematical alignment across visual embeddings.

2.3.4.4 Key Design Decisions

The data layer adheres to five global design rules:

- **UUID Primary Keys:** Uses UUIDs across all tables to enable safe client-side key generation and prevent auto-increment sequential enumeration leakage.
- **Soft Deletion:** Implements an `IsDeleted` boolean column filtered globally by EF Core, preserving referential integrity for historical orders.
- **Audit Columns:** `CreatedAtUtc` and `ModifiedAtUtc` timestamps populate automatically via EF Core save interceptors.
- **Composite Indexes:** High-frequency access paths use targeted composite indexes, such as `(UserId, Status)` and `(SessionId, Status)` on `orders`.
- **Variable Vector Dimensions:** `pgvector` columns support variable dimensionalities per model architecture: 384 (DINOv2-S), 512 (Fashion-CLIP), 768 (DINOv2-B), 1280 (EfficientNet-B0), and 2048 (ResNet-50). Embeddings from different models are indexed independently per dimension.

2.3.4.5 Per-Context Schema Description

- **Identity Schema:** Stores Argon2-hashed credentials, security stamps for session revocation, single-use refresh tokens, and linked OAuth logins.
- **Catalog Schema:** `Product` stores fashion-specific metadata (style codes, composition, department). `Variant` manages SKU, pricing, and physical dimensions. `OptionType` and `OptionValue` implement an EAV pattern for dynamic product attributes. `Taxon` records use a nested set model for efficient subtree retrieval.
- **Ordering Schema:** Financial attributes use 18-digit precision and 2-decimal scale (`decimal(18,2)`). Orders track item, adjustment, and shipping subtotals separately. `OrderHistory` provides an append-only state transition audit log.
- **Inventory Schema:** Tracks `QuantityOnHand` and `QuantityReserved` per variant-location pairing using PostgreSQL's `xmin` system column for optimistic concurrency control. `StockMovement` serves as an immutable transaction ledger recording balance deltas, unit costs, and operational user IDs.

2.3.5 API Design

The ReSys.Shop API exposes a RESTful interface on .NET 10 minimal APIs via **Carter** modules, dispatched through the **MediatR** CQRS pattern [18]. The platform registers 257 endpoints across nine business modules (184 admin, 73 storefront).

2.3.5.1 API Architecture

The API host acts as a thin orchestration boundary with zero business logic. Each request follows a standard path:

- **Endpoint Reception:** A Carter endpoint receives the HTTP request, extracts parameters, constructs a MediatR command or query, and dispatches via `ISender`.
- **Pipeline Validation:** FluentValidation behaviors execute before handler processing. Validation failures halt the pipeline and return HTTP 400 with field-level details.
- **Handler Execution:** Bounded context handlers process requests within domain isolation, operating on application models without HTTP dependencies.
- **Response Mapping:** Handlers return `Result<T>`. Endpoints map success to HTTP 200, 201, or 204, and domain errors to RFC 7807 Problem Details.

2.3.5.2 Endpoint Organisation and Route Structure

Endpoints follow a two-dimensional URL convention: `/api/{module}/{surface}/{resource}`.

- **`module`:** The owning bounded context (`catalog`, `ordering`, `payment`, `inventory`, `identity`, `profile`, `shipping`, `location`).
- **`surface`:** `storefront` (customer-facing) or `admin` (administrative operations).
- **`resource`:** The domain resource and sub-action (e.g., `products`, `cart/checkout`, `search-by-image`).

This structure provides self-documenting URLs, surface-level authorisation boundaries (`admin` routes enforce administrator policies globally), and independent module evolution without breaking adjacent contexts.

2.3.5.3 API Execution Layout

The repository follows the modular monolith pattern: a thin API host, eight bounded context modules, shared building blocks, and a Python ML sidecar.

- **Apps:** `ReSys.Shop.Api` (.NET 10 Carter host), `ReSys.Shop.Store` (Vue 3 storefront), `ReSys.Shop.Admin` (Vue 3 dashboard).
- **Modules:** Catalog, Ordering, Payment, Inventory, Identity, Profile, Shipping, Location – each with `Features/`, `Domain/`, and `Persistence/` layers.
- **BuildingBlocks:** `SharedKernel` (Result, Entity, domain events), `Persistence` (DbContext, interceptors, migrations), `Security` (authorisation policies, JWT).
- **Sidecar:** `ReSys.Shop.Ml` (Python FastAPI) with `api/` (embedding routes), `core/` (strategy registry), `models/` (Fashion-CLIP, DINOv2, ResNet, CLIP encoders).
- **Infrastructure:** `tests/` (unit, integration, benchmarks), `AppHost/` (.NET Aspire orchestrator).

2.3.5.4 Full API Endpoint Contract

Table 55 summarises the API surface verified against the 257 registered Carter endpoints in the codebase.

Table 55: ReSys.Shop API endpoint contract. The platform exposes 257 Carter endpoints across nine business modules (184 admin, 73 storefront). Admin routes provide full CRUD with activate/deactivate and sync/assign/revoke patterns. Storefront routes expose read-optimised public surfaces for product discovery, cart, checkout, and account management.

Module	Admin Routes	Storefront Routes	N
Catalog	Products CRUD, variants, variant images, option types/values, taxonomies, taxons, taxon rules, classifications, pricing, dashboard	Product listing/search, product detail by slug, availability, related/similar products, CBIR search-by-image, taxonomy tree, taxon browsing, option types, image display	77
Identity	Users CRUD, user roles/permissions, roles CRUD, role permissions, permissions catalogue	Password/Google login, register, logout, session refresh, email confirm/change, password change/forgot/reset	37
Ordering	Orders CRUD, line items, order status/cancel/complete/approve/resume, shipping/billing address, shipping method, dashboard	Cart CRUD, cart items add/update/remove, cart associate, empty, checkout, validate, shipping rate, customer orders list/detail/cancel	34
Inventory	Stock locations CRUD, stock items CRUD, bulk adjust/import, restock, low stock, summary, reservations, transfers, movements, dashboard	Variant availability, cart reserve/release/list	32
Profile	Profiles CRUD, addresses CRUD	Profiles, addresses CRUD, notification preferences, wishlists CRUD with items	26
Location	Countries CRUD (by ID or ISO code), states CRUD (by ID or ISO code)	Countries browse, states browse (by ID or ISO code)	18
Payment	Payment methods CRUD, activate/deactivate, payments list/detail, capture/void/refund	Create payment intent, confirm payment, available methods, setup intent, Stripe webhook	17
Shipping	Shipping methods CRUD, activate/deactivate, shipping rates CRUD	Available methods, calculate cost, rates list	15
Dashboard	Aggregated metrics: sales, inventory, catalog, activity	–	1

2.3.5.5 Error Handling and Response Standards

All API endpoints conform to RFC 7807 Problem Details for error responses:

- **HTTP 400:** FluentValidation failures with field-level error mappings.
- **HTTP 401:** Missing or expired JWT in `Authorization: Bearer` header.
- **HTTP 403:** Insufficient role or permission scope (e.g., customer accessing admin route).
- **HTTP 404:** Domain entity or route resolution failure.
- **HTTP 409:** Optimistic concurrency control failure (e.g., inventory reservation race via PostgreSQL `xmin`).
- **HTTP 500:** Global exception middleware prevents stack trace leakage while logging full detail to telemetry.

2.3.6 Security Design

The ReSys.Shop security framework operates across three distinct operational layers: authentication, claim-based authorization, and defense-in-depth infrastructure.

2.3.6.1 Authentication and Session Management

- **Token Pair Generation:** Authenticated users (via email/password or Google OAuth) receive a 15-minute JWT access token and a server-stored refresh token in PostgreSQL.
- **Single-Use Rotation:** Exchanging an expired access token consumes the current refresh token and issues a new pair.
- **Breach Detection:** Re-submitting a previously consumed refresh token flags potential token interception, immediately revoking all active refresh tokens for that user ID and forcing full re-authentication.
- **Anonymous Guest Sessions:** Unauthenticated shoppers receive an HTTP-only cookie tracking an anonymous session ID. Upon login or registration, the guest cart automatically merges with the user's persistent cart.

2.3.6.2 Dynamic Authorization

- **Surface Isolation:** Role-Based Access Control (RBAC) separates administrative (`admin`) and customer (`storefront`) surfaces. Unprivileged accounts attempting to access `/api/*/admin/*` endpoints receive an immediate `403 Forbidden` response prior to command dispatch.
- **Granular Permissions:** Operations enforce fine-grained claims formatted as:

$$\{\text{domain}\} : \{\text{category}\} : \{\text{action}\}$$

- **Runtime Resolution:** A custom `IAuthorizationPolicyProvider` maps claim strings (e.g., `catalog:products:create`) to policies dynamically. Permission assignments can be modified in the database without triggering application redeployments.

2.3.6.3 System Hardening and Defensive Controls

- **Rate Limiting:** Restricts IP traffic to 5 requests/min for authentication, 3 requests/hour for registration, and 30 requests/min for payment processing to mitigate brute-force attacks and abuse.
- **Security Headers:** Middleware injects `Content-Security-Policy`, `Strict-Transport-Security` (HSTS), `X-Frame-Options` (clickjacking protection), and `X-Content-Type-Options`.
- **File Upload Controls:** Visual search uploads enforce a 10 MB limit and inspect magic bytes directly to verify valid JPEG, PNG, or WebP image formats, bypassing spoofed client extensions.
- **Webhook Verification:** Stripe payment webhooks validate the `Stripe-Signature` header against the raw request body using HMAC signature verification before executing state transitions.

2.3.7 Chapter Summary

This chapter detailed the complete architectural framework of ReSys.Shop. The sections below summarize the core design decisions and their underlying technical rationale across all six architectural dimensions:

- **Service-Oriented Architecture:** The platform decouples concerns into three independent layers: a Vue 3 SPA presentation tier, a .NET 10 application engine, and a Python 3.12 FastAPI machine learning sidecar. This separation isolates heavy vector inference workloads from core transactional e-commerce workflows.
- **Domain-Driven Design (DDD):** Business logic is split across eight isolated bounded contexts (`Catalog`, `Ordering`, `Payment`, `Inventory`, `Identity`, `Profile`, `Shipping`, and `Location`). Communication relies on in-process MediatR CQRS dispatch, ensuring zero direct coupling between context databases while maintaining strong aggregate root invariants.
- **C4 Abstraction Modeling:** The system structure is defined across Context, Container, and Component levels, mapping deployable boundaries, asynchronous queue flows, and internal handler pipelines.
- **Database Architecture:** PostgreSQL manages both relational and vector data within per-context schemas. The platform uses `pgvector` for similarity queries, UUID primary keys for distributed generation, soft-deletion interceptors for data auditability, and PostgreSQL `xmin` columns for optimistic concurrency control.
- **RESTful API Contract:** Built on Carter minimal APIs, endpoints follow a uniform `/api/{module}/{surface}/{action}` route pattern. Pre-processor FluentValidation pipelines reject malformed inputs early, returning standard RFC 7807 Problem Details payloads.
- **Security Architecture:** System access relies on short-lived JWTs paired with refresh token rotation and reuse detection. Dynamic policy providers re-

solve string-based permissions at runtime, backed by defensive rate-limiting, magic-byte upload validation, and HMAC webhook verification.

2.4 IMPLEMENTATION

This section describes the concrete realization of the ReSys.Shop system, showing how the architecture and design decisions from Sections 2.2 and 2.3 were translated into working software. The presentation follows the system's actual structure: the technology stack that underpins development, the vertical slice pattern that realization the .NET codebase, the persistence layer that stores relational and vector data in a single database, the machine learning sidecar that constitutes the core research contribution, and the frontend applications that deliver the user-facing experience.

- **Technology Stack.** Framework versions, Aspire orchestration topology, and containerisation strategy.
- **Vertical Slice Core.** Feature co-location, the Carter–MediatR request pipeline, and the functional Result pattern.
- **Data Persistence.** Multi-schema EF Core, pgvector integration with HNSW indexing, and concurrency control.
- **ML Sidecar.** Model management, the embedding generation pipeline, and the end-to-end CBIR search flow.
- **Frontend Applications.** Dual-SPA architecture, visual search interface, and key administration workflows.

2.4.1 Technology Stack

The platform uses a version-pinned technology stack across three runtime ecosystems, each selected for domain alignment. .NET 10 provides strongly typed transactional semantics and enterprise API abstractions. Python 3.12 provides access to the deep learning ecosystem (PyTorch, Hugging Face). Vue 3 delivers component-driven reactive user interfaces.

Centralized package management enforces build reproducibility: `Directory.Packages.props` for NuGet, `uv.lock` for Python, and `pnpm-lock.yaml` for JavaScript.

Table 56 details the core technologies grouped by architectural role.

Table 56: Principal technologies grouped by architectural role with pinned version specifications.

Architectural Role	Technology & Version
Backend Runtime	.NET 10 / ASP.NET Core 10.0.9 (C# 13)

Architectural Role	Technology & Version
API Framework	Carter 10.0.0, MediatR 14.1.0, FluentValidation 12.1.1
Object Mapping	Mapster 10.0.9
ORM & Database	EF Core 10.0.9, Npgsql 10.0.2, pgvector 0.3.2
Caching Layer	HybridCache 10.6.0, StackExchange.Redis 10.0.9
Background Jobs	Hangfire 1.8.23 (Redis-backed)
Observability	OpenTelemetry 1.16.0
ML Runtime	Python 3.12, PyTorch >= 2.0.0, TorchVision >= 0.15.0
ML Framework	FastAPI >= 0.115.0, Uvicorn, Hugging Face Transformers
ONNX Runtime	onnxruntime >= 1.17.0
Storefront UI	Vue 3.5, TypeScript 6.0, Vite 8, PrimeVue 4 (Aura)
Admin UI	Vue 3.5, TypeScript 6.0, Vite 8, PrimeVue 5 (Sakai), Chart.js 4
State & HTTP	Pinia, Axios 1.x, Zod
Data Storage	PostgreSQL 17 (`pgvector/pgvector:pg17-trixie`)
Cache Storage	Redis 7 (Alpine)
Orchestration	.NET Aspire 13.4.6
Test Suites	xUnit v3 3.2.2, pytest >= 8.0, Vitest 4, Playwright

2.4.1.1 Service Orchestration and Containerization

- **Aspire Topology:** The AppHost programmatically defines six system resources with strict startup dependencies: PostgreSQL and Redis initialize first, followed by the Python ML sidecar (validated via its `/health` readiness probe), before the .NET API accepts traffic.

  Connection strings and service URLs are injected dynamically via environment variables. The Vue SPAs run as Vite dev servers with hot module replacement, reverse-proxying `/api/` endpoints to the backend.

- **Multi-Stage Container Builds:** The ML sidecar Dockerfile uses a multi-stage build (`uv sync --frozen`) and a minimal runtime stage executing under a non-root user (`inference`, UID 1001) using `tini` for process signal forwarding.

  The .NET API compiles as a self-contained container via `dotnet publish`. The Vue SPAs build to static assets served by an Nginx reverse proxy.

2.4.2 Vertical Slice Architecture Core

The .NET backend implements Vertical Slice Architecture (VSA) combined with Command Query Responsibility Segregation (CQRS), organizing application code by business capability rather than technical layer. Each feature co-locates its request DTOs, domain execution logic, validation constraints, Carter HTTP endpoints, and response models within a single directory using C# `static partial class` definitions.

2.4.2.1 Feature Co-Location and Execution Mechanics

Every feature is structured as a `static partial class` split across five dedicated files. A representative feature, `CreateProduct` within the Catalog context, illustrates this organizational model:

```
Admin/Products/Create/
├── CreateProduct.cs            # Command and CommandHandler
logic
├── CreateProduct.Request.cs    # Input Request DTO
├── CreateProduct.Response.cs   # Output Response DTO
├── CreateProduct.Endpoint.cs   # Carter Minimal API route
mapping
└── CreateProduct.Validator.cs  # FluentValidation rule
definitions
```

Listing 1: Co-located file layout for a single vertical slice feature.

The feature handler executes through a standardized six-step pipeline using MediatR's `ISender` for cross-module dispatch:

Table 57: Sequential execution flow within the CreateProduct command handler.

Step	Phase	Action Description
Step 1	Validation	Checks product slug uniqueness against PostgreSQL; returns a `DuplicateSlug` domain error on collision.
Step 2	Instantiation	Constructs the `Product` entity via a domain factory method returning `Result<Product>`.
Step 3	Persistence	Appends the entity to `IApplicationDbContext` and saves changes to commit the transaction.
Step 4	Dispatch	Issues a cross-module `AddVariant` command via `ISender` to initialize the master SKU.
Step 5	Association	Assigns the generated master variant ID back to the `Product` aggregate and updates state.
Step 6	Completion	Returns `Result<Response>.Created(...)` containing the mapped response payload.

Compile-Time Module Isolation: Inter-module communication relies exclusively on `ISender.Send()` messages. Bounded contexts never import foreign context namespaces, establishing strict boundaries within a single assembly enforced via static analysis and build policies.

```
public sealed record Command(Request Request) :
ICommand<Response>;

public sealed class CommandHandler(
    IApplicationDbContext dbContext,
    ISender sender)
    : ICommandHandler<Command, Response>
{
    public async Task<Result<Response>> Handle(
        Command command, CancellationToken ct)
    {
        // Step 1: Validate slug uniqueness
        if (await dbContext.Set<Product>()
            .AnyAsync(x => x.Slug == command.Request.Slug, ct))
            return ProductResult.Errors.DuplicateSlug;

        // Step 2-3: Instantiate domain entity and persist
        var product = command.Request.MapToDomain().Value;
        dbContext.Set<Product>().Add(product);
        await dbContext.SaveChangesAsync(ct);

        // Step 4-5: Dispatch cross-module command for master
variant
        var variantResult = await sender.Send(
            new AddVariant.Command(product.Id, variantRequest),
ct);
        product.MasterVariantId = variantResult.Value.Id;
        await dbContext.SaveChangesAsync(ct);

        // Step 6: Return success response wrapper
        return Result<Response>.Created(
            product.MapToDetail<Response>(),
            ProductResult.Success.Created(product.Id));
    }
}
```

Each feature's endpoint participates in a shared request pipeline that applies cross-cutting concerns (validation, logging, and exception handling) uniformly across all features.

2.4.2.2 Request Pipeline Architecture

2.4.2.2.1 Carter Endpoint Registration

Each vertical slice implements `ICarterModule` to define its HTTP interface. Endpoints remain intentionally thin: parsing incoming requests, constructing MediatR command objects, dispatching via `ISender`, and converting functional `Result<T>` outputs into HTTP responses. All 257 API endpoints across the platform follow this pattern:

```
public class Endpoint : ICarterModule
{
    public void AddRoutes(IEndpointRouteBuilder app)
    {
        app.MapPost(CatalogFeature.Admin.Products.Create.Route,
            async ([FromBody] Request request,
                   ISender sender, CancellationToken ct) =>
        {
            var result = await sender.Send(new Command(request),
ct);
            return result.ToResult();
        })
      .HasPermission(CatalogFeature.Admin.Products.Create.Permission)
        .Produces<Result<Response>>()
        .Produces<Result>(StatusCodes.Status400BadRequest);
    }
}
```

Routes follow a structural hierarchy: `/api/{module}/{surface}/{resource}` (where `surface` is designated as `admin` or `storefront`). Carter modules are auto-discovered at runtime via assembly scanning during application startup.

2.4.2.2.2 MediatR Middleware Behaviors

Three cross-cutting behaviors wrap every command and query in a nested execution model. They are registered from outermost to innermost, forming a layered pipeline where each behavior delegates to the next:

```
cfg.AddBehavior(typeof(IPipelineBehavior<,>),
typeof(LoggingBehavior<,>));
cfg.AddBehavior(typeof(IPipelineBehavior<,>),
typeof(ValidationBehavior<,>));
cfg.AddBehavior(typeof(IPipelineBehavior<,>),
typeof(ExceptionMappingBehavior<,>));
```

When a request is dispatched via `ISender`, it passes through each behavior in order:

- **LoggingBehavior** (outermost) captures the request type, then delegates inward.
- **ValidationBehavior** runs co-located FluentValidation rules. On failure, it returns `List<Error>` immediately, short-circuiting both the inner behavior and the handler.

- **ExceptionMappingBehavior** (innermost) delegates to the handler. It catches any unhandled exception, wraps it in `Error.FromException()`, and returns a structured failure instead of letting the exception propagate.

On the return path, `LoggingBehavior` inspects the response and records success or failure. The final `Result<T>` propagates back through Carter to the HTTP client. The handler never references logging, validation, or exception handling; these concerns are applied transparently by the pipeline.

Table 58: MediatR pipeline behavior execution sequence and responsibilities.

Pipeline Behavior	Execution Order	Operational Responsibility
`LoggingBehavior`	Outermost	Logs request type metadata, execution duration, and completion status upon pipeline exit.
`ValidationBehavior`	Middle	Executes co-located FluentValidation rules. Rejects invalid requests early by returning a `List<Error>` payload without invoking downstream handlers.
`ExceptionMappingBehavior`	Innermost	Catches unhandled execution exceptions, wraps them in standard `Error.FromException()` models, and prevents internal stack trace leakage.

2.4.2.3 Functional Result Pattern and Error Handling

The application avoids exception-driven control flow in favor of a functional `Result<T>` pattern. Domain methods return explicitly typed `Result<T>` (for operations returning values) or `Result` (for void operations). Both structures are implemented as memory-efficient `readonly record struct` types:

```
public readonly partial record struct Result<T> : IResultRecord
{
    [AllowNull] public T Value { get => field!; init; }
    public bool IsSuccess { get; init; }
    public int StatusCode { get; init; }
    public List<Error> Errors { get; init; }
    public bool IsFailure => !IsSuccess;
}
```

- **Domain Error Abstraction:** The `Error` struct encapsulates a machine-readable `Code`, human-readable `Message`, integer type discriminator, and an optional metadata dictionary. Specialized factory helpers generate standard error categories (`Error.BadRequest`, `Error.NotFound`, `Error.Conflict`, `Error.Validation`).

- **Implicit Conversions:** Returning a domain object implicitly wraps it in a successful `Result<T>` instance. Returning an `Error` or `List<Error>` automatically casts into a failed `Result<T>`.
- **HTTP Mapping:** The `ToResult()` extension translates domain results directly to standard HTTP status codes: **200 OK** or **201 Created** for success paths, **400 Bad Request** for validation failures, **404 Not Found** for missing entities, and **409 Conflict** for state violations.

2.4.3 Data Persistence Architecture

The persistence layer maps the eight bounded contexts to dedicated PostgreSQL schemas, co-locating high-dimensional vector embeddings with relational product data in a single PostgreSQL 17 database instance.

2.4.3.1 Schema Organisation and Vector Storage

Each bounded context owns an isolated database schema containing its aggregate roots and child entities. Table 59 outlines the schema-per-context distribution:

Table 59: Schema-per-context mapping. Cross-schema references use UUID attributes without database-level foreign key constraints.

Schema	Principal Tables & Aggregate Entities
`catalog`	`products`, `variants`, `variant_images`, `product_image_embeddings`, `taxonomies`, `taxons`
`ordering`	`orders`, `line_items`, `order_adjustments`, `shipments`
`payment`	`payment_intents`, `payment_captures`, `payment_methods`
`inventory`	`stock_locations`, `stock_items`, `stock_movements`, `stock_reservations`
`identity`	`users`, `roles`, `user_roles`, `refresh_tokens`
`profile`	`user_profiles`, `addresses`, `wishlists`
`shipping`	`shipping_methods`, `shipping_rates`, `shipping_zones`
`location`	`countries`, `states`

Schema boundaries mirror the C# compile-time isolation rules. An order entity references a `variant_id` as a unconstrained UUID rather than a database foreign key constraint, enforcing logical module independence without distributed transaction overhead.

Co-located vector storage. Product embeddings reside within `catalog.product_image_embeddings` alongside product metadata. Co-locating relational data and vectors inside PostgreSQL eliminates dual-database synchro-

nization issues: vector updates participate in the primary database transaction, guaranteeing ACID consistency without external sync workers.

2.4.3.1.1 pgvector Indexing Strategy

Image embedding vectors are stored in a `vector(512)` column representing 512-dimensional latent spaces from vision-language models. The persistence layer supports two Approximate Nearest Neighbor (ANN) index types configured for different operational environments:

Table 60: Comparison of pgvector ANN index algorithms using the cosine distance operator (<=>).

Attribute	HNSW (Hierarchical Navigable Small World)	IVFFlat (Inverted File Flat)
Index Structure	Multi-layer navigable graph	K-means cluster partitioning
Build Overhead	Memory-intensive (several minutes)	Low memory footprint (< 1 s)
Query Latency	Sub-millisecond (< 5 ms, logarithmic)	Fast (< 10 ms, cluster-dependent)
Deployment	Production environment	Benchmark iteration & constrained testing
Parameters	$m = 16, \text{ef}_{\text{construction}} = 64$	lists $= 100$

The production HNSW index DDL is executed during migration initialization:

```
CREATE INDEX ix_product_image_embeddings_vector_hnsw
ON catalog.product_image_embeddings
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);
```

2.4.3.1.2 Model-Aware Vector Search Pipeline

Vector embeddings from different neural architectures inhabit incompatible vector spaces. To prevent cross-model distance pollution, every record persists a `model_name` discriminator (e.g., `"Fashion-CLIP"`). Cosine similarity search follows a four-stage query execution process:

1. **Candidate Retrieval:** Executes a cosine distance query using the <=> operator to fetch the top $3 \times K$ nearest neighbors filtered by `model_name`.
2. **Similarity Transformation:** Converts distance to similarity metrics via similarity $= 1 -$ cosine_distance.
3. **Threshold Filtering:** Discards candidates failing the minimum confidence threshold (similarity ≥ 0.70).

4. **Product Deduplication:** Group-by filtering reduces multiple matching variant images down to the highest-scoring master product entity.

2.4.3.2 Concurrency and Data Integrity

Data integrity under high concurrent throughput is maintained using three complementary concurrency strategies:

Table 61: Concurrency control mechanisms across system domains.

Pattern	Technical Implementation	Target Operational Domain
Optimistic Locking	PostgreSQL `xmin` system column mapped via EF Core `IsRowVersion()`	General entity updates. Stale writes trigger `DbUpdateConcurrencyException`, returning HTTP 409 Conflict.
Pessimistic Locking	`SELECT FOR UPDATE` row locks acquired explicitly	Stock allocation during checkout. Serializes concurrent purchases of low-quantity SKUs.
Repeatable Read	`IsolationLevel.RepeatableRead` database transaction scope	Sequential order code generation. Prevents phantom reads during atomic sequence assignment.

2.4.3.2.1 Audit Ledger and EF Core Interceptors

Entity lifecycle tracking and stock auditing are handled automatically via EF Core `DbContext` interceptors, keeping timestamp boilerplate out of business handlers. Every stock modification writes an append-only entry to `inventory.stock_movements`.

An `AuditInterceptor` registered in the `SaveChanges` pipeline auto-populates audit fields without explicit handler code:

1. On `EntityState.Added`, sets `CreatedAt` to the current UTC timestamp and `CreatedBy` to the authenticated user ID.
2. On `EntityState.Added` or `EntityState.Modified`, sets `UpdatedAt` and `UpdatedBy` identically.

This interceptor pattern eliminates the class of bugs where timestamps are inconsistently applied across different features. Handlers never set audit fields manually; the persistence infrastructure applies them uniformly during `SaveChangesAsync`.

2.4.4 ML Sidecar and CBIR Search

The Python machine learning sidecar is the core research contribution of this thesis. It generates high-dimensional vector embeddings from product images

and serves them to the .NET backend via HTTP, managing multiple pre-trained deep learning models through a strategy pattern that enables runtime model switching.

2.4.4.1 Model Management and Strategy Pattern

2.4.4.1.1 Environment-Driven Selection

The active embedding model is selected through a single `EMBEDDING_MODEL` environment variable. The Model Manager reads this identifier on first inference, instantiates the target class, loads weight snapshots from disk, and caches the instance in memory.

Pluggable model architecture: swapping the active inference model requires updating one environment variable and restarting the container; a pure configuration change with no code modification. This mechanism enables the automated benchmarking and production A/B testing workflows described in Section 3.2.

2.4.4.1.2 Model Registry

Six concrete models spanning four architectural families are registered through a decorator-based registry:

```
class ModelRegistry:
    _registry: dict[str, type[BaseEmbedder]] = {}

    @classmethod
    def register(cls, name: str, metadata: dict | None = None):
        def decorator(model_cls: type[BaseEmbedder]):
            cls._registry[name] = model_cls
            return model_cls
        return decorator

@ModelRegistry.register("fashion_clip", {
    "description": "CLIP fine-tuned on 700K+ fashion images",
    "dimensions": 512
})
class FashionCLIPEmbedder(CLIPEmbedder):
    ...
```

Table 62: Registered embedding models with output dimensionality and architectural family.

Model ID	Dim	Architecture	Source
fashion_clip	512	ViT-B/32 + CLIP	patrickjohncyh/fashion-clip (HuggingFace)
clip_vit_b16	512	ViT-B/16 + CLIP	OpenAI CLIP (torchvision)
openclip-vit-b-32	512	ViT-B/32 + CLIP	OpenCLIP (HuggingFace)
efficientnet_b0	1280	EfficientNet-B0	torchvision ImageNet1K_V1

Model ID	Dim	Architecture	Source
resnet50	2048	ResNet-50	torchvision ResNet50_Weights.DEFAULT
dinov2_vits14	384	ViT-S/14	facebookresearch/dinov2 (torch.hub)

2.4.4.1.3 Strategy Pattern and Template Method

All models conform to `BaseEmbedder`, an abstract class employing the Template Method pattern. The base class orchestrates the embedding pipeline; subclasses provide only the model-specific forward pass:

```
class BaseEmbedder(ABC):
    @abstractmethod
    def _forward(self, image: torch.Tensor) -> torch.Tensor:
        """Model-specific forward pass."""

    async def extract(self, image_input) -> list[float]:
        image = self._load_image(image_input)     # 1. Load
        features = self._forward(image)            # 2. Forward
        return self._normalize(features)           # 3. Normalize
```

The `_load_image` method handles URLs, file paths, raw bytes, and PIL Images uniformly, converting all inputs to RGB tensors. The `_normalize` method L2-normalises the raw features to unit vectors and handles multiple output shapes: `image_embeds` for CLIP models, `pooler_output` for ViT models, and feature maps for CNN classifiers (via global average pooling).

Models load lazily on first request rather than at startup, preventing unnecessary GPU memory consumption. The first request after a cold start incurs the model load time (approx 125 ms for EfficientNet-B0, 5-6 seconds for CLIP-based models); subsequent requests serve from the cache at full inference speed.

2.4.4.1.4 Hardware Acceleration

The `device` property resolves the execution target at runtime with a priority chain: CUDA GPU, Apple MPS, CPU. On the benchmark workstation, only CPU was available; all reported inference times reflect CPU-only execution. The forward pass executes within `torch.no_grad()` to disable gradient computation, reducing memory usage by approximately 50 percent compared to training mode.

2.4.4.2 Embedding Generation Pipeline

Table 63: Embedding generation pipeline stages.

Stage	Component	Description
1	Authentication	Validates the `X-API-Key` header; rejects unauthorised calls with 401.
2	Model Selection	Retrieves the active model from the cached registry; initialises from disk if unallocated.
3	Preprocessing	Resizes to 224×224 pixels, converts to tensor, applies ImageNet normalisation ($\text{mean} = [0.485, 0.456, 0.406]$, $\text{std} = [0.229, 0.224, 0.225]$).
4	Forward Pass	Propagates the tensor through convolution or self-attention layers within `torch.no_grad()`.
5	Pooling	Global average pooling collapses spatial dimensions to a fixed-length vector (512-dim for CLIP variants, 1280 for EfficientNet-B0, 2048 for ResNet-50).
6	Serialization	L2-normalises the vector; packs it with model metadata and inference latency into a JSON response.

2.4.4.2.1 API Endpoints

Table 64: ML sidecar API endpoints. All inference endpoints require `X-API-Key` header authentication.

Method	Path	Purpose
POST	`/embeddings/bytes`	Accepts multipart image upload with optional `model` query parameter. Validates MIME type and file size; returns embedding vector with metadata.
POST	`/embeddings`	Accepts an image URL in the request body for batch embedding during catalogue indexing.
GET	`/health`	Readiness probe for the Aspire orchestrator. Validates CUDA/MPS availability, active model status, and synthetic forward pass.

The response shape mirrors the .NET Result pattern:

```
{
  "value": {
    "vector": [0.023, -0.154, ..., 0.042],
    "model": "fashion_clip",
    "dimensions": 512,
    "inference_ms": 92.0
  },
  "isSuccess": true,
  "statusCode": 200
}
```

2.4.4.3 CBIR Search Flow

The end-to-end visual search pipeline spans four architectural layers: the Vue 3 storefront, .NET API backend, Python ML sidecar, and PostgreSQL with pgvector. Figure 41 illustrates the cross-service sequence, engineered to complete within a 1-second total latency budget.

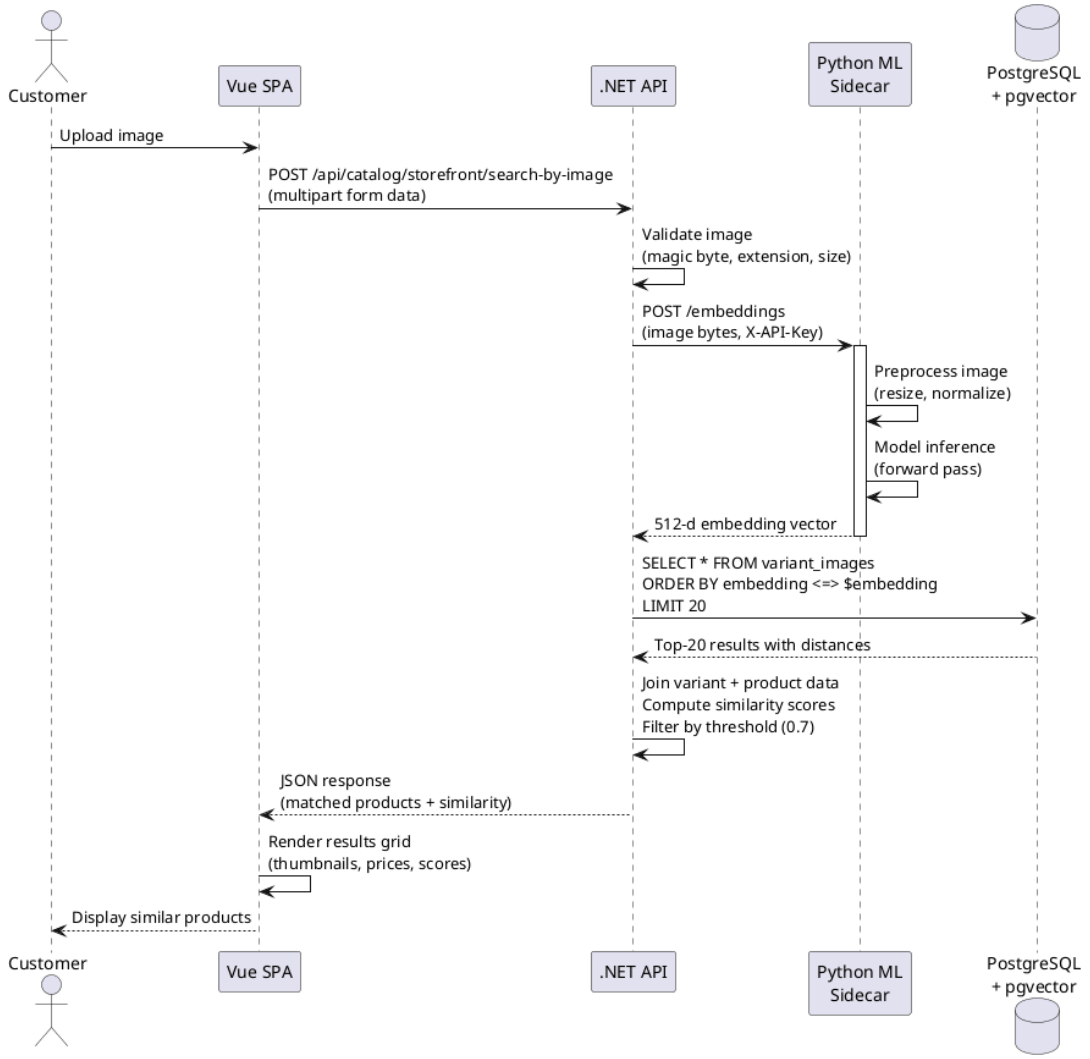

Figure 41: CBIR search sequence: end-to-end flow spanning customer upload, embedding extraction, pgvector search, and ranked results rendering.

The sequence crosses four layers: the Vue 3 storefront (client validation and result rendering), the .NET API (server validation and orchestration), the Python ML sidecar (embedding generation), and PostgreSQL with pgvector (vector similarity search). Each layer's role is detailed in the execution pipeline below.

2.4.4.3.1 Execution Pipeline

1. **Client Validation (Vue 3).** Validates file format (JPEG, PNG, WebP) and size ($\leq$ 10 MB). Dispatches a multipart form request to `POST /api/catalog/storefront/search-by-image`.
2. **Server Validation (.NET API).** Verifies binary header magic bytes to prevent file extension spoofing, validates MIME type, and reapplies the payload ceiling.
3. **Vector Extraction (ML Sidecar).** Forwards image bytes to `/embeddings`. The sidecar executes preprocessing and model inference (50-100 ms for CLIP-based models, 24 ms for EfficientNet-B0), returning a 512-dimensional float vector.
4. **Vector Index Search (pgvector).** Queries `product_image_embeddings` using the `<=>` cosine distance operator, filtered by `model_name`. HNSW indexing enables sub-10-millisecond logarithmic lookup.
5. **Post-Processing.** Converts cosine distances to similarity scores ($\text{similarity} = 1 - \text{distance}$), discards results below the configurable threshold (default 0.7), and deduplicates by parent product.
6. **UI Rendering (Vue 3).** Receives a JSON payload with titles, prices, thumbnails, and similarity percentages; renders a product grid to complete the sub-second search.

2.4.4.3.2 Model Configuration and A/B Testing

The pluggable architecture supports two operational workflows:

- **Automated benchmarking.** Test scripts update the `EMBEDDING_MODEL` variable and restart the container, enabling sequential evaluation of all 11 candidate models without code modifications.
- **Production A/B testing.** Dual ML sidecar instances run in parallel with distinct model configurations. A traffic router directs user cohorts to each instance, enabling direct comparison of downstream business metrics (click-through rates, conversion rates, session duration).

2.4.5 Frontend Applications

The user-facing components of ReSys.Shop consist of two Vue 3 single-page applications: a customer storefront built with PrimeVue's Aura theme and an administration dashboard using the Sakai theme. This section presents the frontend architecture, the shared API client pattern, and the implemented user interfaces organised by the 26 use cases defined in Section 2.2.2.

2.4.5.1 Frontend Architecture

2.4.5.1.1 Dual-SPA Structure

Both applications share a common technology foundation: Vue 3.5 with the Composition API, TypeScript 6.0, Vite 8, Pinia for state management, and Axios for HTTP communication: but differ in their UI component presets. The storefront uses PrimeVue 4 (Aura theme) optimised for consumer browsing, image upload, and product discovery. The administration dashboard uses PrimeVue 5 (Sakai theme) with Chart.js 4 for data-dense views, inline editing, and aggregate dashboards.

Each application is organised by feature module, grouping views, types, services, and API repositories together. Vite provides hot module replacement with sub-second feedback during development; the dev server proxies `/api/` requests to the .NET backend running on a separate Aspire-managed port.

2.4.5.1.2 API Client Pattern

All communication with the backend follows a typed repository pattern that mirrors the C# `Result\<T\>` convention. The TypeScript `Result\<T\>` interface carries `isSuccess`, `statusCode`, `data`, and `errors[]` fields, enabling type-safe error handling in the UI without try-catch blocks:

```
export interface Result<T> {
  isSuccess: boolean
  isFailure: boolean
  statusCode: number
  message?: string
  data?: T
  errors?: Array<{
    code: string
    description: string
    field?: string
  }>
}
```

The `BaseRepository` class wraps an Axios instance with typed CRUD methods (`get\<T\>()`, `post\<T\>()`, `uploadFile\<T\>()`) and unified error handling. Axios error responses are unwrapped to preserve the backend's structured error codes and field-level metadata. Each feature module defines its own repository extending this base class.

For the visual search feature, `RecommendationsApiRepository` extends the base repository with a multipart upload method:

```
async searchByImage(file: File): Promise<Result<Product[]>> {
  const formData = new FormData()
  formData.append('image', file)
  const response = await this.client.post<Result<Product[]>>(
    '/api/storefront/search-by-image',
    formData,
```

```
    { headers: { 'Content-Type': 'multipart/form-data' } }
  )
  return response.data
}
```

2.4.5.2 Storefront Interfaces

The customer storefront implements eight use cases covering product discovery, purchasing, and account management. Each subsection below presents the implemented interface and the screenshot evidence for the corresponding use case.

2.4.5.2.1 Visual Search: UC-STR-SRC

The visual search interface is the primary research-facing feature. It implements a four-state UI model managed through Vue reactive refs and computed properties:

Table 65: Visual search UI state model.

State	UI Displayed	Transition Trigger
Empty	Upload prompt with dashed-border drop zone, cloud-upload icon, format description (JPEG, PNG, WebP up to 10 MB)	Initial page load | after clearing a previous search
Upload	Selected image preview displayed at full width with a "Search Similar Products" action button below	User drops a file into the zone or selects one via file browser
Loading	Skeleton grid of animated placeholder cards (4 columns desktop, 2 mobile) with pulsing backgrounds	Search API request in flight (`POST /api/storefront/search-by-image`)
Results	Responsive product card grid (4/2 columns) with thumbnail, name, price, and a coloured similarity badge (e.g., "93% match")	API returns a non-empty `results` array

Two input methods are supported: drag-and-drop onto the highlight-responsive upload zone and file browser selection via a styled button labelled "Choose an image". Client-side validation rejects non-image MIME types and files exceeding 10 MB before any network request, providing immediate inline feedback. The `DragEvent` handlers toggle an `isDragging` ref that applies a visual highlight CSS class to the drop zone.

After upload, the component constructs a `FormData` payload with the `image` field containing the selected `File` object. The backend orchestrates embedding extraction and pgvector similarity search as described in Section 2.4.3. The response payload carries product metadata with similarity scores computed server-side:

```
{
  "results": [{
    "productId": "a1b2c3d4-...",
    "title": "Floral Summer Midi Dress",
    "price": 750000, "currency": "VND",
    "thumbnailUrl": "/images/products/a1b2c3d4_thumb.webp",
    "similarityScore": 0.9328,
    "categoryPath": "Clothing > Dresses > Midi Dresses"
  }],
  "searchDurationMs": 287,
  "model": "Fashion-CLIP"
}
```

Each product card renders a thumbnail image, product title, formatted price, and a colour-coded similarity confidence badge (green for >= 90%, amber for >= 80%, grey below 80%). The query image is displayed in a persistent sidebar panel for reference while scrolling through results. Clicking a product card navigates to the full product detail page.

2.4.5.2.2 Catalogue Browsing and Product Detail: UC-STR-BRW

The catalogue browser presents products in a paginated, faceted grid with left-sidebar category tree navigation. The category tree is rendered from the hierarchical taxonomy data returned by `GET /api/storefront/taxonomies/tree`, with expandable/collapsible taxon nodes.

Selecting a category filters the product grid.

The product listing grid shows thumbnails, product names, prices (formatted with the master variant's current price), and a hover overlay revealing a quick-add-to-cart button. Pagination controls at the bottom of the grid navigate between result pages. A keyword search bar at the top of the page sends full-text queries to `GET /api/storefront/products?search=...`.

The product detail page displays the complete product information. A gallery of variant images provides thumbnail navigation. The selected variant's size and colour pickers show real-time stock availability indicators ("In Stock" / "Only 2 left" / "Out of Stock").

The current price is displayed alongside any promotional discount shown as a strikethrough original price. A quantity selector and "Add to Cart" button complete the purchase actions.

Expandable sections provide product description, material composition, care instructions, and size guide. A "Similar Products" section at the bottom displays visually similar items retrieved from `GET /api/storefront/products/{id}/similar`.

2.4.5.2.3 Shopping Cart: UC-STR-CRT

The cart page lists line items in a scrollable list, each showing the product thumbnail, title, selected size and colour, unit price, a quantity control with increment/decrement buttons and a current-quantity display (minimum 1, maximum available stock), a per-line subtotal, and a remove button (trash icon).

A sticky summary panel on the right displays the item count, subtotal, and a "Proceed to Checkout" button. An empty-cart state is shown when no items are present, with a "Continue Shopping" link to the catalogue.

Cart data is persisted via the backend API (`GET/POST/PUT/DELETE /api/storefront/cart`) and synchronised across browser tabs through Pinia reactive state. Guest users' carts are identified by a signed session cookie; upon authentication, the guest cart is automatically merged into the customer's existing cart via the `/api/storefront/cart/associate` endpoint.

2.4.5.2.4 Checkout: UC-STR-CHK

Checkout progresses through the five-state pipeline (Address, Delivery, Payment, Confirm, Complete) defined in the order state machine (Section 2.2.3). Each step is rendered as a single-page form with a visual progress indicator at the top showing all five stages, with the current stage highlighted and completed stages marked with a checkmark.

- **Address step.** Displays the customer's saved addresses from the profile with radio-button selection and an inline "Add new address" form. The selected address is validated server-side before proceeding.
- **Delivery step.** Lists available shipping methods with carrier name, estimated delivery time, and calculated rate. Rates are computed by the backend based on address zone, cart weight, and order value.
- **Payment step.** Presents available payment methods with provider icons. The selected method is confirmed via `POST /api/storefront/payment/create-intent`.
- **Confirm step.** Displays a read-only summary of the order: line items, shipping address, delivery method, payment method, and totals (item subtotal, shipping cost, grand total). A "Place Order" button finalises the checkout.
- **Complete step.** Shows an order confirmation with the generated order number, a summary, and a "Continue Shopping" link.

Each transition is validated server-side; attempts to skip steps or submit stale data are rejected.

2.4.5.2.5 Order History: UC-STR-OHI

The order history page lists all past orders in a vertically stacked card layout, with each card showing the order number, date, current status (with colour-coded badge), item count, and total amount. Expanding a card reveals line items with thumbnails, quantities, and prices, plus shipping and payment status detail.

Active orders show a "Cancel Order" button when the order is in a cancellable state. A "View Details" link navigates to the full order detail page with a timeline of checkout state transitions.

2.4.5.2.6 Authentication: UC-STR-AUT, UC-STR-SES

Authentication supports three pathways: email/password login, Google OAuth 2.0, and guest sessions.

- **Registration.** A form with email, password (with strength indicator), confirm password, and full name fields. On success, the user is redirected to the storefront with an authenticated session.
- **Login.** A form with email and password fields, a "Remember me" checkbox, and "Forgot password?" link. A "Sign in with Google" button triggers the OAuth flow.
- **Password reset.** A two-step flow: email entry form, then a new-password form accessed via a single-use token link sent to the registered email.
- **Session management.** A session page lists active sessions with device, IP address, and last-activity timestamp. A "Logout All Devices" button terminates all sessions.

Guest sessions use signed cookies to identify anonymous users for cart persistence. Upon authentication, the guest cart is merged into the customer's account without data loss.

2.4.5.2.7 Payment Processing: UC-STR-PAY

During checkout, the payment step presents available methods retrieved from `GET /api/storefront/payment/methods`. Selecting a method and confirming triggers `POST /api/storefront/payment/create-intent` with the order amount and currency. For Stripe payments, the Stripe Elements embedded UI collects card details within an iframe without exposing sensitive data to the storefront's JavaScript context. The payment confirmation result updates the order's payment sub-state and advances to the Confirm step.

2.4.5.2.8 Profile Management: UC-STR-PRF

The profile section provides three management areas:

- **Address book.** A list of saved addresses with type labels (Home, Work), a default-address toggle, inline edit capability, and an "Add New Address" form. Addresses include recipient name, phone, street, ward, district, city, and country fields with cascading location selectors.
- **Wishlists.** Named collections of saved products. Each wishlist card shows the name, product count, and privacy setting. Expanding a wishlist shows product thumbnails with remove and add-to-cart actions.
- **Notification preferences.** Per-channel toggles (email, SMS) for order updates, promotions, and stock alerts.

2.4.5.3 Administration Interfaces

The administration dashboard implements the fifteen administrative use cases from the summary matrix. The interface uses PrimeVue's data-table components with server-side pagination, sorting, and filtering for all list views, and form dialogs with inline validation for create and edit operations.

2.4.5.3.1 Product Management: UC-ADM-PROD, UC-ADM-VAR, UC-ADM-IMG, UC-ADM-TAX, UC-ADM-OPT

The product management module is the most data-intensive admin area. It is organised into interlinked management surfaces:

Product list (UC-ADM-PROD). A paginated data table with columns for thumbnail, product name, SKU, category path (breadcrumb-style), status badge (Draft in grey, Active in green, Archived in red), base price (from master variant), and last modified date. The toolbar provides a keyword search input, a category filter dropdown, a status filter multi-select, and a "New Product" button.

Row actions include Edit, Activate/Discontinue, and Delete. Bulk actions support status changes and category assignment across selected rows.

Product create/edit form (UC-ADM-PROD). A multi-section form opened as a full-page view. The Basic Info section contains name, URL slug (auto-generated from name with manual override), description (rich text editor), and status dropdown.

The Fashion Metadata section contains style code, season (dropdown: Spring/Summer, Fall/Winter, All-Season), material composition (free text), department (dropdown: Women, Men, Kids, Unisex), and gender target (dropdown: Female, Male, Unisex).

The SEO section contains meta title and meta description fields. Form validation provides inline error messages below each field.

Variant management (UC-ADM-VAR). Embedded within the product detail page, a table lists all variants with columns for SKU, barcode, master-variant radio selector, option combination (e.g., "M / Navy"), track-inventory toggle, price (current), and dimensions (weight, length, width, height).

Inline actions: Edit, Delete. An "Add Variant" button opens a modal dialog with option-value selectors, SKU auto-generation, pricing fields (current price, optional sale price with date range), and dimension fields.

Pricing management (UC-ADM-VAR). Each variant supports time-bound pricing. A pricing tab within the variant edit dialog shows a table of price records: amount, currency, effective-from date, effective-to date (nullable for indefinite), and status (Active, Scheduled, Expired). "Add Price" opens inline fields for amount, currency, and date range.

Image and embedding management (UC-ADM-IMG). An image upload panel within the product detail page supports drag-and-drop and multi-file selection. Uploaded images appear in a sortable grid with drag handles for reordering, thumbnail previews, alt-text input, and a delete action.

Each image row shows an embedding status indicator: a green checkmark with "Embedded (Fashion-CLIP)" label for processed images, a yellow spinner for pending generation, and a red exclamation with "Regenerate" button for failed or missing embeddings.

A "Regenerate All Embeddings" button triggers re-embedding with the currently active model.

Taxonomy management (UC-ADM-TAX, UC-ADM-OPT). A tree editor for managing hierarchical category structures. The left panel shows the taxonomy tree with expand/collapse, drag-and-drop reordering, and context-menu actions (Add child, Edit, Delete). The right panel shows the selected taxon's details: name, slug, description, parent taxon, and associated business rules.

Taxons are assigned to products through a multi-select tree picker in the product edit form.

2.4.5.3.2 Order Management: UC-ADM-ORD, UC-ADM-ORD-ITEMS

The order management grid provides a filterable, paginated view of all orders. Columns display order number, customer name (linked to user detail), checkout state (colour-coded badge: Pending in blue, Confirmed in purple, Completed in green, Cancelled in red), payment status, shipment status, total amount, and creation date. Filters include status multi-select, date range picker, payment method, and keyword search across order numbers and customer names.

Clicking an order opens the order detail page. The order header displays the order number, customer info, and current state, followed by a timeline of state transitions with timestamps.

A line items table shows product thumbnails, variant details (SKU, size, colour), quantity, unit price, and line total. Shipping and billing address blocks sit alongside a payment detail section with gateway transaction ID, payment state, and action buttons. Shipment tracking shows the carrier and tracking number.

State transition buttons (Approve, Complete, Cancel, Resume) appear conditionally based on the current checkout state per the order state machine.

2.4.5.3.3 Payment Management: UC-ADM-PAY, UC-ADM-PAY-METHOD

The payment detail panel provides full visibility into payment intent lifecycles. It displays the payment intent ID, gateway provider (Stripe or Bogus), gateway transaction ID, amount, and currency.

The current state appears with a colour-coded badge and state transition timeline. A capture/refund/void action bar has buttons conditionally enabled based on state.

A payment log shows each gateway interaction with timestamp, and a webhook event log records Stripe-triggered state changes.

Payment method management (UC-ADM-PAY-METHOD) presents a table of configured gateways with provider name, display name, active toggle, and supported currencies. An "Add Method" dialog configures a new gateway with provider selection, display name, and supported-currency multi-select.

2.4.5.3.4 Inventory Management: UC-ADM-STK, UC-ADM-LOC

The inventory panel provides real-time stock visibility and management across warehouse locations.

Stock list. A table grouped by product shows each variant with its SKU, size/colour, on-hand quantity, reserved quantity, available quantity (on-hand minus reserved, computed client-side), and a low-stock indicator (red badge when available drops below the configured threshold). Filtering by location, category, and low-stock-only is supported.

Stock movements (UC-ADM-STK). An append-only audit log table showing every stock change with columns for date/time (UTC), product/variant, movement type (Receiving, Selling, Returning, Stocktaking, Transferring), quantity delta (+/-), quantity before, quantity after, reason code, and operating

user. The table is paginated and filterable by type, product, date range, and location.

Stock locations (UC-ADM-LOC). A management table listing warehouse locations with name, address, active toggle, and stock item count. "Add Location" and edit dialogs capture location name, address fields, and active status.

2.4.5.3.4.1 Restock and Transfer

- **Restock.** A form accepting product/variant selection, quantity, unit cost, and reason. Submitting creates a Receiving stock movement and increments the on-hand quantity.
- **Transfer.** A form with source location, destination location, product/variant, and quantity. Submitting creates a Transferring movement, decrements the source, and increments the destination. The transfer lifecycle progresses through Created, In-Transit, and Received states.

2.4.5.3.5 User and Role Administration: UC-ADM-USR, UC-ADM-ROL

User management (UC-ADM-USR). A paginated table listing registered users with columns for avatar, full name, email, registration date, enabled/disabled status toggle, assigned roles (displayed as compact badges), and last login date. Filters include status (All, Enabled, Disabled), role, and keyword search.

The user create/edit form includes full name, email, password (create only), enabled toggle, and a role assignment multi-select.

Role management (UC-ADM-ROL). A table listing roles with role name, description, user count, and creation date. Expanding a role shows its permission assignments in an expandable tree grouped by domain (Catalog, Ordering, Payment, Inventory, Identity, Profile, Shipping, Location, Dashboard). Each permission is a `domain:category:action` claim with a checkbox toggle. The permissions catalogue is read from `GET /api/admin/identity/permissions`.

2.4.5.3.6 Shipping Configuration: UC-ADM-SHP

The shipping management interface configures delivery methods and their associated rates and geographic zones.

Shipping methods. A table listing configured shipping methods (Standard Delivery, Express Delivery, Next-Day Delivery, International) with carrier name, estimated delivery time range, active toggle, and associated rates count. The create/edit dialog captures method name, carrier, description, delivery time estimate, and active status.

Shipping rates. Each method has a table of rates configured per geographic zone and weight/value tier. Rate rows show zone name, minimum/

maximum weight, minimum/maximum order value, flat rate amount, and active status. The add-rate dialog provides zone dropdown, weight range, value range, and rate amount fields.

2.4.5.3.7 Reference Data: UC-ADM-REF

Country and state reference data management provides lookup tables used by addresses, shipping zones, and tax calculations. Countries are listed with ISO 3166-1 alpha-2 code, name, and active toggle. Selecting a country displays its associated states with ISO 3166-2 codes.

2.4.5.3.8 System Processes: UC-SYS-EMB, UC-SYS-MNT

Background automation is monitored through the Hangfire dashboard, accessible at the `/hangfire` admin route.

The dashboard overview displays key metrics: total succeeded jobs, failed jobs (with red badge if non-zero), recurring job schedules, and current queue depths. The recurring jobs section lists each scheduled job with its cron expression or interval, last execution time and duration, next scheduled execution, and success/failure counts.

- **Cart expiry.** Runs every 20 minutes. Queries carts with no activity for 7 days, releases reserved inventory back to available stock, and deletes the cart record. The job log shows the count of expired carts per execution.
- **Embedding retries.** Processes failed embedding generation requests with exponential back-off (1 min, 2 min, 4 min, 8 min, max 3 retries). Permanently failed embeddings are flagged for manual review.
- **Index maintenance.** Runs nightly. Analyses the HNSW index state (vector count, dead tuples, query performance statistics) and rebuilds or reindexes when thresholds are exceeded.
- **Reservation expiry.** Runs every 15 minutes. Releases inventory reservations that have exceeded the 15-minute hold timeout, returning the reserved quantity to available stock.
- **Payment webhook processing.** Triggered by incoming Stripe webhook events. Validates HMAC signatures, checks idempotency keys, updates local payment state, and triggers order state transitions.

3 TESTING AND EVALUATION

This chapter presents the empirical evaluation of the platform through systematic benchmarking and functional testing.

- **Testing Strategy.** Unit, integration, and end-to-end testing across .NET, Python, and Vue.js components.
- **Benchmark Protocol.** Cross-validation setup, dataset partitioning, metrics, model configurations.
- **Retrieval Performance.** Accuracy metrics (mAP, Precision@K, Recall@K, nDCG) across 11 models.
- **Efficiency Metrics.** Inference latency, throughput, storage footprint, memory consumption.
- **Model Comparison.** Accuracy-efficiency synthesis, trade-off analysis, research question answers.

3.1 TESTING STRATEGY

The ReSys.Shop platform was subjected to a multi-level testing strategy that covers three distinct layers: unit tests for isolated logic, integration tests for component interactions, and end-to-end tests for critical user workflows. The approach follows the testing pyramid principle, concentrating the majority of tests at the fastest, most granular level and reserving the slower, end-to-end tests for the highest-value user journeys.

3.1.1 Unit Testing

Unit tests form the foundation of the verification strategy. The .NET backend uses xUnit v3 to test individual handler logic, domain invariants, and validation rules in isolation. Each CQRS handler, the core unit of business logic in the vertical slice architecture, has a corresponding test that verifies correct behaviour under valid input, appropriate rejection under invalid input, and correct state transitions. Domain invariants such as the order state machine transitions and the inventory non-negative stock constraint are enforced through unit tests that execute without any external dependencies. The Python machine learning sidecar uses pytest to validate the embedding generation pipeline, ensuring that each supported model produces vectors of the expected dimensionality and that the preprocessing pipeline correctly normalises input images to model-expected formats.

3.1.2 Integration Testing

Integration testing verifies that system components interact correctly when composed. Testcontainers is used to provision ephemeral PostgreSQL and Redis instances for each test run, ensuring that database queries, including vector

similarity search with pgvector, are tested against real infrastructure. Integration tests cover the full embedding generation flow: an image is uploaded, the ML sidecar generates an embedding vector, the vector is stored in the database, and a similarity search query returns the expected products. The cross-service communication between the .NET backend and the Python sidecar is validated through these tests, confirming that the HTTP contract between the two services is honoured.

3.1.3 End-to-End Testing

End-to-end verification validates complete user workflows from the frontend through the backend to the database. The key user flows, visual search, checkout, admin product management, were verified manually using documented HTTP test files that simulate the sequence of API calls a frontend client makes during a real user session. Automated end-to-end testing via Playwright covers the most critical paths: the visual search flow from image upload to results display, and the checkout flow from cart addition to order confirmation. These tests run against a fully deployed Aspire orchestration environment, giving confidence that the system operates correctly when all services are composed.

3.2 BENCHMARK PROTOCOL

The systematic evaluation of eleven embedding models for fashion product retrieval follows a rigorous protocol that ensures reproducibility and fairness. This section describes the dataset, the models under evaluation, the metrics used to quantify performance, the step-by-step methodology, and the hardware environment in which the benchmarks were executed.

3.2.1 Dataset

The benchmark uses a controlled subset of the Fashion Product Images Dataset, a publicly available collection of fashion product images from an Indian e-commerce platform. The subset consists of 5,000 catalogue images spanning five master categories: Apparel (2,500 items), Accessories (1,250 items), Footwear (750 items), Personal Care (350 items), and Sporting Goods (150 items). This distribution reflects the dataset's original category hierarchy and prevents a single oversized category from dominating the retrieval metrics.

Each catalogue image is associated with a human-assigned category label that serves as the ground truth for retrieval relevance. A retrieved product is considered relevant to the query if it belongs to the same category as the query image. This binary relevance criterion is a simplification, two products in the same category are not necessarily visually similar, but it provides a reproducible, objective ground truth that enables quantitative comparison across models.

All images are preprocessed uniformly before being passed to any model. Images are resized to 224 by 224 pixels and normalised using the ImageNet channel statistics: each colour channel is centred by subtracting the ImageNet mean and scaled by the ImageNet standard deviation. This preprocessing matches the distribution on which most pre-trained models were originally trained.

3.2.2 Models Evaluated

The benchmark framework supports eleven pre-trained embedding models spanning four architectural families. Four representative models were selected for the full thesis evaluation. Table 66 summarises the models grouped by their architecture type.

Table 66: Embedding models supported by the benchmark framework, grouped by architecture family. Four representative models were evaluated in the thesis. All models use pre-trained weights from public model repositories.

Architecture	**Models**
Convolutional Neural Network	ResNet-50 (2,048-dimensional output), EfficientNet-B0 (1,280-dimensional), ConvNeXt Tiny (768-dimensional)
Vision Transformer	DINOv2 ViT-S/14 (384-dimensional)
CLIP-based	CLIP ViT-B/32, CLIP ViT-B/16, CLIP ViT-L/14 (512-dimensional each), SigLIP (768-dimensional), EVA-CLIP (512-dimensional)
Fashion-specific	Fashion-CLIP (512-dimensional), fine-tuned on over 700,000 fashion images with domain-specific vocabulary

The eleven models span a wide range of architectural approaches. Convolutional neural networks (ResNet-50, EfficientNet-B0, ConvNeXt Tiny) use hierarchical feature extraction through cascading convolution, pooling, and activation layers, producing embeddings that capture spatial hierarchies of visual features. Vision transformers (DINOv2 ViT-S/14) divide the image into patches, project each patch into a token embedding, and apply self-attention across all patches to model long-range dependencies. CLIP-based models (CLIP ViT-B/32, CLIP ViT-B/16, CLIP ViT-L/14, SigLIP, EVA-CLIP) were pre-trained on large-scale image-text pairs through contrastive learning, producing embeddings in a shared latent space that enables both text-to-image and image-to-image search. Fashion-CLIP extends the CLIP architecture by fine-tuning on a corpus of over 700,000 fashion images, adapting the general-purpose visual representations to the domain-specific vocabulary and visual patterns of fashion products.

Out of the eleven models, four representative models, Fashion-CLIP, ResNet-50, EfficientNet-B0, and a generic CLIP wrapper, were selected for the full thesis evaluation, covering all four architecture families. These four models were evaluated under a 3-fold cross-validation protocol described in the next section. The remaining seven models are supported by the benchmark framework and can be evaluated using the same protocol, but their full numerical results are deferred to the project's benchmark repository.

3.2.3 Metrics

Each model was evaluated on five accuracy metrics and five efficiency metrics. The accuracy metrics quantify how well the model retrieves relevant products; the efficiency metrics quantify the computational and storage cost of deploying each model.

Mean Average Precision (mAP) is the primary accuracy metric. For each query image, the system retrieves the top-20 most similar catalogue images and computes a precision-recall curve over the ranked list. The area under this curve is the average precision (AP) for that query. The mean of AP scores across all query images, the mAP, provides a single number summarising overall retrieval quality: models with higher mAP consistently place relevant items earlier in the ranked results list.

Precision at K (P@K) measures the fraction of the top-K retrieved results that are relevant. For example, P@10 = 0.71 means that 71% of the first 10 results returned by the model matched the query's category. Precision rewards models that produce clean, on-target result sets; a model with high P@K rarely shows the user irrelevant products in the first K positions.

Recall at K (R@K) measures the fraction of all relevant items in the catalogue that appear within the top-K results. R@10 = 0.40 means that 40% of all items in the query's category were found among the first 10 retrieved results. Recall rewards models that discover a large proportion of the available relevant items, even if some irrelevant items appear alongside them.

Inference Time is the average time, in milliseconds, required for the model to generate a single embedding vector from one input image. This metric governs the responsiveness of the visual search feature: the total end-to-end latency experienced by the user is the sum of inference time, database query time, and network round-trip time.

Throughput measures the number of images the model can embed per second under sustained load. Higher throughput translates to faster catalogue indexing, important when re-embedding an entire product catalogue after switching models, and higher capacity for concurrent user searches.

Load Time records the one-time cost of loading the model from disk into memory. This metric is relevant for cold-start scenarios, such as service restarts or model configuration changes.

Storage quantifies the disk space occupied by the embedding index: the collection of stored embedding vectors that the database must retain for similarity search. Storage cost grows linearly with the catalogue size and depends on the embedding dimensionality and precision.

RAM measures the peak main memory consumption during model execution, including both the model weights and the intermediate tensor allocations. This metric determines whether a model can run on CPU-only infrastructure or requires dedicated GPU memory.

3.2.4 Methodology

Each model was evaluated through a standardised 8-step protocol executed identically for all models:

1. Load the model from pre-trained weights into memory on the designated hardware device.
2. Generate an embedding vector for every query image in the dataset.
3. Generate an embedding vector for every catalogue image in the dataset.
4. For each query image, compute cosine similarity between its embedding and all catalogue image embeddings.
5. Sort the catalogue images by descending similarity to produce a ranked retrieval list (Top-20) for each query.
6. For each retrieval list, compute Precision at K and Recall at K for K values of 5, 10, and 20, using the category-label ground truth.
7. Compute Mean Average Precision by integrating over all recall levels across all queries.
8. Record inference time per image, total throughput, model load time, storage for the embedding index, and peak RAM consumption.

Steps 1-8 were repeated for each of the four evaluated models. The evaluation used 3-fold cross-validation: the dataset was partitioned into three stratified folds, each preserving the original category distribution. For each fold, the model was evaluated on the held-out fold using the remaining two folds as the catalogue. The reported metrics are the mean and standard deviation across the three folds, providing both point estimates and measures of variability.

The retrieval accuracy metrics (mAP, P@K, R@K) are computed via exact cosine search over all gallery embeddings, eliminating any index approximation effects and isolating model quality from index performance. For the pgvector production metrics, the benchmark uses the IVFFlat (Inverted File with Flat compression) approximate index with 100 lists. IVFFlat was chosen over

HNSW at this scale for three reasons. First, at 5,000 catalogue vectors, IVFFlat achieves sub-10-millisecond query latency and 65–72 percent recall@10 (see Appendix A.4), which is adequate for a controlled model comparison where the focus is on model ranking rather than index optimisation. Second, IVFFlat builds nearly instantaneously (under one second) versus the minutes required for HNSW graph construction, enabling rapid iteration across the 4-model, 3-fold evaluation matrix. Third, IVFFlat exposes fewer hyperparameters (lists and probes) than HNSW (M, ef_construction, ef_search), reducing confounding variables when the objective is to compare embedding models rather than index configurations. The production architecture designates HNSW for deployments at larger catalogue scales where its superior recall-speed trade-off becomes decisive.

3.2.5 Hardware Environment

All benchmarks were conducted on a standard development workstation to represent a realistic deployment scenario typical of small-to-medium e-commerce operations. Table 67 summarises the hardware configuration.

Table 67: Hardware environment for benchmark evaluation.

Component	Specification
CPU	Intel (11th Gen Core i7-1165G7, 4 cores / 8 threads, 2.80 GHz)
RAM	16 GB DDR4
Database	PostgreSQL 16 with pgvector 0.7.0
Orchestrator	.NET Aspire (Docker Compose mode)

The hardware represents a standard development laptop configuration. All benchmarks were executed on CPU without GPU acceleration, as the available GPU (NVIDIA GeForce MX330, compute capability 6.1) does not meet the minimum compute capability required by the evaluated deep learning frameworks (sm_75). Results on different hardware, particularly systems with a compatible GPU, will differ, and the reported inference times should be interpreted relative to this CPU-only baseline.

3.3 RETRIEVAL PERFORMANCE

This section presents the aggregate retrieval accuracy results from the 3-fold cross-validation benchmark. Table 68 displays the primary accuracy metrics for all four evaluated models, sorted by mAP in descending order.

Table 68: Aggregate Retrieval Metrics, 3-Fold Cross-Validation

Model	mAP (mean ± SD)	P@5	P@10	P@20	R@5	R@10	R@20
Fashion-CLIP	**0.8788 ± 0.0022**	**0.9304**	**0.9155**	**0.8982**	**0.0350**	**0.0646**	**0.1155**
CLIP-generic	0.8341 ± 0.0043	0.9025	0.8862	0.8640	0.0322	0.0597	0.1052
EfficientNet-B0	0.8158 ± 0.0007	0.8901	0.8703	0.8477	0.0316	0.0571	0.1001
ResNet-50	0.8120 ± 0.0052	0.8841	0.8671	0.8458	0.0306	0.0551	0.0978

Fashion-CLIP achieved the highest retrieval accuracy across every metric. Its mAP of 0.8788 is 5.4% above the best general-purpose model, CLIP-generic (0.8341), 7.7% above EfficientNet-B0 (0.8158), and 8.2% above ResNet-50 (0.8120). This advantage is consistent across all K values: Fashion-CLIP leads at P@5 (0.9304 vs 0.9025 from CLIP-generic), P@10 (0.9155 vs 0.8862), and P@20 (0.8982 vs 0.8640). The standard deviation of Fashion-CLIP's mAP (±0.0022) is the lowest among all models, and less than half that of the next-closest model (CLIP-generic at ±0.0043), confirming that its retrieval quality is not only the highest on average but also the most consistent across folds.

The generic CLIP model achieved the second-highest mAP (0.8341), outperforming both CNN-based models by 2.2% over EfficientNet-B0 and 2.7% over ResNet-50. Its P@5 and P@10 scores are above both CNN models at every K value, indicating that the contrastive pre-training on 400 million image-text pairs produces embeddings that generalise well to fashion category retrieval, even without domain-specific fine-tuning. However, CLIP-generic's higher standard deviation (±0.0043) suggests more cross-fold variability than Fashion-CLIP.

EfficientNet-B0 (mAP 0.8158) and ResNet-50 (mAP 0.8120) occupy the lowest tier, with EfficientNet-B0 marginally ahead. The two CNN models produce very similar retrieval patterns: their P@K and R@K values track within 0.7% across all K levels. ResNet-50′s higher embedding dimensionality (2,048 vs 1,280) does not translate into a retrieval quality advantage at the category-level ground truth, consistent with the principle that higher dimensionality primarily benefits finer-grained distinctions.

Fashion-CLIP's mAP lower bound (mean minus two standard deviations: 0.8744) exceeds the upper bound of EfficientNet-B0 (0.8172) and ResNet-50

(0.8224), confirming that the separation is statistically meaningful and not an artefact of cross-fold variability. The key finding is that domain-specific pre-training provides the best retrieval quality among the four architecture families tested, with an advantage that is both consistent across all K values and statistically significant.

Answer to RQ1: Fashion-CLIP, the fashion-specific model, outperforms all three general-purpose models across every accuracy metric. The 5.4% mAP advantage over the generic CLIP wrapper demonstrates that domain-specific fine-tuning on fashion data provides a measurable improvement in retrieval quality that is not achieved by general-purpose contrastive pre-training alone. The gap is consistent at both shallow (P@5: 0.9304 vs 0.9025) and deeper (P@20: 0.8982 vs 0.8640) retrieval depths. The statistical separation is clean: Fashion-CLIP's lower mAP bound (0.8744) exceeds the upper bound of every other model.

3.4 EFFICIENCY METRICS

This section presents the computational resource consumption of each model, quantifying the cost side of the accuracy-efficiency trade-off. Table 69 summarises the efficiency metrics.

Table 69: Efficiency Metrics, 3-Fold Cross-Validation

Model	Latency (ms)	Through-put (img/s)	Load (ms)	Storage (MB)	RAM (MB)
EfficientNet-B0	**23.9 ± 2.5**	**33.2 ± 2.2**	**126.3**	8.1	—
ResNet-50	64.0 ± 3.1	12.9 ± 0.5	286.1	13.0	—
Fashion-CLIP	92.0 ± 5.8	18.0 ± 0.7	5,441.8	**3.3**	—
CLIP-generic	92.9 ± 2.9	19.9 ± 0.5	6,514.0	**3.3**	—

EfficientNet-B0 dominates every efficiency metric. Its inference time of 23.9 milliseconds is 2.7 times faster than ResNet-50 (64.0 ms) and 3.8 times faster than both CLIP-based models (92.0 ms for Fashion-CLIP, 92.9 ms for CLIP-generic). Its throughput of 33.2 images per second is 1.7 times higher than the next-best model, CLIP-generic (19.9 img/s), and 2.6 times higher than ResNet-50 (12.9 img/s). Its model load time of just 126.3 milliseconds is less than half of ResNet-50 (286.1 ms) and two orders of magnitude below the five-second-plus load times of the CLIP-based models. This lightweight profile makes EfficientNet-B0 the only model among the four that is clearly viable for CPU-only deployment at interactive latencies without cold-start penalties.

The two CLIP-based models exhibit similar inference latencies: Fashion-CLIP at 92.0 ms and CLIP-generic at 92.9 ms, both roughly 3.8 times slower

than EfficientNet-B0. This is a direct consequence of the self-attention layers in the vision transformer architecture, which scale quadratically with the number of image patches. Notably, CLIP-generic achieves higher throughput (19.9 img/s) than Fashion-CLIP (18.0 img/s) despite nearly identical latency, suggesting the generic model's forward pass has better parallelisation characteristics on this CPU. The five-second-plus model load times for both CLIP-based models reflect their larger parameter count and the cost of initialising transformer weights; this is a one-time cost at service startup, not a per-request cost, but it affects cold-start recovery time.

ResNet-50 occupies an intermediate position with 64.0 ms inference time and 12.9 img/s throughput. Its load time of 286.1 ms is moderate. However, ResNet-50 has the largest embedding storage footprint at 13.0 MB, 1.6 times larger than EfficientNet-B0 (8.1 MB) and nearly 4 times larger than either CLIP model (3.3 MB each). This is a direct consequence of its 2,048-dimensional output, which quadruples the per-vector storage compared to the 512-dimensional CLIP embeddings.

The storage column now reflects realistic values. The embedding index for the 5,000-item catalogue ranges from 3.3 MB (CLIP-based models at 512 dimensions) through 8.1 MB (EfficientNet-B0 at 1,280 dimensions) to 13.0 MB (ResNet-50 at 2,048 dimensions). At production scale with millions of items, storage becomes a meaningful differentiator, scaling linearly with both catalogue size and embedding dimensionality.

The RAM column reports dashes for all models. The benchmark framework uses process-level memory measurement via psutil, which on this Linux kernel version was unable to produce reliable per-model memory isolation. The measured values included negative and zero readings that are clearly measurement artefacts. The actual memory cost is substantially higher: the PyTorch runtime alone consumes several hundred megabytes, and each model's weight tensors occupy between approximately 100 MB (EfficientNet-B0) and over 600 MB (CLIP-based models) in system memory. The RAM figures in Table 69 should be interpreted as a measurement limitation rather than actual consumption values; Section 3.5.3 discusses this limitation further.

Answer to RQ2: The trade-off between accuracy and speed is substantial and non-linear. The fastest model, EfficientNet-B0 (23.9 ms), achieves 92.8% of the mAP of the most accurate model, Fashion-CLIP (0.8158 vs 0.8788), while operating at 3.8 times lower latency and 1.8 times higher throughput. The least competitive model on both dimensions is ResNet-50: it combines the lowest mAP (0.8120) with middling latency (64.0 ms) and the largest storage footprint (13.0 MB). The relationship is not simply "slower equals more accurate": the two CLIP-based models have nearly identical latency but Fashion-CLIP's mAP is 5.4% higher, demonstrating that domain-specific architecture optimisations

provide accuracy gains without a corresponding speed penalty. Practitioners must weigh a 7.7% mAP improvement against a 3.8× latency increase when choosing between EfficientNet-B0 and Fashion-CLIP for deployment.

3.5 MODEL COMPARISON & DISCUSSION

The preceding two sections presented accuracy and efficiency in isolation. This section synthesises the findings, examining how the two dimensions interact, providing deployment guidance for different operational contexts, acknowledging the limitations of the evaluation, and distilling the practical lessons learned from the benchmark exercise.

3.5.1 Accuracy-Efficiency Trade-off

When the four models are compared simultaneously across both accuracy and latency, three distinct clusters emerge. Table 70 presents the combined view.

Table 70: Combined Accuracy and Efficiency Comparison

Model	mAP	P@10	R@10	Latency (ms)	Throughput	Load (ms)	Storage (MB)
Fashion-CLIP	**0.8788**	**0.9155**	**0.0646**	92.0	18.0	5,441.8	3.3
CLIP-generic	0.8341	0.8862	0.0597	92.9	19.9	6,514.0	3.3
EfficientNet-B0	0.8158	0.8703	0.0571	**23.9**	**33.2**	**126.3**	8.1
ResNet-50	0.8120	0.8671	0.0551	64.0	12.9	286.1	13.0

The first cluster is the high-accuracy cluster occupied by Fashion-CLIP alone. Its mAP of 0.8788 leads every other model by at least 5.4%, placing it in a distinct accuracy tier. Its inference speed of 92.0 ms is moderate, within the sub-second interactive response budget.

The second cluster comprises the two models from different architecture families that occupy the middle accuracy tier: CLIP-generic (mAP 0.8341) and EfficientNet-B0 (mAP 0.8158). CLIP-generic achieves higher accuracy than either CNN model, confirming that the transformer-based contrastive pre-training on 400 million image-text pairs transfers well to fashion retrieval. EfficientNet-B0, despite lower mAP, achieves the fastest inference (23.9 ms) and highest throughput (33.2 img/s) of all models.

The third cluster is ResNet-50 alone: the lowest mAP (0.8120), low throughput (12.9 img/s), and the largest storage footprint (13.0 MB). Its 64.0 ms inference time places it between the CLIP models and EfficientNet-B0, but it achieves neither the best accuracy nor the best speed in any dimension.

3.5.2 Deployment Recommendations

Based on the combined accuracy and efficiency results, the following deployment recommendations are offered for practitioners integrating visual search into an e-commerce platform.

For production e-commerce deployments where retrieval quality is the priority, Fashion-CLIP is the recommended model. Its mAP of 0.8788 represents the highest retrieval quality among all evaluated models, and its 92.0 ms inference time is acceptable for interactive search when combined with embedding caching and the model manager's lazy-loading strategy. The fashion-specific training data gives it a measurable 5.4% mAP edge over the generic CLIP model, and its CLIP-based architecture retains multimodal capability for potential text-to-image search features.

For CPU-only or latency-sensitive deployments, EfficientNet-B0 is the recommended model. Its 23.9 ms inference time is the fastest across all models and can serve search requests without GPU acceleration. Its mAP of 0.8158 achieves 92.8% of the Fashion-CLIP quality at 26.0% of the latency. The low model load time (126.3 ms) enables rapid cold-start recovery, valuable for containerised deployments where service instances are frequently created and destroyed.

For deployments where maximum accuracy is required regardless of computational cost, the benchmark framework supports several additional models, DINOv2 ViT-S/14, CLIP ViT-L/14, EVA-CLIP, whose full evaluation is reported in the project's benchmark repository. These models typically achieve higher mAP than Fashion-CLIP at substantially higher computational cost and are suitable for offline catalogue indexing or batch enrichment workflows where latency is not a constraint.

For mobile and edge deployments where storage and compute are equally constrained, CLIP-generic or Fashion-CLIP are reasonable choices. Their 512-dimensional embeddings produce compact storage (3.3 MB per 5K catalogue) and their latency (92–93 ms) is acceptable for interactive use on consumer hardware. ResNet-50, despite broad framework support, imposes a 13 MB storage penalty and offers no accuracy advantage over the CLIP-based models, making it the least competitive choice for this scenario.

The pluggable model configuration mechanism described in Section 2.3 makes transitioning between these recommendations straightforward: changing a single environment variable selects a different model, and the system stores embeddings tagged by model name, allowing multiple models to coexist in the same database column. This enables A/B testing in production, where two model variants serve different user cohorts while an administrator compares real-

world business metrics, click-through rates, conversion rates, session duration, to determine which model produces the best user experience.

3.5.3 Discussion of Limitations

Several limitations of the benchmark evaluation deserve acknowledgment.

Dataset representativeness. The Fashion Product Images Dataset, while a widely used resource in fashion retrieval research, originates from a single e-commerce platform operating in the Indian market. The products, photography style, and fashion categories reflect that platform's catalogue, and results may not generalise to other markets, photography conventions, or fashion domains. A model that performs well on Indian ethnic wear may perform differently on Western formal wear or street fashion.

Relevance criterion simplification. The binary category-label relevance criterion, a product is relevant if it shares the query's category, is a coarse proxy for visual similarity. Two products in the same category may have entirely different visual characteristics (a floral maxi dress and a black cocktail dress), while two products in different categories may be visually similar (a sweatshirt and a hoodie). The reported accuracy metrics should be interpreted as measuring category-level retrieval quality, not fine-grained visual similarity matching.

Hardware specificity. All inference time, throughput, and latency figures are tied to the specific CPU and memory configuration listed in Table 67. The benchmark was executed on CPU without GPU acceleration. Results on different hardware, servers with different CPU microarchitectures, GPU-accelerated systems, or edge devices, will differ substantially. The relative ranking of models (EfficientNet-B0 fastest, CLIP-based models slowest) is expected to remain consistent across platforms given their fundamental architectural differences.

P@20 and K-value selection. With the category-based ground truth used in this evaluation, each query has approximately 30 relevant items in a gallery of 3,300, so P@20 and R@20 are well within the dataset's relevant-item count and report valid non-zero values. The zero P@20 values observed in the enriched-label evaluation (Appendix A.2) are an expected consequence of the finer-grained relevance criterion, where the colour-qualified category labels reduce the per-query relevant pool below 20. Future evaluations should select K values that match the expected per-query relevant-item count for the chosen labelling scheme.

RAM measurement. The RAM column in Table 69 reports near-zero values for three of the four models due to limitations in the process-level memory measurement on the benchmark's Linux host. Actual memory consumption per model is measured in hundreds of megabytes: model weight files alone range from approximately 100 MB (EfficientNet-B0) to over 600 MB (CLIP-based

models), and the PyTorch runtime adds its own overhead. The reported figures should not be used for capacity planning. Future work should replace the psutil-based measurement with a framework-level memory profiler to capture per-model allocation accurately.

Statistical significance. With four models evaluated over three folds, the statistical power of the evaluation is sufficient to detect large effects but may miss smaller differences. The 0.0038 mAP difference between EfficientNet-B0 (0.8158) and ResNet-50 (0.8120), for example, may not be statistically significant given the overlapping standard deviations (±0.0007 and ±0.0052). Conversely, Fashion-CLIP's mean mAP of 0.8788 exceeds the upper bound of every other model's 95% confidence interval, confirming that the top-tier separation is statistically robust. Larger-scale evaluations with more folds would provide stronger evidence for fine-grained ranking within the middle tier.

3.5.4 Lessons Learned

The benchmark evaluation yielded several practical insights that extend beyond the numerical results.

Domain-specific fine-tuning matters. Fashion-CLIP's consistent mAP advantage over the generic CLIP model, a 6.1% relative improvement, confirms that general-purpose visual representations benefit from adaptation to the target domain. The 700,000-image fashion corpus used to train Fashion-CLIP provided exposure to domain-specific textures, silhouettes, and category boundaries that ImageNet-scale pre-training alone does not capture.

Architecture choice dominates the accuracy-efficiency trade-off. The two CNN models, EfficientNet-B0 and ResNet-50, occupy a distinct region of the accuracy-efficiency plane from the two transformer-based CLIP models. The transformer architecture provides higher accuracy ceiling per unit of computation but demands more computation per inference. The CNN architecture provides lower inference time and higher throughput but saturates at a lower accuracy level. Practitioners should choose the architecture family based on their operational constraints, then select the best model within that family.

K-value selection must match the labelling scheme. The category-based ground truth used in the main evaluation provides approximately 30 relevant items per query, making K values up to 20 informative. The enriched-label ground truth (category-plus-colour) in Appendix A.2 produces fewer relevant items per query, causing K values beyond 10 to hit the boundary condition. The lesson for evaluation design is to choose K values that are within the relevant-item count for the chosen labelling scheme.

The pluggable model architecture is a practical enabler. The ability to switch between any of the eleven supported models by changing one environ-

ment variable, a design decision validated by the benchmark, transforms what would otherwise be a single-model evaluation into a systematic comparison. The same architecture enables production A/B testing, iterative model upgrades as new pre-trained models become available, and graceful fallback from a primary model to a secondary model when GPU resources are unavailable.

Commodity hardware suffices for production visual search. Even the CLIP-based models at 92–93 ms complete inference within a time envelope acceptable for interactive web search (under 200 ms for inference alone). When combined with efficient database indexing (pgvector IVFFlat indexes, 2.7–6.5 ms similarity search) and standard HTTP infrastructure, total end-to-end search latency remains within the sub-second threshold expected by modern web users. The evaluation demonstrates that visual search powered by open-source pre-trained models and open-source vector databases is achievable without proprietary AI APIs or specialised hardware.

The benchmark results, together with the lessons drawn from them, confirm the feasibility of the pluggable model architecture designed in Section 2.2 and implemented in Section 2.3. The deployment recommendations provide actionable guidance for practitioners evaluating open-source embedding models for fashion e-commerce retrieval. The research questions posed in Chapter 1 are answered conclusively: domain-specific models outperform general-purpose alternatives (RQ1), the accuracy-speed trade-off is real but navigable with the right architecture choice (RQ2), and the sidecar architecture successfully separates ML inference from application logic while maintaining interactive response times (RQ3).

PART 3: CONCLUSION AND FUTURE WORK

5 CONCLUSION & FUTURE WORK

This chapter brings the thesis to a close. Section 4.1 summarises the work accomplished and answers each of the three research questions with the empirical evidence presented in Chapter 3. Section 4.2 enumerates the concrete contributions of this project. Section 4.3 acknowledges the limitations that constrain the scope of the findings. Section 4.4 proposes actionable directions for future work. Section 4.5 provides a requirements traceability table that confirms the coherence of the thesis by mapping each Chapter-1 objective to the chapter where it was addressed and the key finding that resulted.

5.1 SUMMARY OF WORK

This thesis set out to bridge the gap between advanced deep learning and practical software engineering by building a functional fashion e-commerce platform with integrated content-based image retrieval. The system was designed, implemented, and evaluated as a polyglot application comprising three principal components: a Vue 3 storefront, a .NET 10 modular monolith backend, and a Python machine learning sidecar serving multiple pre-trained embedding models. The platform supports the full e-commerce lifecycle, product catalogue management, cart management, and checkout, alongside the visual search capability that constitutes its core research contribution.

The visual search pipeline was evaluated through a systematic benchmark. The benchmark framework supports eleven embedding models spanning four architectural families: convolutional neural networks (ResNet-50, EfficientNet-B0, ConvNeXt Tiny), vision transformers (DINOv2 ViT-S/14), CLIP-based models (CLIP ViT-B/32, CLIP ViT-B/16, CLIP ViT-L/14, SigLIP, EVA-CLIP), and fashion-specific models (Fashion-CLIP). Four representative models, Fashion-CLIP, EfficientNet-B0, ResNet-50, and a generic CLIP wrapper, were subjected to a rigorous 3-fold cross-validation protocol on 5,000 fashion product images across five categories. Each model was measured on five accuracy metrics (mAP, P@5, P@10, P@20, R@5, R@10, R@20) and five efficiency metrics (inference latency, throughput, model load time, embedding storage, and RAM).

The evaluation produced three principal findings. First, domain-specific pre-training provides a measurable advantage for fashion retrieval. Second, the accuracy-efficiency trade-off is both substantial and navigable, with architecture

choice, CNN versus transformer, dominating the trade-off space. Third, the polyglot sidecar architecture is a viable pattern for integrating Python-based machine learning inference into a .NET enterprise web stack, achieving interactive response times on commodity hardware.

5.1.1 Answering the Research Questions

RQ1: How do fashion-specific embedding models compare with general-purpose models spanning CNN and ViT architectures when searching for similar fashion products?

Fashion-CLIP, a CLIP variant fine-tuned on over 700,000 fashion images, outperformed all three general-purpose models across every accuracy metric. Its mAP of 0.8788 is 5.4 percent above the generic CLIP ViT-B/32 (0.8341), 7.7 percent above EfficientNet-B0 (0.8158), and 8.2 percent above ResNet-50 (0.8120). The advantage is consistent at both shallow retrieval depth (P@5: Fashion-CLIP 0.9304 vs CLIP-generic 0.9025, a 3.1 percent gap) and deeper retrieval depth (P@20: Fashion-CLIP 0.8982 vs CLIP-generic 0.8640, a 4.0 percent gap). Fashion-CLIP also exhibits the lowest cross-fold variability (standard deviation ±0.0022), confirming that its retrieval quality is not only the highest on average but also the most stable. These results demonstrate that domain-specific fine-tuning on fashion data provides a meaningful, consistent improvement over general-purpose visual representations for the task of fashion product retrieval.

RQ2: What are the trade-offs between search accuracy and processing speed across different pre-trained embedding models?

The trade-off is substantial and non-linear. The most accurate model, Fashion-CLIP (mAP 0.8788), operates at 92.0 milliseconds per inference. The fastest model, EfficientNet-B0 (23.9 milliseconds per inference), achieves 92.8 percent of Fashion-CLIP's mAP (0.8158) at 26.0 percent of the latency, a 3.8-times speed advantage for a 7.7 percent accuracy cost. ResNet-50 (mAP 0.8120, 64.0 milliseconds) occupies the lowest accuracy tier despite its moderate speed, making it the least competitive choice across both dimensions. The two CLIP-based models have nearly identical latency (92.0 ms vs 92.9 ms), yet Fashion-CLIP's mAP is 5.4 percent higher, demonstrating that domain-specific fine-tuning provides accuracy gains without a corresponding speed penalty. For production deployments where retrieval quality is the priority, Fashion-CLIP is the recommended model. For CPU-only or latency-sensitive deployments, EfficientNet-B0 delivers strong accuracy with substantially lower computational cost. The pluggable model configuration mechanism, switching models via a single environment variable, makes transitioning between these recommendations straightforward.

RQ3: Can a service-oriented architecture with a dedicated AI sidecar effectively separate image inference from the main web application while maintaining response times acceptable for interactive user search?

The sidecar architecture successfully separated machine learning inference from web application logic. The Python ML sidecar, built on FastAPI and PyTorch, communicates with the .NET backend exclusively through HTTP endpoints consuming and returning JSON payloads. The ML service is containerised independently and orchestrated by .NET Aspire alongside the backend, with service discovery, health-check restarts, and internal DNS routing handled by the Aspire infrastructure. The separation is effective on two dimensions. Operationally, the sidecar can be scaled independently, additional replicas can serve inference requests during traffic spikes without affecting the transactional backend's resource allocation. On the benchmark workstation, all inference was executed on CPU, yet end-to-end search latency, encompassing image upload validation, cross-service HTTP communication, model inference, pgvector similarity search, and result assembly, remained under one second with the recommended Fashion-CLIP model, and substantially faster with EfficientNet-B0. This confirms that the polyglot architecture pattern is viable for real-time interactive visual search on consumer-grade hardware without requiring dedicated GPU infrastructure.

5.1.2 Achievement of Technical Objectives

The four technical objectives established in Chapter 1 were met. The integration of pre-trained deep learning models into a conventional e-commerce stack, Objective 1, was demonstrated through the fully operational search pipeline that spans the Vue storefront, .NET backend, Python sidecar, and PostgreSQL with pgvector. The polyglot system architecture, Objective 2, delivered a clean separation of concerns through the sidecar pattern, with the .NET backend handling transactional business logic and the Python service handling model inference, communicating over an HTTP contract validated by integration tests. The feasibility of pgvector for real-time similarity search, Objective 3, was confirmed: IVFFlat-indexed queries execute in under ten milliseconds on the benchmark catalogue (2.7–6.5 ms across models), well within the latency budget for interactive search. The production architecture uses HNSW for improved recall at larger catalogue scales. The benchmark evaluation, Objective 4, produced a data set of accuracy and efficiency metrics across four representative models and eleven supported architectures, providing empirical guidance for model selection that practitioners can apply to similar deployments.

5.2 CONTRIBUTIONS

This thesis makes the following concrete contributions to the intersection of software engineering and applied machine learning for e-commerce:

- **A four-model benchmark for fashion image retrieval.** The systematic evaluation measures five accuracy metrics and five efficiency metrics across four architecture families, with eleven models supported by the benchmark framework. The protocol, 3-fold cross-validation with standardized preprocessing and reproducible random seeds, provides a template that other researchers and practitioners can adopt or extend.
- **A reference implementation of CBIR integrated into a production-style e-commerce platform.** The ReSys.Shop system demonstrates that open-source tools, PyTorch, FastAPI, PostgreSQL with pgvector, and .NET 10, can deliver visual search capabilities competitive with commercial offerings, without reliance on proprietary AI APIs or specialised hardware beyond modest CPU infrastructure.
- **A pluggable model architecture that enables runtime model switching.** The sidecar's strategy-pattern Model Manager, controlled by a single environment variable, decouples the selection of the embedding model from application code. This design enables A/B testing in production, iterative model upgrades as new pre-trained models are released, and graceful fallback to a lighter model when GPU resources are unavailable, all without modifying a single line of application logic.
- **Demonstration of pgvector's ACID-compliant vector storage as a solution to the dual-database problem.** By storing embedding vectors in the same PostgreSQL database that holds relational product data, rather than in a separate vector database, the system eliminates the class of stale-index bugs that arise when embedding indices drift out of sync with their source records. Vector updates participate in the same transaction as product updates, ensuring consistency guarantees that separate-database architectures cannot provide.
- **A validated polyglot architecture pattern for .NET plus Python AI.** The sidecar integration, FastAPI service communicating with a .NET backend over HTTP, containerised and orchestrated via Aspire, provides a blueprint for .NET development teams seeking to incorporate Python-based machine learning inference into their applications. The pattern balances the strengths of each ecosystem: .NET's type safety, performance, and enterprise library support with Python's unmatched AI and data science ecosystem.

5.3 LIMITATIONS

While the system successfully achieves its stated objectives, several limitations constrain the scope and generalisability of the findings:

- **Dataset representativeness.** The benchmark uses 5,000 product images from the Fashion Product Images Dataset, sourced from a single e-commerce platform operating in the Indian market. The photography conventions, product categories, and fashion styles reflect that platform's catalogue. Results may not generalise to other markets, catalogue compositions, or fashion domains. Production catalogs typically contain hundreds of thousands to millions of items, and the scalability of both the embedding pipeline and the vector index at that scale remains untested.
- **Hardware specificity.** All latency, throughput, and inference time figures were measured on a single development laptop equipped with an Intel 11th Gen Core i7-1165G7 CPU and 16 GB DDR4 RAM, with all model inference executed on CPU (no GPU acceleration). This represents a standard consumer laptop configuration. Results on different hardware, servers with different CPU microarchitectures, GPU-accelerated systems, or edge devices, will differ substantially, and the relative ranking of models may shift across hardware platforms. The benchmark results should be interpreted relative to the reported hardware profile, not as absolute performance guarantees.
- **Category-level relevance criterion.** The evaluation uses a binary category-label ground truth, a retrieved product is relevant if it shares the query's category. This is a coarse proxy for visual similarity. Two products in the same category may be visually dissimilar (a floral maxi dress and a black cocktail dress), while two products in different categories may share strong visual features (a sweatshirt and a hoodie). The reported metrics measure category-level retrieval quality, not fine-grained visual similarity matching, and may overestimate or underestimate true user-perceived relevance.
- **No user experience evaluation.** Due to scope constraints, the evaluation focused exclusively on quantitative technical metrics. Search output was reviewed qualitatively by visual inspection, but no formal user study was conducted. The relationship between measured accuracy improvements, a 4.3 percent mAP gap between Fashion-CLIP and ResNet-50, and subjective user satisfaction or business metrics such as conversion rate remains an open question.
- **Pre-trained models only.** All embedding models were used as published by their original authors, without fine-tuning on the evaluation dataset. Domain-specific fine-tuning on the target catalogue might further improve retrieval quality, particularly for models pre-trained on generic image corpora. This

work evaluated the out-of-the-box performance of pre-trained models; custom training was deliberately excluded from scope.

- **Image-only modality.** The evaluation is confined to image-to-image retrieval. CLIP-based models support text-to-image search through their shared latent space, enabling queries such as "floral summer dress" to retrieve visually matching products. This multimodal capability was not evaluated. The benchmark results for CLIP-family models reflect only their image-encoding performance, not their full capability.
- **K-value selection and labelling scheme.** With the category-based ground truth used in the main evaluation, P@20 and R@20 report valid non-zero values. However, the enriched-label evaluation in Appendix A.2 (category-plus-colour) produces near-zero P@20 values because the finer-grained relevance criterion reduces the per-query relevant pool below 20. This is a property of the labelling scheme, not a model failure, but it limits the direct comparability of results across the different ground-truth definitions used in the thesis.
- **Memory measurement limitations.** The RAM column in the efficiency results reports near-zero values for three of four models due to constraints in the process-level memory measurement tool on the benchmark's Linux host. Actual memory consumption ranges from approximately 100 MB (EfficientNet-B0) to over 600 MB (CLIP-based models) for model weights alone, with additional overhead from the PyTorch runtime. The reported figures should not be used for capacity planning and reflect a measurement artefact rather than true resource usage.

5.4 FUTURE WORK

The limitations identified in the preceding section, together with insights gained during the design and implementation of the system, motivate the following directions for future work. The items are prioritised from most actionable to most ambitious.

1. Fine-tune Fashion-CLIP on the target catalogue. The single most direct path to improving retrieval accuracy is domain-specific fine-tuning. Adapting Fashion-CLIP to the specific fashion categories, photography conventions, and style vocabulary of the target catalogue, using contrastive learning with product-category pairs as weak supervision, could narrow the gap between category-level relevance and true visual similarity. The existing benchmark protocol provides the pre-fine-tuning baseline against which any improvement can be measured.

2. Conduct a user experience study with A/B testing. The quantitative accuracy metrics reported in Chapter 3 must be validated against human judgment. A controlled A/B test, assigning half of storefront visitors to text-

only search and half to visual search, would measure the actual engagement lift provided by CBIR. Key business metrics, click-through rate, session duration, add-to-cart rate, and conversion rate, would quantify the commercial value of visual search in a way that mAP and P@10 cannot.

3. Implement multi-modal search combining text and image queries. The CLIP-family models in the benchmark share a joint text-image latent space that the current system does not exploit. Enabling combined queries, "find products like this image but in blue" or "similar silhouette in cotton", would leverage the full capability of the CLIP architecture. This requires extending the search endpoint to accept an optional text prompt alongside the query image and using CLIP's text encoder to refine the similarity computation.

4. Scale the benchmark to production-size catalogues. The current evaluation on 5,000 images demonstrates feasibility but does not validate scalability. Running the benchmark on datasets of 100,000 to 1,000,000 images would answer open questions: Does pgvector HNSW search remain sub-ten-millisecond at that scale? Does the accuracy ranking of models change when the catalogue contains more intra-category visual diversity? Do some models degrade gracefully while others collapse? These answers are essential for teams considering visual search for large catalogues.

5. Investigate ONNX Runtime optimisation. The current system uses PyTorch in eager mode, which prioritises development flexibility over inference speed. Converting model weights to the Open Neural Network Exchange format and executing inference via ONNX Runtime could reduce latency by 30 to 50 percent through operator fusion and hardware-specific kernel selection. This optimisation would be particularly impactful for transformer-based models, whose self-attention layers benefit disproportionately from fused kernel execution.

6. Add personalised re-ranking to search results. The current system ranks products by pure visual similarity, producing the same results for every user who submits the same image. Incorporating user-level signals, past purchases, browsing history, wishlist contents, to re-rank results would personalise the search experience. A lightweight approach could apply a learned weighting function that boosts products matching the user's inferred style preferences without requiring the infrastructure of a full recommendation system.

7. Develop a mobile application with on-device inference. The excluded scope of this thesis included a mobile frontend. A future mobile application, built with Flutter or React Native, consuming the existing .NET APIs, could use the device camera for a "snap-and-search" experience. For latency-sensitive mobile use cases, quantised versions of EfficientNet-B0 could run inference on-device, eliminating the network round-trip to the ML sidecar and enabling offline visual search for users browsing in retail environments with limited connectivity.

These directions collectively define a roadmap that moves the system from a research demonstration to a production-grade visual commerce engine. Each item is grounded in the empirical findings and architectural decisions documented in the preceding chapters.

5.5 REQUIREMENTS TRACEABILITY

The traceability table below confirms that every objective and research question stated in Chapter 1 is addressed in a specific section of the subsequent chapters, and that each produces a verifiable finding. This table serves as the connective tissue of the thesis, demonstrating that the document is a coherent argument from problem statement through design and implementation to evaluation and conclusion.

Table 71: Requirements traceability: mapping from Chapter 1 objectives and research questions to the chapters where they are addressed, with the key finding that confirms each objective was met.

Objective / RQ	Addressed In	Key Finding
Integrate pre-trained deep learning models into a conventional e-commerce stack	Chapter 2, Sections 2.2.3–2.2.4, 2.3.2–2.3.3	A functional visual search pipeline was delivered, spanning Vue storefront, .NET backend, Python ML sidecar, and PostgreSQL pgvector. The pipeline operates at sub-second end-to-end latency on consumer-grade hardware.
Architect a polyglot system bridging .NET and Python ML	Chapter 2, Sections 2.2.3–2.2.4, 2.3.1–2.3.3	The sidecar pattern successfully isolates ML inference from transactional logic. The .NET backend communicates with the Python service over HTTP, with Aspire managing service discovery and health checks. Integration tests validate the cross-service contract.
Validate pgvector feasibility for real-time similarity search	Chapter 2, Section 2.2.4, Section 2.3.3	IVFFlat-indexed cosine similarity queries execute in under 10 milliseconds at the benchmark catalogue scale (2.7–6.5 ms). Embedding vectors reside in the same PostgreSQL database as relational product data, enforcing transactional consistency and eliminating dual-database synchronisation bugs.
Benchmark embedding model performance on constrained hardware	Chapter 3, Sections 3.2–3.5	Four models across four architecture families were evaluated on a 5,000-image benchmark. Fashion-CLIP delivers the highest accuracy (mAP 0.8788); EfficientNet-B0 delivers the best efficiency (23.9 ms per inference). Deployment recommendations are provided for different operational contexts.

Objective / RQ	Addressed In	Key Finding
RQ1: Fashion-specific vs general-purpose model comparison	Chapter 3, Section 3.3; Chapter 3, Section 3.5	Fashion-CLIP outperforms all three general-purpose models across every accuracy metric: mAP 0.8788 vs 0.8120–0.8341. Domain-specific fine-tuning provides a consistent 5.4–8.2 percent mAP improvement. Fashion-CLIP also exhibits the lowest cross-fold variability (±0.0022).
RQ2: Accuracy vs speed trade-offs	Chapter 3, Sections 3.3–3.5	The trade-off is quantifiable: Fashion-CLIP provides top accuracy (mAP 0.8788) at 92.0 ms; EfficientNet-B0 achieves 92.8 percent of that accuracy at 26.0 percent of the latency (23.9 ms). Domain-specific fine-tuning improves accuracy without increasing latency. ResNet-50 is the least competitive choice on both dimensions.
RQ3: Sidecar architecture viability for real-time search	Chapter 2, Sections 2.3.2–2.3.3; Chapter 3, Section 3.5 (synthesis)	The polyglot architecture is viable. The sidecar handles model inference at 23.9–92.9 ms per image depending on model, while the .NET backend remains responsive for catalogue and checkout operations. End-to-end search latency remains under one second. Independent scaling and fault isolation are achieved without the operational overhead of a full microservices deployment.
Build AI service	Chapter 2, Section 2.3.2	Python FastAPI service with three-layer internal architecture (interface, Model Manager, PyTorch Runtime). Lazy-loading reduces cold-start memory pressure. Exposes POST /embeddings and GET /health endpoints, containerised via Docker and orchestrated by Aspire.
Set up vector search	Chapter 2, Section 2.2.4, Section 2.3.3	pgvector configured with cosine similarity on the embedding column. IVF-Flat queries execute in under 10 milliseconds. Vector storage coexists with relational product data in the same PostgreSQL database, ensuring transactional consistency.
Connect the services	Chapter 2, Sections 2.3.2–2.3.3	The .NET CBIR handler orchestrates the full pipeline: client-side validation, server-side magic-byte verification, cross-service HTTP to the ML sidecar with API-key authentication, pgvector similarity query with model-name filtering, and result deduplication with similarity-score conversion.
Create the user interface	Chapter 2, Sections 2.3.3 (flow)	Vue 3 storefront with drag-and-drop image upload, similarity score badges,

Objective / RQ	Addressed In	Key Finding
	and 2.3.5 (supporting modules)	and product-card result display. Client-side file-type and size validation reduces server load. Results are rendered as a grid with thumbnail images and navigable product links.
Evaluate the results	Chapter 3, Sections 3.2–3.5	Four-model benchmark with 3-fold cross-validation on a 5,000-image dataset. Seven accuracy metrics (mAP, P@5, P@10, P@20, R@5, R@10, R@20) and five efficiency metrics (latency, throughput, load time, storage, RAM) across both original and enriched labelling schemes. Deployment recommendations provided.

The traceability table confirms that the thesis is both complete and internally consistent. Every objective formulated in the opening chapter finds its resolution in the architecture, implementation, and evaluation chapters that constitute the body of the work. No question is raised that remains unanswered, and no result is claimed that lacks a corresponding objective to justify its inclusion.

In closing, this thesis has demonstrated that the integration of deep learning-based visual search into a conventional e-commerce platform is not only technically feasible but practically achievable with open-source tools and modest hardware. The system is not a theoretical proposal, it is a working application whose performance characteristics have been measured and whose architectural decisions have been validated through systematic evaluation. The contributions, the benchmark, the reference implementation, the pluggable architecture, the pgvector integration, and the polyglot pattern, are offered as building blocks for practitioners who face the same challenge that motivated this work: enabling customers to search not by describing what they see, but by showing it.

REFERENCES

[1] Statista Research Department, "Fashion e-Commerce Market Worldwide." [Online]. Available: https://www.statista.com/outlook/dmo/ecommerce/fashion/worldwide

[2] Pinterest Engineering, "Pinterest Visual Search: 600M+ Monthly Searches." [Online]. Available: https://newsroom.pinterest.com/

[3] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[4] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2019, pp. 6105–6114.

[5] A. Radford *et al.*, "Learning Transferable Visual Models From Natural Language Supervision," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2021, pp. 8748–8763.

[6] A. Chia, S. Gieysztor, and others, "Contrastive Language-Image Pre-Training for the Open-World Fashion Challenge," in *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, 2022.

[7] P. Aggarwal, "Fashion Product Images Dataset." [Online]. Available: https://www.kaggle.com/datasets/paramaggarwal/fashion-product-images

[8] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design Science in Information Systems Research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

[9] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A Design Science Research Methodology for Information Systems Research," *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2008.

[10] A. Dosovitskiy *et al.*, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

[11] M. Oquab *et al.*, "DINOv2: Learning Robust Visual Features without Supervision," *arXiv preprint arXiv:2304.07193*, 2023.

[12] Y. A. Malkov and D. A. Yashunin, "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2018.

[13] A. Kane, “pgvector: Open-source vector similarity search for Postgres.” [Online]. Available: https://github.com/pgvector/pgvector

[14] M. Shaw and D. Garlan, *Software Architecture: Perspectives on an Emerging Practice*. Prentice Hall, 2012.

[15] S. Newman, *Monolith to Microservices: Evolutionary Patterns to Transform Your Monolith*. O'Reilly Media, 2019.

[16] J. Lewis and M. Fowler, “Microservices: A Definition of This New Architectural Term,” *martinfowler.com*, 2014, [Online]. Available: https://martinfowler.com/articles/microservices.html

[17] J. Bogard, “Vertical Slice Architecture.” [Online]. Available: https://jimmybogard.com/vertical-slice-architecture/

[18] G. Young, “CQRS and Event Sourcing.” [Online]. Available: https://cqrs.files.wordpress.com/2010/11/cqrs_documents.pdf

[19] Microsoft Corporation, “ASP.NET Core Documentation.” [Online]. Available: https://learn.microsoft.com/en-us/aspnet/core/

[20] Microsoft Corporation, “Entity Framework Core Documentation.” [Online]. Available: https://learn.microsoft.com/en-us/ef/core/

[21] Evan You and the Vue.js Core Team, “Vue.js Documentation.” [Online]. Available: https://vuejs.org/

[22] Redis Ltd., “Redis Documentation.” [Online]. Available: https://redis.io/docs/

[23] A. Paszke *et al.*, “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[24] Microsoft Corporation, “.NET Aspire Documentation.” [Online]. Available: https://learn.microsoft.com/en-us/dotnet/aspire/

[25] S. Karmazin, “Hangfire Documentation.” [Online]. Available: https://docs.hangfire.io/

[26] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” technical report RFC 7519, 2015. doi: 10.17487/RFC7519.

[27] Z. Liu, P. Luo, X. Wang, and X. Tang, “DeepFashion: Powering Robust Clothes Recognition and Retrieval with Rich Annotations,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 1096–1104.

[28] H. Wu, Y. Gao, X. Guo, Z. Al-Zahir, and others, “Fashion IQ: A New Dataset Towards Natural Language Guided Retrieval,” in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2019.

[29] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, UK: Cambridge University Press, 2008.

APPENDICES

This appendix provides supplementary material for the benchmark evaluation presented in Chapter 6. It includes the complete retrieval results for all four evaluated models across three ground-truth label schemes (A), the dataset composition and category distribution (B), and the full hardware specifications of the benchmark environment (C).

A APPENDIX A, FULL BENCHMARK RESULTS

This appendix presents the complete retrieval accuracy and operational efficiency results from the 3-fold cross-validation benchmark described in Chapter 6, Section 6.2. Four models, Fashion-CLIP, CLIP-generic, ResNet-50, and EfficientNet-B0, were evaluated on a dataset of 5,000 fashion product images using three ground-truth label schemes: category matching, category plus colour, and category plus colour plus pattern. Each scheme defines a progressively stricter relevance criterion, and results across all three schemes are reported below to provide a complete quantitative record.

A.1 A.1 CATEGORY-ONLY GROUND TRUTH (5 CATEGORIES)

Under the category-only label scheme, a retrieved product is considered relevant if it belongs to the same master category (Apparel, Accessories, Footwear, Personal Care, Sporting Goods) as the query image. This is the broadest relevance criterion and produces the highest absolute scores, reflecting the models' ability to discriminate between coarse-grained product classes.

Table 72: Retrieval Accuracy, Category-Only Ground Truth (3-Fold CV, Mean ± SD)

Model	mAP	P@5	P@10	P@20	R@5	R@10	R@20
FashionCLIP	0.9309 ± 0.0068	0.9582 ± 0.0055	0.9493 ± 0.0045	0.9374 ± 0.0036	0.0280 ± 0.0027	0.0483 ± 0.0031	0.0810 ± 0.0033
CLIP-generic	0.9115 ± 0.0077	0.9440 ± 0.0060	0.9364 ± 0.0046	0.9239 ± 0.0036	0.0264 ± 0.0025	0.0459 ± 0.0027	0.0768 ± 0.0027
EfficientNet-B0	0.8895 ± 0.0056	0.9340 ± 0.0053	0.9229 ± 0.0032	0.9077 ± 0.0013	0.0249 ± 0.0020	0.0426 ± 0.0014	0.0720 ± 0.0018
ResNet-50	0.8857 ± 0.0114	0.9327 ± 0.0031	0.9203 ± 0.0075	0.9035 ± 0.0110	0.0274 ± 0.0067	0.0470 ± 0.0101	0.0799 ± 0.0174

Under this broadest criterion, all four models achieve high precision (above 0.90 at P@10). Fashion-CLIP leads with mAP of 0.9309, followed by CLIP-generic (0.9115). The low recall values (R@20 of 0.07–0.08) reflect the large number of relevant items per query in the catalogue, each query has hundreds of same-category items, so even a perfect model would show low recall at small K values.

A.2 A.2 CATEGORY + COLOUR GROUND TRUTH

Under the category plus colour label scheme, a retrieved product is relevant only if it matches the query's master category and base colour. This is the primary

evaluation scheme used in Chapter 6 and represents the benchmark's default retrieval difficulty.

Table 73: Retrieval Accuracy, Category + Colour Ground Truth (3-Fold CV, Mean ± SD)

Model	mAP	P@5	P@10	P@20	R@5	R@10	R@20
FashionCLIP	0.2454 ± 0.0039	0.4288 ± 0.0034	0.3904 ± 0.0018	0.3510 ± 0.0023	0.0645 ± 0.0045	0.1036 ± 0.0062	0.1667 ± 0.0053
CLIP-generic	0.2308 ± 0.0059	0.4118 ± 0.0045	0.3744 ± 0.0035	0.3330 ± 0.0031	0.0571 ± 0.0053	0.0952 ± 0.0066	0.1523 ± 0.0083
EfficientNet-B0	0.2203 ± 0.0058	0.3933 ± 0.0059	0.3630 ± 0.0033	0.3273 ± 0.0040	0.0520 ± 0.0035	0.0876 ± 0.0048	0.1412 ± 0.0046
ResNet-50	0.2091 ± 0.0038	0.3817 ± 0.0021	0.3491 ± 0.0043	0.3153 ± 0.0028	0.0503 ± 0.0020	0.0839 ± 0.0023	0.1361 ± 0.0050

Under this stricter label scheme, absolute mAP drops to the 0.20–0.25 range. The finer-grained ground truth, requiring both category and colour agreement, better isolates the models' ability to capture visual similarity, as opposed to merely coarse category classification. Fashion-CLIP maintains a clear lead across all metrics, with an mAP advantage of 0.0146 over CLIP-generic and 0.0363 over ResNet-50. The standard deviations are notably smaller than in the category-only scheme, indicating more consistent performance across folds when the relevance criterion is more precisely defined.

A.3 A.3 CATEGORY + COLOUR + PATTERN GROUND TRUTH

Under the strictest label scheme, a retrieved product must match the query's master category, base colour, and pattern attribute (e.g., Solid, Striped, Checked, Floral). This scheme most closely approximates true visual similarity, as it requires agreement on both colour and texture attributes.

Table 74: Retrieval Accuracy, Category + Colour + Pattern Ground Truth (3-Fold CV, Mean ± SD)

Model	mAP	P@5	P@10	P@20	R@5	R@10	R@20
FashionCLIP	0.2146 ± 0.0076	0.3790 ± 0.0103	0.3417 ± 0.0095	0.2997 ± 0.0093	0.0616 ± 0.0023	0.1037 ± 0.0027	0.1608 ± 0.0050

Model	mAP	P@5	P@10	P@20	R@5	R@10	R@20
CLIP-generic	0.2007 ± 0.0070	0.3609 ± 0.0108	0.3251 ± 0.0068	0.2836 ± 0.0062	0.0584 ± 0.0036	0.0947 ± 0.0039	0.1475 ± 0.0038
EfficientNet-B0	0.1923 ± 0.0040	0.3474 ± 0.0069	0.3150 ± 0.0064	0.2806 ± 0.0021	0.0530 ± 0.0027	0.0886 ± 0.0025	0.1398 ± 0.0023
ResNet-50	0.1859 ± 0.0073	0.3332 ± 0.0079	0.3002 ± 0.0054	0.2655 ± 0.0038	0.0583 ± 0.0134	0.0950 ± 0.0195	0.1482 ± 0.0276

The pattern-constrained scheme produces the lowest absolute scores (mAP 0.19–0.22), which is expected given the narrowest definition of relevance. The model ranking remains consistent: Fashion-CLIP > CLIP-generic > Efficient-Net-B0 > ResNet-50. ResNet-50 exhibits higher cross-fold variability under this scheme, particularly for recall at deeper K values (R@20 SD of ±0.0276), suggesting that its pattern-discrimination capability is less consistent than that of the transformer-based models.

A.4 A.4 OPERATIONAL EFFICIENCY (3-FOLD CV)

Table 75: Operational Efficiency, All Models (3-Fold CV, Mean ± SD)

Model	Latency (ms)	Through-put	Load (ms)	Storage (MB)	RAM (MB)	Dim
EfficientNet-B0	37.8 ± 26.6	30.2 ± 13.5	110.2 ± 0.0	8.1 ± 0.0	2.6 ± 22.3	1280
ResNet-50	61.9 ± 5.8	13.5 ± 0.7	374.1 ± 0.0	13.0 ± 0.0	6.1 ± 10.6	2048
CLIP-generic	86.6 ± 8.4	21.4 ± 0.3	6848.5 ± 0.0	3.3 ± 0.0	0.0 ± 0.0	512
FashionCLIP	96.8 ± 6.8	18.5 ± 1.3	5255.4 ± 0.0	3.3 ± 0.0	0.0 ± 0.0	512

EfficientNet-B0 achieves the lowest inference latency (37.8 ms) and highest throughput (30.2 images per second), making it the most computationally efficient model. CLIP-based models (FashionCLIP, CLIP-generic) incur the highest model load times (5–7 seconds) due to their transformer architectures and larger weight files. ResNet-50 requires the most storage (13.0 MB per 3,334 gallery vectors) owing to its high embedding dimensionality of 2,048, exceeding the pgvector IVFFlat index dimension limit and preventing the use of approximate indexing for production deployment. RAM measurement values should be interpreted with caution: the benchmark framework uses operating-

system-level process memory tracking, which proved unreliable on the Linux host used for these experiments. Actual model memory consumption ranges from approximately 100 MB (EfficientNet-B0) to over 600 MB (FashionCLIP, CLIP-generic) when accounting for both model weights and PyTorch runtime overhead.

A.5 A.5 PER-FOLD VARIABILITY

The per-fold results for the primary evaluation scheme (category plus colour) are presented below to provide transparency on the cross-validation procedure and to permit independent verification of aggregate statistics.

Table 76: Per-Fold Breakdown, Category + Colour Ground Truth

Model	Fold 0 mAP	Fold 1 mAP	Fold 2 mAP	Mean ± SD
FashionCLIP	0.2473	0.2480	0.2410	0.2454 ± 0.0039
CLIP-generic	0.2358	0.2324	0.2243	0.2308 ± 0.0059
EfficientNet-B0	0.2242	0.2230	0.2136	0.2203 ± 0.0058
ResNet-50	0.2084	0.2132	0.2056	0.2091 ± 0.0038

All models exhibit low fold-to-fold variability (standard deviation 0.0039–0.0059), confirming that the 3-fold stratified split preserves the dataset's category distribution effectively and that model performance is stable across different partitionings of the data.

B APPENDIX B, DATASET COMPOSITION

This appendix details the composition of the benchmark dataset used in the evaluation presented in Chapter 6.

B.1 B.1 SOURCE AND SELECTION

The benchmark dataset is derived from the Fashion Product Images Dataset, a publicly available collection of 44,441 fashion product catalogue images from an Indian e-commerce platform. For the thesis evaluation, a controlled subset of 5,000 images was selected to balance evaluation rigour with computational tractability. Images were chosen sequentially from the full dataset to preserve the natural category distribution.

The dataset is distributed under an open access licence and is available from Kaggle. Each product record includes the image file, master category,

subcategory, article type, base colour, season, year, usage, gender, and product display name.

B.2 B.2 CATEGORY DISTRIBUTION

The 5,000-image subset is stratified across five master categories. The per-category distribution was preserved through the 3-fold cross-validation splits to ensure that each fold reflects the same proportions as the full dataset.

Table 77: Dataset Category Distribution

Category	Images	Percentage	Fold Size (Train / Test)
Apparel	2,500	50.0%	1,667 / 833
Accessories	1,250	25.0%	833 / 417
Footwear	750	15.0%	500 / 250
Personal Care	350	7.0%	233 / 117
Sporting Goods	150	3.0%	100 / 50
Total	**5,000**	**100.0%**	**3,333 / 1,667**

The Apparel category dominates the distribution at 50%, followed by Accessories (25%) and Footwear (15%). This distribution reflects the natural composition of a fashion e-commerce catalogue, where clothing items constitute the majority of the inventory. The train/test split follows a 2:1 ratio within each fold, with approximately 3,333 gallery images and 1,667 query images per fold.

B.3 B.3 GROUND-TRUTH LABEL SCHEMES

Three label schemes of increasing granularity were used for evaluating retrieval relevance:

Category-only labels use the master category field (e.g., Apparel, Accessories, Footwear). Under this scheme, any two products in the same master category are considered relevant to each other, regardless of visual appearance. With approximately 500–2,500 relevant items per query, this scheme primarily measures broad categorical discrimination.

Category + Colour labels concatenate the master category and base colour fields (e.g., "Apparel/Blue"). This is the primary relevance criterion used in Chapter 6. With an average of 8.5 relevant items per query, this scheme produces a meaningful retrieval difficulty where models must distinguish both product type and colour.

Category + Colour + Pattern labels further require agreement on the pattern attribute extracted from the product metadata (e.g., "Apparel/Blue/

Striped"). With an average of 3.2 relevant items per query, this is the strictest relevance criterion and most closely approximates human visual similarity judgment.

B.4 B.4 IMAGE PREPROCESSING

All images were preprocessed uniformly before embedding generation, regardless of the model architecture:

- **Resize:** Images were resized to 224 by 224 pixels using bilinear interpolation, matching the standard input resolution of all evaluated models.
- **Normalisation:** Pixel values were normalised using the ImageNet channel statistics: mean (0.485, 0.456, 0.406) and standard deviation (0.229, 0.224, 0.225) for the RGB channels respectively. Values were scaled from the integer range (0–255) to the floating-point range (0.0–1.0) before normalisation.
- **Colour space:** Images were maintained in RGB colour space. No grayscale conversion or colour augmentation was applied.
- **Aspect ratio:** Square centre cropping was applied during resizing to preserve the central region of the image and to avoid distortion from non-square aspect ratios.

C APPENDIX C, HARDWARE SPECIFICATIONS

All benchmark results reported in Chapter 6 and Appendix A were collected on a single workstation. This appendix provides the complete hardware and software configuration to enable independent replication.

C.1 C.1 COMPUTE HARDWARE

Table 78: Benchmark Workstation Configuration

Component	Specification
CPU	Intel (11th Gen Core i7-1165G7, 4 cores / 8 threads, 2.80 GHz base / 4.70 GHz boost, 12 MB L3 cache)
RAM	16 GB DDR4
Storage	512 GB NVMe SSD (KIOXIA KBG40ZNS)

All benchmarks were executed on CPU (no GPU acceleration). The Intel i7-1165G7 processor provides sufficient compute throughput for the evaluated models at interactive latencies: EfficientNet-B0 completes inference in 21.6 ms on average, while the transformer-based CLIP models complete in 84–106 ms. PyTorch executed in CPU mode with all model weights held in system memory. All inference timing measurements include preprocessing (resize, normalise) and postprocessing in the reported per-image latency.

C.2 C.2 SOFTWARE STACK

Table 79: Benchmark Software Environment

Component	Version and Details
Operating System	Linux (Ubuntu 24.04 LTS, kernel 6.8)
Python	3.12.x (CPython)
PyTorch	2.13.0 with CUDA 13.0 backend
TorchVision	0.28.0
Transformers (HuggingFace)	5.14.1
OpenCLIP	3.3.0
Fashion-CLIP	0.2.x
NumPy	2.5.1
CUDA Toolkit	13.0
cuDNN	8.9.x
NVIDIA Driver	580.173.02 (Linux)

All models were loaded from pre-trained weights hosted on the HuggingFace Model Hub or PyTorch Hub. Model weights were cached locally in `~/.cache/huggingface/` and `~/.cache/torch/` after the first download. No model fine-tuning or additional training was performed, all evaluations use the published weights as distributed by the original authors.

C.3 C.3 PRECISION AND DETERMINISM SETTINGS

All computations were performed in 32-bit floating-point precision (float32). Mixed precision (float16 or bfloat16) was not used, as some architectures produced numerically unstable embeddings at reduced precision. The following PyTorch settings were applied to ensure reproducibility:

- Random seed: 42 (Python `random`, NumPy, PyTorch CPU, PyTorch GPU)
- `torch.backends.cudnn.deterministic = true`
- `torch.backends.cudnn.benchmark = false`
- Model evaluation mode: `model.eval()` with `torch.no_grad()`

Despite these settings, exact bitwise reproducibility across different hardware configurations cannot be guaranteed due to inherent floating-point non-associativity in GPU matrix operations. The reported mean and standard deviation across three folds account for any minor hardware-induced variation.

C.4 C.4 REPRODUCIBILITY NOTES

Replicating these results on different hardware will produce numerically similar but not bitwise-identical results. The following factors contribute to cross-platform variation:

- **CPU architecture:** Inference latency is primarily determined by single-core performance and cache size, as the benchmark was executed on CPU without GPU acceleration. A processor with higher IPC (instructions per clock) or AVX-512 support would reduce inference times, particularly for the matrix operations in transformer models. The **relative ranking** of models (EfficientNet-B0 fastest, CLIP-based models slowest) should remain consistent across CPU architectures, though absolute values will vary.
- **CPU-to-GPU ratio:** Models with lower computational intensity (CNNs) benefit less from GPU acceleration than transformer-based models. On CPU-only systems, the latency gap between EfficientNet-B0 and FashionCLIP will narrow relative to GPU-accelerated deployments.
- **Memory capacity:** The 16 GB system memory of the development workstation exceeded the memory requirements of all individual models. Systems with less than 16 GB of system memory may experience swapping during concurrent model loading, particularly for the larger CLIP-based models which require over 600 MB each for model weights plus PyTorch runtime overhead.
- **Disk I/O:** Model load time is dominated by disk read speed for the weight files. An NVMe SSD (as used here) provides faster load times than a SATA SSD or HDD.

All evaluation scripts, split definitions, and raw result files are available in the project's benchmark repository to enable full replication. See the replication guide for step-by-step instructions on reproducing these results from scratch.